

Systematic Review

Quantization of Deep Neural Networks for Medical Image Analysis: A Systematic Review and Meta-Analysis

Edgar Fabián Rivera-Guzmán ^{1,2,*} , Luis Fernando Guerrero-Vásquez ³ 
and Vladimir Espartaco Robles-Bykbaev ^{1,*} 

¹ GI-IATa, UNESCO Chair on Support Technologies for Educational Inclusion, Universidad Politécnica Salesiana, Cuenca 010103, Ecuador

² Carrera de Ingeniería Agroindustrial, Universidad Estatal de Bolívar, Guaranda 020150, Ecuador

³ Telecommunications and Telematics Research Group (GITEL), Universidad Politécnica Salesiana, Cuenca 010103, Ecuador; lguerrero@ups.edu.ec

* Correspondence: eriverag2@est.ups.edu.ec or edgar.rivera@ueb.edu.ec (E.F.R.-G.); vrobles@ups.edu.ec (V.E.R.-B.)

Abstract

Neural network quantization has become established as a key strategy for transitioning medical imaging models from research environments to clinical devices and resource-constrained edge platforms; however, the available evidence remains fragmented and focused on highly heterogeneous use cases. This study presents a systematic review of 72 studies on quantization applied to medical images, following PRISMA guidelines, with the aim of characterizing the relationship among quantization technique, network architecture, imaging modality, and execution environment, as well as their impact on latency, memory footprint, and clinical deployment. Based on a structured variable matrix, we analyze—through tailored visualizations—usage patterns of Post-Training Quantization (PTQ), Quantization-Aware Training (QAT), mixed precision, and binary/low-bit schemes across frameworks such as PyTorch V 2.6.0, TensorFlow 2.19.0, and TensorFlow Lite, executed on server-class GPUs, edge/embedded devices, and specialized hardware. The results reveal a strong concentration of evidence in PyTorch/TensorFlow pipelines using INT8 or mixed precision on GPUs and edge platforms, contrasted with limited attention to PACS/RIS interoperability, model lifecycle management, energy consumption, cost, and regulatory traceability. We conclude that, although quantization can approximate real-time performance and reduce memory footprint, its clinical adoption remains constrained by integration challenges, model governance requirements, and the maturity of the hardware–software ecosystem.

Keywords: quantization; medical imaging; deep neural networks; edge computing



Academic Editors: Masateru Kawakubo and Tamás Haidegger

Received: 3 January 2026

Revised: 18 January 2026

Accepted: 20 January 2026

Published: 22 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Medical image analysis has undergone a profound transformation over the past decade, driven by advances in artificial intelligence (AI) and, in particular, deep learning (DL) [1–3]. Convolutional Neural Networks (CNNs) have demonstrated outstanding performance in tasks such as segmentation, classification, and detection of anatomical structures and pathologies in imaging modalities including magnetic resonance imaging (MRI), computed tomography (CT), ultrasound, and radiography [4,5]. Their ability to learn hierarchical representations and extract complex patterns has enabled accuracies comparable to, and in some clinical scenarios even exceeding, those of human specialists [6].

However, the increasing complexity of CNNs—often comprising millions of parameters and demanding substantial computational resources—poses significant challenges for their deployment across sectors such as industry, transportation, education, and healthcare, and even more so in real-world clinical environments [7–10]. Running these models on resource-constrained devices—such as standard hospital workstations, portable devices, or edge systems—is hindered by limitations in memory capacity, processing power, and energy consumption [11]. Consequently, these constraints become particularly critical in settings that require real-time processing or in regions with limited technological infrastructure, motivating the development of optimization techniques that reduce computational burden without compromising diagnostic performance.

In this context, multiple neural network optimization strategies have emerged, notably weight and activation quantization, connection pruning, model compression, and the adoption of specialized architectures such as binary neural networks or mixed-precision networks [12,13], which support clinical decision-making through medical imaging. Quantization, in particular, has proven to be one of the most effective techniques for reducing model size and accelerating inference while incurring only minimal accuracy degradation [14]. Moreover, quantization has attracted increasing attention in the literature, with medical domain applications ranging from accelerating diagnosis on portable devices to optimizing three-dimensional models for volumetric analysis [15].

Nevertheless, the current scientific evidence faces several substantial challenges, including the lack of standardized metrics for evaluating and comparing different methods, limited validation in real clinical settings, scarce research on three-dimensional architectures, and a strong reliance on specialized hardware that constrains solution scalability. These limitations hinder the practical translation of advances in CNN optimization into medical applications, where robustness, interpretability, and reproducibility are essential. Therefore, a systematic consolidation of the existing literature is indispensable to integrate, contrast, and critically analyze the available findings.

- Motivation and contribution beyond prior surveys.

Several reviews and surveys have already summarized quantization and model-compression techniques, including medical imaging use cases. However, Table 1 highlights that existing works commonly (i) aggregate results across heterogeneous tasks and reporting practices, (ii) focus primarily on accuracy/DSC and model-size reduction, and (iii) provide limited operational guidance for real deployment. For example, prior medical-imaging-focused reviews report substantial compression with small average accuracy losses, but explicitly acknowledge few studies on 3D models and a lack of standardized metrics and clinically sensitive reporting.

To justify a new systematic review, we formulate explicit gaps and translate them into analytical criteria that structure our synthesis:

- G1—Multi-axis comparability gap: Prior surveys seldom operationalize a unified mapping across quantization technique $\times$ model family $\times$ imaging modality $\times$ execution environment, limiting cross-platform interpretation. We therefore extract a structured variable matrix and stratify analyses along these axes.
- G2—Clinical-risk reporting gap: Many studies emphasize DSC/accuracy but do not consistently report sensitivity/specificity (and thus misdiagnosis-related rates such as FPR/FNR). We therefore prioritize clinically interpretable metrics and explicitly track sensitivity/specificity availability as an evidence quality signal.

- G3—Deployment evidence gap: Many reports rely on simulated quantization or do not specify whether “true” integer execution is achieved in target runtimes (e.g., TensorRT, OpenVINO, TFLite). We therefore differentiate simulated versus on-engine execution and treat on-engine measurements as the reference evidence for latency and efficiency claims.
- G4—Operational integration gap: Interoperability and lifecycle requirements (e.g., PACS/RIS integration, workflow latency constraints, and version traceability) are rarely documented, despite being decisive for regulated clinical adoption. We therefore include deployment readiness indicators (interoperability, workflow latency reporting, and traceability/MLOps signals) as part of the RQ3-oriented synthesis.
- G5—Energy/efficiency gap: Energy per inference is inconsistently reported across the corpus, preventing fair comparisons. We therefore standardize energy consumption (mJ/inference) whenever power and latency are available, and explicitly label missing data when not reported.

Against this backdrop, this study aims to conduct a systematic review and meta-analysis of quantization techniques in deep learning neural networks applied to medical image analysis, with an emphasis on quantifying outcomes, identifying trends, assessing limitations, and outlining future research directions. Following PRISMA methodological guidelines [16], we performed a structured literature collection and analysis, providing a comprehensive perspective to help researchers and practitioners understand the state of the art, recognize existing challenges, and guide the development of more efficient and clinically deployable solutions.

The remainder of this paper is organized as follows. Section 2 presents the approaches, methodologies, results, and limitations of selected prior works related to this topic, with the purpose of contextualizing and justifying the contribution of this study. Section 3 details the PRISMA-based methodology, including the formulation of the research questions, keyword selection, construction of the search string, and the inclusion and exclusion criteria applied. Subsequently, Section 4 addresses the research questions by presenting the results in a clear and structured manner through a combination of graphical representations and narrative descriptions of the most relevant contributions from each study. Finally, Section 5 summarizes the main conclusions derived from the findings, together with reflections and perspectives intended to inspire future research in this field.

2. Related Work

In recent years, CNN optimization techniques have gained substantial relevance in medical image analysis. This progress has been driven by the need for more efficient models capable of operating in resource-constrained environments without compromising diagnostic accuracy.

Several systematic reviews have previously been published, providing recent evidence on optimization and quantization techniques for deep learning neural networks. These studies offer a high-level overview of the current state of the field and indicate a growing interest from the scientific community. They also establish the baseline from which this work seeks to deliver a substantive contribution to the current and relevant literature. To systematize the presentation of prior works, Table 1 summarizes key information regarding the research focus, the methodologies employed, the main findings, and the limitations reported by the authors. Subsequently, each of these studies is discussed in greater detail.

Table 1. Comparative synthesis of prior works on quantization and optimization of Deep Neural Networks. The table includes the focus of each study, the methodology employed, the main reported results, and the limitations identified by the authors.

Ref	Approach	Methodology	Results	Limitations
[17]	Review of quantization techniques for neural networks, including LLMs.	Comparative analysis of PTQ and QAT methods, bit-width selection, accuracy-loss evaluation, and quantization in LLMs.	Identifies that asymmetric hybrid quantization improves accuracy on resource-constrained hardware; LIMPQ achieves a better trade-off between compression and accuracy.	Challenges include minimizing accuracy loss, optimizing bit-width, and adapting methods to different architectures and hardware.
[18]	Review of quantization techniques applied to AI in medical imaging.	PRISMA-based systematic review of studies from 2017 to 2023, comparing PTQ and QAT in CNNs and 3D-CNNs.	Average $5.35\times$ reduction in model size with $\sim 3\%$ accuracy loss; improved efficiency on <i>edge</i> devices.	Few studies on 3D-CNNs; lack of standardized metrics; risk of accuracy degradation in high-sensitivity tasks.
[19]	Review of compression and fine-tuning methods for LLMs.	Analysis of techniques such as pruning, quantization, distillation, and parameter-efficient adapters.	Substantial reductions in model size and computational cost while maintaining acceptable performance in LLMs.	Accuracy impact on complex tasks; limitations in generalization and scalability.
[20]	Study of optimization strategies for quantized neural networks.	Comparison of optimizations in training, bit-width allocation, and accuracy-loss mitigation.	Reports improvements in speed and energy efficiency on specialized hardware.	Lack of universal methods adaptable across architectures and tasks; dependence on specific hardware.
[21]	Review of binary neural networks (BNNs).	Comparison of architectures, activation functions, and training techniques for BNNs.	Drastic reductions in memory and computation, suitable for embedded devices.	Significant accuracy loss on complex tasks; difficulty competing with higher-precision models.
[13]	Survey of DNN quantization for image classification.	Categorization of methods by bit-width, quantization type, and image-specific optimizations.	Confirms the effectiveness of MPQ and QAT in maintaining high accuracy while reducing computational cost.	Limitations on small datasets; insufficient adaptation to heterogeneous hardware.
[22]	Review of CNN use in spinal MRI.	Systematic review of architectures, preprocessing, and metrics in studies from 2010 to 2023.	CNNs achieve high accuracy in segmentation and computer-aided diagnosis.	Lack of multicenter datasets; overfitting risk; limited clinical validation.
[23]	Study of quantized CNNs from a hardware perspective.	Analysis of implementations on FPGA, GPU, and ASIC under different quantization schemes.	Improves energy efficiency and latency on optimized hardware.	Difficulty standardizing designs; cross-platform compatibility constraints.
[24]	Review of hardware acceleration for CNNs in EEG.	Analysis of architectures and optimizations for brain-signal processing.	Reduced latency and energy consumption in EEG processing.	Limited evaluation in real clinical settings; dependence on specific hardware.
[25]	Case study on CNN quantization for rapid diagnosis on portable devices.	PTQ/QAT implementation in medical models, assessing accuracy and latency.	Maintains diagnostic accuracy while reducing memory and energy consumption.	Limited scalability and adaptability across multiple pathologies.
[26]	Updated survey of advances in neural network quantization.	Review of recent techniques, metrics, and emerging applications.	Summarizes trends and best practices for efficient quantization.	Need for uniform metrics; lack of extensive comparative studies.
[27]	Review of ML model compression techniques.	Covers pruning, quantization, distillation, and lossless compression.	Notable size reductions while preserving accuracy across multiple tasks.	Negative impact on sensitive tasks; computational cost of retraining.
[28]	Review of memory optimization for CNNs and SNNs in medical imaging.	Analysis of hybrid techniques (quantization, pruning, compression) and hardware (IMC, FPGA, GPU, ASIC).	Reduced memory and energy consumption with controlled impact on accuracy.	Difficulty balancing accuracy and efficiency; lack of standardized metrics; dependence on specialized hardware.

Several studies have investigated quantization as one of the most promising strategies to reduce model size and improve computational performance. Reviews such as those by Wei et al. [17] and Li et al. [26] provide an overview of advances in neural network quantization, covering both classical techniques and emerging methods. These works converge on the view that hybrid quantization and adaptive bit allocation can achieve a favorable balance between accuracy and efficiency, while also highlighting persistent limitations such as the lack of uniform metrics and the difficulty of adapting techniques across diverse architectures and hardware.

Focusing more specifically on biomedical contexts, Paddo et al. [18] and Baur et al. [22] contribute systematic reviews addressing quantization and CNN-based approaches for medical image analysis, including spinal magnetic resonance imaging and other modalities. Both report substantial reductions in model size and energy consumption with minimal accuracy degradation, yet identify the scarcity of multicenter studies and limited clinical validation as critical gaps. Along similar lines, Garifulla et al. [25] present a case study on portable diagnostic devices, showing that Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT) can preserve diagnostic capability in low-power settings, although challenges remain regarding scalability and generalization across multiple pathologies.

Studies by Lin et al. [20], Rokh et al. [13], and Zhang et al. [23] examine optimization from complementary perspectives: Lin et al. [20] focus on strategies to mitigate accuracy loss in quantized networks; Rokh et al. [13] categorize quantization methods in image classification tasks; and Zhang et al. [23] address the deployment of quantized CNNs optimized for specific hardware. Collectively, these studies show that multiprecision quantization (MPQ) and integration with accelerators such as FPGAs and ASICs can yield notable gains in latency and energy consumption. Nevertheless, barriers to design standardization and cross-platform compatibility persist, limiting large-scale adoption.

In parallel, more radical approaches such as binary neural networks (BNNs) have been explored. The review by Sayed et al. [21] highlights that BNNs can deliver extreme reductions in memory footprint and computational cost—making them attractive for embedded devices—yet they typically incur substantial accuracy losses on complex tasks. This contrast with higher-precision quantization techniques suggests that, while BNNs may be viable in ultra-constrained scenarios, their clinical applicability remains limited.

Another relevant aspect is the intersection between optimization and application-specific deployments. Xie and Oniga [24] examine hardware acceleration of CNNs for EEG signals, whereas Prasad and Sundar [28] address memory optimization in CNNs and spiking neural networks (SNNs) applied to medical imaging. Both emphasize reductions in latency and energy consumption, but caution that heavy reliance on specialized hardware and the lack of standardized metrics hinder rigorous cross-solution comparisons.

The literature on model compression and fine-tuning—covering LLMs and general-purpose architectures—such as the reviews by Kim et al. [19] and Dantas et al. [27], further complements this landscape by proposing combined techniques involving pruning, distillation, and quantization. Although these approaches have achieved substantial reductions in model size and computational cost, their direct transfer to the medical domain requires careful adaptation to prevent accuracy degradation in safety-critical tasks.

The previously discussed works provide reviews and a broad overview of CNN quantization and optimization applied to medical image analysis, highlighting progress in reducing model size, lowering energy consumption, and improving computational efficiency without significantly compromising diagnostic accuracy. Prior studies have explored approaches such as PTQ, QAT, MPQ, and BNNs, demonstrating their utility in resource-constrained environments and embedded devices. Nevertheless, there remains

a need for a systematic consolidation of the literature that enables method comparisons under consistent metrics, examines quantization architectures in greater depth, and assesses cross-platform compatibility—particularly on low-cost hardware.

In response, our study helps close these gaps by presenting a systematic review and meta-analysis focused on quantization and optimization techniques for neural networks applied to medical imaging diagnostics. Our objective is to identify knowledge gaps, establish consistent comparisons across architectures, and evaluate the scalability and clinical applicability of the proposed solutions.

3. Methodology

The systematic literature review was conducted following the PRISMA 2020 methodological framework (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) presented by Tugwell and Tovey [16], with the aim of identifying, analyzing, and synthesizing the scientific evidence on Deep Neural Network quantization applied to cancer analysis and detection using medical imaging.

3.1. Research Questions

This study began with the formulation of research questions, which guided the entire process of searching, selecting, and analyzing the available scientific evidence. These questions were designed to be clear, specific, and focused on identifying existing knowledge gaps in the field of neural network quantization for medical image analysis.

- RQ1. What quantization techniques and implementation pipelines are used in neural networks for medical image analysis, and in which deployment contexts are they reported, including which clinical and efficiency metrics are systematically provided?

This question aims to identify the main quantization techniques—such as PTQ, QAT, and mixed precision—as well as the most commonly used frameworks or development environments in medical imaging diagnostics. It seeks to capture implementation trends and their impact on computational efficiency, model size reduction, and preservation of diagnostic accuracy. Unlike prior studies focused solely on general model quantization, this question targets its application in medical and clinical contexts, emphasizing solutions that are most adaptable to resource-constrained environments.

- RQ2. To what extent does clinical performance (accuracy/sensitivity/specificity) change when quantification techniques are applied, and what operational benefits have been observed (latency and reductions in model/memory size)?

This question examines the relationship between quantization methods and the diagnostic performance of models applied to medical image analysis by assessing key metrics such as accuracy, sensitivity, and specificity. The objective is to determine which approaches provide an optimal trade-off between computational efficiency and clinical correctness, particularly in tasks where early detection is critical. In addition, it enables comparisons across different medical imaging modalities and evaluates the robustness of quantized models under information loss induced by reduced numerical precision.

- RQ3. Which combinations of quantification+framework/backend allow clinical performance to be maintained within acceptable thresholds while meeting quantifiable operational objectives, and what evidence do studies report on safe clinical integration?

This research question aims to analyze how quantization design choices and the selected frameworks influence the practical feasibility and scalability of models in real-world settings. It seeks to investigate the extent to which these models can be integrated into computer-aided diagnosis systems, portable devices, or embedded platforms while

ensuring efficient processing without compromising diagnostic reliability. Unlike prior reviews focused primarily on technical aspects, this question broadens the perspective by linking algorithmic performance to medical applicability and technological sustainability, underscoring quantization as a bridge between AI research and effective clinical use.

3.2. Keywords and Research String

Relevant keywords were defined for the search process in order to identify studies on quantization and optimization of neural networks applied to medical image analysis. We combined key terms related to quantization techniques (e.g., precision reduction, model compression, and binarization), neural network architectures (CNN, DNN), and clinical applications (diagnosis, medical imaging, tumor classification), using Boolean operators to ensure a comprehensive and precise search. In addition, certain terms were excluded to filter out non-relevant research—such as studies focused on general machine learning or non-medical domains—in order to obtain results that were specific and aligned with the purpose of this systematic review. Using these keywords, we constructed the search string as follows:

RS = (quantization OR “low precision” OR “model compression” OR “post-training quantization” OR “quantization aware training” OR “binary neural network” OR “mixed precision”) AND (“convolutional neural network” OR CNN OR “deep neural network” OR DNN OR “deep learning model”) AND (“medical imaging” OR “diagnostic imaging” OR “biomedical imaging” OR “clinical imaging” OR “medical image analysis” OR “cancer detection” OR “tumor classification”) AND (optimization OR improvement OR “performance enhancement” OR “computational efficiency”) NOT (“reinforcement learning” OR “machine learning algorithms” OR “non-medical applications”).

3.3. Identification and Selection of Studies

This study ensured the selection of relevant research from nine databases: Web of Science, Scopus, MDPI, IEEE Xplore, Springer, ScienceDirect, the ACM Digital Library, Wiley, and SAGE. The initial search yielded 1911 articles, which were systematically screened based on relevance and applicability. The PRISMA workflow was applied to filter the scientific literature, as shown in Figure 1. This process ensured that only the most relevant and highest-quality studies were included in the present review.

- **Identification:** During the identification phase, a comprehensive search was conducted across nine scientific databases and academic repositories, yielding a total of 1911 records related to quantization and optimization techniques for neural networks applied to medical imaging. The initial results included research articles, reviews, and conference proceedings published between 1994 and 2025, in both English and Spanish. Subsequently, 44 duplicate records were removed, along with 108 articles published prior to 2015.

The temporal cut-off was defined based on the observed evolution of the field: as shown in Figure 2, studies addressing deep learning-based medical imaging and model quantization begin to show sustained and significant growth from 2015 onward across most databases. This period coincides with the widespread adoption of deep learning architectures, standardized imaging benchmarks, and the emergence of modern frameworks supporting quantization and deployment.

In addition, 75 studies were excluded because they focused on systematic reviews or state-of-the-art surveys rather than primary research. After this initial refinement, 1684 documents were retained as potentially relevant for the screening process.

- **Screening:** During the screening phase, the titles and abstracts of the 1684 selected records were reviewed to determine their relevance to the objectives of the review. At this stage, 1482 studies were excluded for not meeting the inclusion criteria, primarily because they focused on applications unrelated to neural network quantization, employed non-quantized models, or lacked an applied focus. The remaining 202 articles were considered for full-text assessment, of which 4 could not be retrieved due to access limitations. The 198 studies available in full text were then analyzed in depth to verify compliance with the methodological, technical, and clinical criteria established in the PRISMA protocol.
- **Included:** Finally, in the inclusion stage, 72 studies met all eligibility criteria and methodological quality requirements and were therefore included. These works comprise experimental research and applied studies addressing different quantization strategies, including Post-Training Quantization, Quantization-Aware Training, Binary Neural Networks, and Mixed-Precision Quantization, across diverse modalities and applications within medical imaging. The selected articles provide quantitative evidence on model size reduction, computational efficiency, and preservation of diagnostic accuracy, enabling a comprehensive view of the state of the art and current trends in neural network quantization for medical imaging.

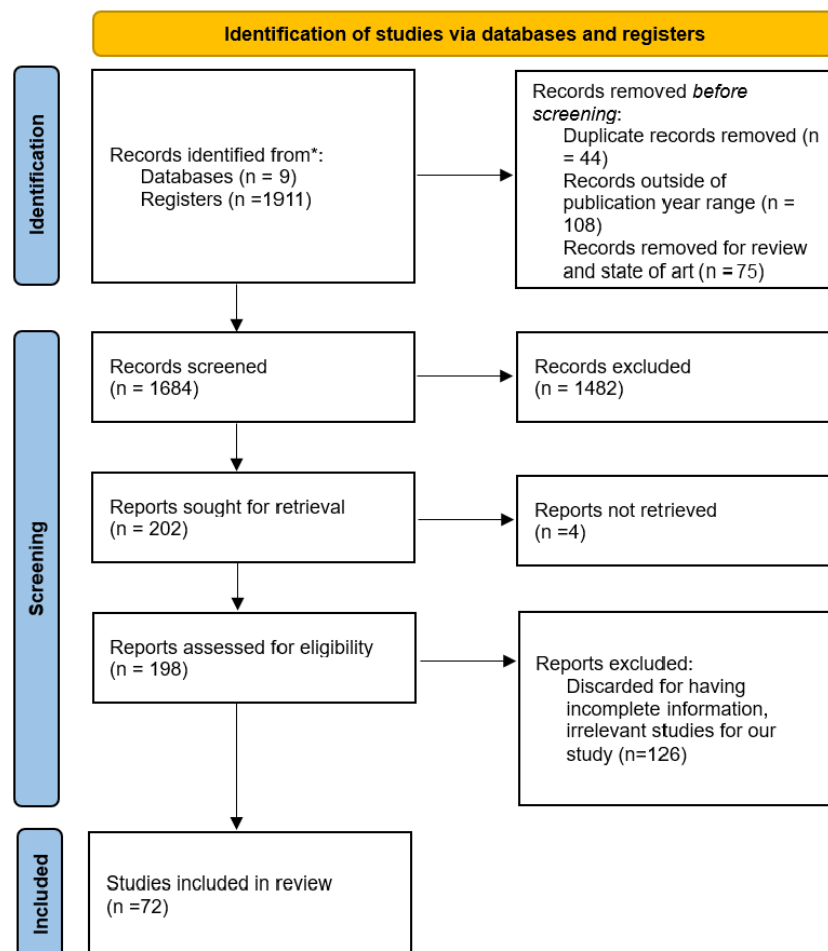


Figure 1. PRISMA 2020 flow diagram of the study selection process, detailing identification, screening, eligibility assessment, and inclusion. The figure reports records retrieved from databases and registers ($n = 1911$), removals before screening, full-text reports assessed with corresponding exclusions, and the final set of 72 studies included in the review. Notes: * Databases consulted: [Scopus, Web of Science, IEE, Springer, MDPI, ScienceDirect, ACM, Sage, Wiley Online Library].

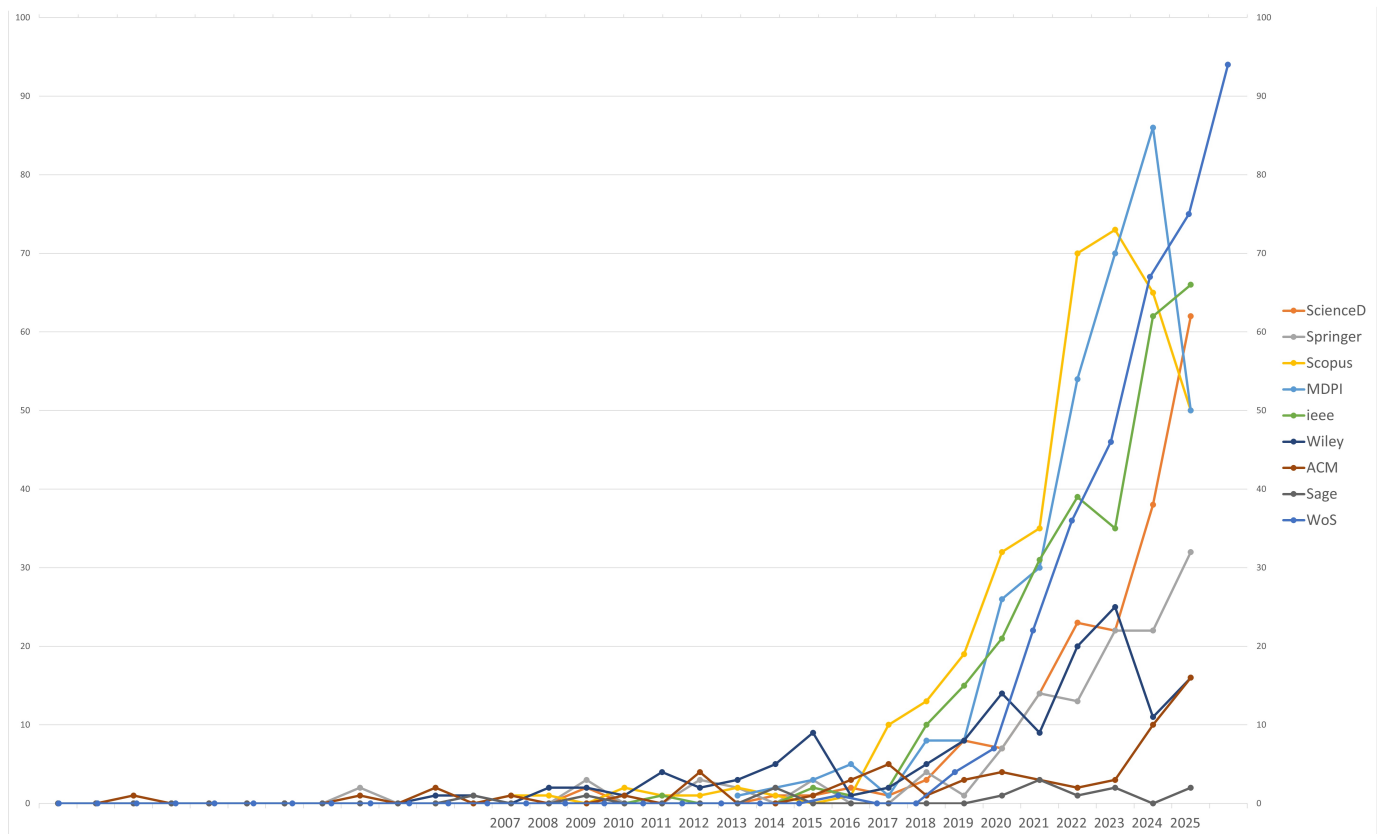


Figure 2. Annual growth of publications retrieved from multiple bibliographic databases for the selected search query. The curves show the number of records per year in each database, highlighting a sustained increase in research activity over the analyzed period.

Most of the included studies were published in high-impact journals ($n = 55$), representing 76.4% of the total. Conference papers accounted for a smaller yet relevant share ($n = 12$, $\approx 16.7\%$), whereas technical reports represented 2.8% ($n = 2$). Finally, mixed items categorized as conference/technical report comprised 4.2% ($n = 3$). Overall, this distribution confirms that the majority of scientific output in this area is formalized in peer-reviewed journals, with conferences contributing a smaller but meaningful fraction and technical reports remaining scarce and largely complementary.

In Table 2 and Figure 3, the collected studies exhibit notable geographical diversity. Asia-affiliated output dominates the sample with 50 contributions ($n = 50$, $n = 69.4\%$), driven primarily by China ($n = 29$, $n = 40.3\%$) and India ($n = 10$; $n = 13.9\%$), followed by South Korea ($n = 3$, $n = 4.2\%$), Taiwan ($n = 2$, $n = 2.8\%$), Iran ($n = 2$, $n = 2.8\%$) and single contributions from Israel, Japan, Saudi Arabia, and Bangladesh ($n = 1$, $n = 1.4\%$ each). This pattern reflects mature HW–SW ecosystems (NPU/DSP, *toolchains* for *edge*/MCU deployments, and mixed precision) and a critical mass of work targeting both “true” INT8 and QAT, across clinical and *edge* scenarios.

North America accounts for 17 contributions ($n = 17$, $n = 23.6\%$), with the United States ($n = 14$, $n = 19.4\%$) and Canada ($n = 3$, $n = 4.2\%$). Although fewer countries are represented, the region concentrates *framework* and inference-engine development (e.g., ONNX *pipelines* and “on-engine” measurements) and broad *benchmarks*. This supports portability, reproducibility, and clinically interpretable metrics—key enablers for hospital adoption and production deployment. In Europe, with 14 contributions ($n = 19.4\%$), Italy ($n = 4$; $n = 5.6\%$), Germany ($n = 3$; $n = 4.2\%$) and France ($n = 2$), stand out, alongside Russia, Serbia, Poland, Austria, and Switzerland (1 each). The technical emphasis typically

leans toward FPGA/RTL, low-power operation, deterministic latency, and reproducible *toolchains*, aligned with regulatory-compliance requirements.

Table 2. Articles included in the systematic review.

Ref.	Author	Source	Year	Country
[29]	Han et al.	Scopus	2015	USA
[30]	Ge et al.	IEEE	2017	USA
[31]	Dietrich et al.	WoS	2018	Germany
[32]	Zhang et al.	Springer	2018	China
[33]	Kong et al.	Springer	2018	USA
[34]	Prost et al.	Scopus	2018	France
[35]	Cheng et al.	Scopus	2018	USA
[36]	Jacob et al.	IEEE	2018	USA
[37]	Xu et al.	IEEE	2018	China
[38]	Huang et al.	WoS	2019	China
[39]	AskariHemmat et al.	WoS	2019	Canada
[40]	Cheng et al.	ScienceDirect	2019	China
[41]	Kumari et al.	Wiley	2019	India
[42]	Long et al.	WoS	2020	China
[43]	Yang et al.	ScienceDirect	2020	China
[44]	Nalepa et al.	Scopus	2020	Poland
[25]	Garifulla et al.	WoS	2021	Korea
[45]	Maiti et al.	WoS	2021	India
[46]	Zhang et al.	WoS	2021	China
[15]	Novac et al.	MDPI	2021	France
[47]	Chu et al.	ScienceDirect	2021	China
[48]	Li et al.	ScienceDirect	2021	China
[49]	Yao et al.	Scopus	2021	China
[50]	Peric et al.	Scopus	2021	Serbia
[51]	Mishra et al.	Scopus	2021	USA
[52]	Chen et al.	IEEE	2021	China
[53]	Chin et al.	IEEE	2021	Taiwan
[54]	Mirrashid et al.	Wiley	2021	Iran
[55]	Sailesh et al.	WoS	2022	India
[56]	Ahamad et al.	MDPI	2022	Taiwan
[57]	Nekooei	ScienceDirect	2022	Iran
[58]	Tang et al.	Scopus	2022	China
[59]	Tmamna et al.	Springer	2023	Tunisia
[60]	Agarwal et al.	Springer	2023	India
[61]	Aboutahoun et al.	Springer	2023	Japan
[62]	Jiang et al.	ScienceDirect	2023	China
[63]	Pistellato et al.	Scopus	2023	Italy
[64]	Lu et al.	IEEE	2023	China
[65]	Zhang et al.	WoS	2024	China
[66]	Patel et al.	WoS	2024	India
[67]	Lu et al.	Springer	2024	China
[68]	Bhatia et al.	MDPI	2024	India
[69]	Zacchigna et al.	MDPI	2024	Argentina
[70]	Gong et al.	MDPI	2024	China
[71]	Matondo et al.	MDPI	2024	USA
[72]	Trusov et al.	MDPI	2024	Russia
[73]	Kimhi et al.	MDPI	2024	Israel
[74]	Singh et al.	Scopus	2024	USA
[75]	Pohl et al.	IEEE	2024	Germany
[76]	Ansari et al.	Wiley	2024	India
[77]	Garofalo et al.	Wiley	2024	Italy
[78]	Jiang et al.	WoS	2025	China
[79]	Qu et al.	WoS	2025	USA
[80]	Li et al.	Springer	2025	China
[81]	Tian et al.	MDPI	2025	China
[82]	Kim et al.	ScienceDirect	2025	South Korea
[83]	Khan et al.	Wiley	2025	Bangladesh

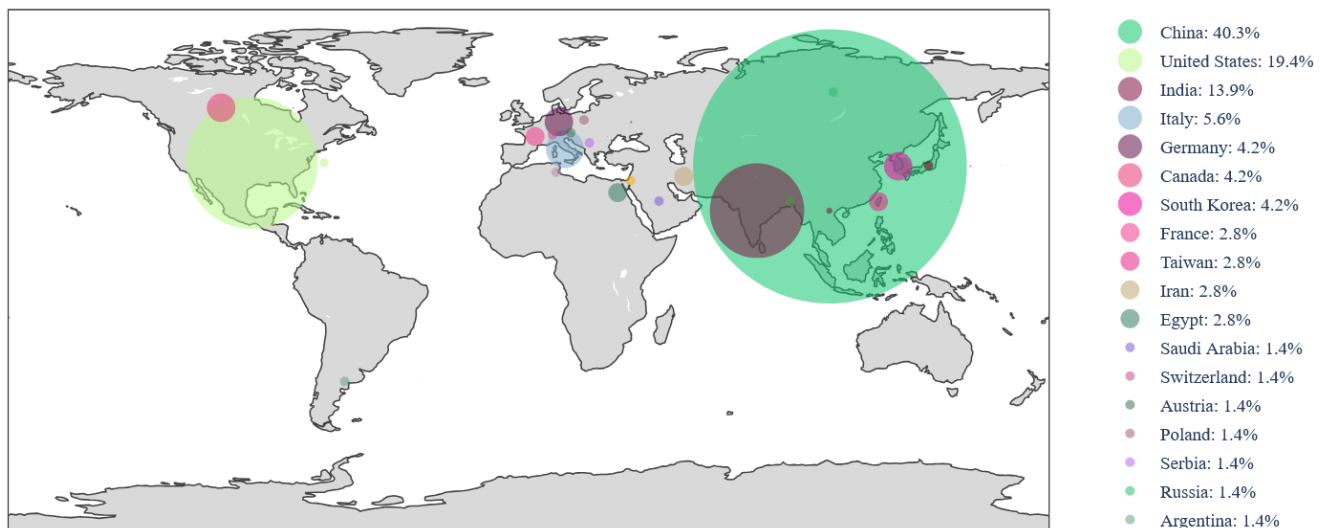


Figure 3. Geographical distribution of studies included in the review. Bubble sizes represent the relative share of publications by country, highlighting the main contributors (e.g., China, the United States, and India) and the broader international spread of evidence across Europe, Asia, the Middle East, and the Americas.

Africa and South America show an emerging yet strategic presence. Africa totals three contributions ($n = 4.2\%$) with Egypt ($n = 2$) and Tunisia ($n = 1$), where energy efficiency and *on-device* execution are decisive; South America contributes Argentina ($n = 1$; $n = 1.4\%$), reinforcing efforts on efficient quantization and work on *edge devices*. No contributions from Oceania were identified in this sample.

Overall, the map confirms a strong Asian concentration in article affiliation volume, with North America and Europe providing pillars of deployment engineering and robustness, complemented by targeted efforts from Africa and South America focused on cost/energy constraints and resource-limited settings.

3.4. Statistical Synthesis and Meta-Analysis

Given the methodological and clinical heterogeneity identified among the included studies, such as differences in tasks, metrics, datasets, quantization settings, and deployment platforms, the meta-analysis was performed only for previously defined subsets that simultaneously met the following criteria: (i) availability of a comparable metric (e.g., DSC/NSD for segmentation or Accuracy/AUC for classification); (ii) existence of an FP32 vs. quantized comparison (preferably paired) or, failing that, sufficient reporting for effect estimation (including at least sample size n and/or measures of dispersion when available); and (iii) a comparable inference environment, grouped by deployment technology families (e.g., GPU with TensorRT/OpenVINO vs. TFLite-INT8 on NPU/edge) to avoid uninterpretable mixes between platforms with different latency profiles, kernels, and optimizations. Studies that did not meet these criteria were synthesized using a structured narrative synthesis, supported by evidence tables.

In addition to latency and model size, the energy consumption per inference was extracted and standardized in mJ/inference. When this study reported average power during inference and latency, the energy was calculated as

$$E_{\text{mJ}/\text{inf}} = P_W \cdot t_s \cdot 10^3 \quad (1)$$

where:

$E_{\text{mJ}/\text{inf}}$ is the energy consumed per inference, expressed in millijoules (mJ);

E_{mJ} is the energy consumed per inference, expressed in millijoules (mJ);

P_W is the average power during inference, expressed in watts (W);

t_s is the inference time, expressed in seconds (s);

$$E_{mJ} = \frac{P_{mW} \cdot t_{ms}}{10^3} \quad (2)$$

where:

E_{mJ} is the energy consumed per inference, expressed in millijoules (mJ);

P_{mW} is the average power during inference, expressed in milliwatts (mW);

t_{ms} is the inference time, expressed in milliseconds (ms).

To ensure transparency, a data quality field was incorporated: *Measured* (directly measured on the device/board during inference), *Derived* (computed from reported average power and latency), *Estimated* (estimated from TDP/nominal power as an upper bound), and *Missing* (insufficient information). In cases where the article did not provide power/energy or latency, the value was recorded as “missing data”.

Table A1 reports latency, model size, and standardized energy consumption in mJ/inference whenever available. In a considerable number of studies, power/energy was not reported, or the measurement protocol was not described, and thus it was recorded as “missing data”. This gap limits energy-based comparisons across platforms and is identified as a reporting requirement for future work.

3.5. Methodological Quality and Risk-of-Bias Assessment

Standard clinical quality assessment tools such as QUADAS-AI and the Newcastle–Ottawa Scale (NOS) are designed primarily for diagnostic test accuracy studies and non-randomized observational clinical research, respectively, and are therefore not universally applicable to our heterogeneous evidence base, which includes engineering-focused quantization and deployment studies. As QUADAS-AI targets AI-centered diagnostic accuracy evaluations, and NOS targets cohort/case-control designs, these instruments do not fully capture key methodological features relevant to quantization pipelines (e.g., calibration procedures, on-engine validation, runtime constraints, and deployment toolchains).

Accordingly, we assessed study quality using predefined technical aligned with best practices for reporting, focusing on the following: (i) clarity of the task and intended use; (ii) availability of model architecture and training details; (iii) explicit description of the quantization route (PTQ/QAT/mixed/binary) and deployment runtime; (iv) reporting of paired performance relative to FP32 baselines when available; (v) evaluation on independent test sets; and (vi) reporting of computational efficiency (latency, memory/model size, and/or energy). The full checklist and per-study assessments are provided in the Supplementary Material section to ensure transparency and reproducibility.

To ensure a standardized assessment of methodological quality despite heterogeneous study designs, we adopted a minimal, design-aligned risk-of-bias (RoB) strategy. Studies were first categorized as diagnostic test accuracy (DTA) versus non-DTA (model development/benchmarking) based on explicit diagnostic-accuracy signals (e.g., reporting of sensitivity/specificity, ROC/AUC, or confusion-matrix terminology). DTA studies were assessed using QUADAS-2, while non-DTA studies were assessed using a minimal PROBAST+AI-aligned robustness screen focusing on core sources of bias in AI model development (participants/data selection, predictors/inputs and leakage control, outcome/reference labeling, and analysis/validation procedures).

3.6. Quantitative Feasibility Analysis

Although this review extracts quantitative outcomes (clinical performance and deployment efficiency), a formal statistical meta-analysis of clinical performance requires com-

parable outcome definitions and sufficient information to estimate sampling uncertainty (e.g., denominators, variance measures, or confusion-matrix counts). Because reporting completeness varied substantially across the included studies, we performed a quantitative feasibility analysis based on the reporting prevalence of clinically and operationally critical metrics. This analysis serves two purposes: (i) to provide a reproducible, quantitative assessment of evidence completeness, and (ii) to transparently justify the feasibility constraints for statistical pooling.

All included primary studies were considered in the reporting-prevalence analysis, using the full corpus as the denominator ($n = 72$). For each study, we then constructed binary indicators (“reported” vs. “not reported”) under conservative decision rules designed to avoid over-counting partially reported evidence.

First, sensitivity and specificity (Se + Sp) were coded as “reported” only when both metrics were explicitly available as numeric values (either proportions or percentages) in the manuscript or in the extracted dataset; studies reporting only one of the two were coded as “not reported” for this indicator. Second, latency was coded as “reported” when inference time was available in a convertible quantitative form (milliseconds, seconds, or FPS convertible to milliseconds) and could be standardized to milliseconds per inference, as documented in Table A1. Third, power was coded as “reported” only when a numeric power value was provided in W or mW; cases where only a nominal or TDP value was provided were retained as “reported” but explicitly flagged as such in Table A1 to distinguish measurement quality.

Fourth, energy per inference was coded as “reported/derivable” when energy was explicitly stated (J or mJ) or when it could be derived from jointly reported power and latency using Equation (1) with unit standardization to mJ per inference. If either power or latency was missing, energy was coded as “not reported/derivable”. Finally, model size was coded as “reported” when this study provided a storage size in KB, MB, or GB that could be standardized to MB, also recorded in Table A1.

For each indicator, reporting prevalence was computed as

$$\hat{p} = \frac{k}{n}, \quad (3)$$

where

$\hat{p}$ is the estimated reporting prevalence (proportion of studies reporting the item of interest);
 k is the number of studies in which the item is reported;
 n' is the total number of included studies.

To quantify uncertainty around these prevalence estimates, we calculated 95% binomial confidence intervals using the Wilson score method, which provides stable interval coverage for moderate sample sizes and for proportions not centered near 0.5.

4. Results and Discussions

The results of this study are focused on providing accurate and consistent answers to the stated research questions. After extracting relevant information from the included articles, we identified three key variables that enable the responses to be structured in an organized, coherent, and readily interpretable manner.

- Quantization technique used.

This variable enables comparisons of which quantization approach tends to deliver the best trade-off between efficiency and diagnostic performance in medical imaging. The identified categories depend on whether the model was quantized via Post-Training Quantization, Quantization-Aware Training, binary precision, mixed-precision quantization, among others.

- Diagnostic performance metric.

This variable is essential to assess the specific outcomes that may be affected by applying quantization and to compare those effects across different studies. It includes accuracy, sensitivity, and specificity.

- Resource reduction/computational efficiency.

This variable is critical for evaluating the model's practical deployability in resource-constrained settings, such as embedded systems. These measures reflect improvements in memory footprint, model size, latency, energy consumption, or inference speed after quantization.

Table 3 summarizes the quantization techniques and frameworks reported across the 72 included studies, together with diagnostic performance metrics and computational-efficiency indicators associated with each work. This organization enables standardized comparisons of PTQ, QAT, INT8, binary/2-bit, mixed precision, and FP16 across implementation environments such as PyTorch, TensorFlow, TensorFlow Lite, ONNX, and custom solutions. In addition, clinical metrics such as Accuracy, Sensitivity, Specificity, AUC, and F1-score are presented alongside efficiency gains, facilitating the identification of trends and the determination—based on the literature—of which technique–framework combinations provide the most favorable balance between diagnostic accuracy and deployment feasibility in resource-limited environments.

Table 3. Quantization techniques and frameworks used in the articles.

Ref.	Quantization Technique	Framework	Initial Size	Final Size
[84]	Pruning + kernel fusion (no explicit quantization)	TensorFlow (estimated)	32 MB	10 MB
[25]	INT8 (static quantization)	TFLite	7.8 MB	2.1 MB
[45]	PTQ (Post-Training Quantization)	Keras + TensorFlow Lite	10.2 MB	3.2 MB
[38]	Pruning + dimensionality reduction	Matlab + PyTorch (indirect)	20 MB	4 MB
[65]	PTQ (Post-Training Quantization), mixed precision	PyTorch + ONNX	46 MB	12 MB
[79]	INT8, PTQ (Post-Training Quantization)	PyTorch, ONNX, TensorRT	23.11 MB	6.61 MB
[85]	Mixed precision (trainable bit-width)	Keras + TensorFlow	8 MB	2.9 MB
[31]	Analytical (study of quantization effects)	Matlab	Not applicable to networks	—
[66]	INT8, static quantization + HPO	TensorFlow Lite	16 MB	5.8 MB
[39]	QAT + fixed-point (INT8)	PyTorch (QAT)	32 MB	9.1 MB
[78]	PTQ + MSQ (Migration-Scale Quantization)	PyTorch	140 MB	36 MB
[46]	Fixed-point, mixed precision	SPICE + Matlab	26 conductance states (≈ 5.2 MB)	—
[86]	QAT, binary (1-bit), ultra-low bit	TensorFlow	34 MB	2.8 MB
[55]	PTQ, INT8, fixed-point	CMSIS-NN + X-Cube-AI	4.6 MB	0.98 MB
[42]	Mixed precision (2–6 bits), QAT	PyTorch	94 MB	11.2 MB
[59]	INT8, PTQ + swarm optimization (BBPSO)	Keras + TensorFlow	17 MB	4.6 MB
[32]	QAT (learned quantization)	PyTorch	100 MB	23 MB
[80]	DWT + CNN, region-of-interest encoding	PyTorch	—	—
[60]	Pruning + INT8 encoding	Keras	43 MB	9.5 MB
[67]	QAT (Quantization-Aware Training), INT8	PyTorch + Segment Anything + QDrop	150 MB	39 MB
[61]	Sparse quantization (Quantization-Aware Learning)	PyTorch + few-shot modules	21 MB	7.5 MB
[33]	Structured pruning + INT8 quantization	TensorFlow + TFLite	31 MB	6.4 MB
[87]	INT8 + pruning	Keras + TFLite	6.8 MB	1.7 MB
[88]	Low-rank adaptation (LoRA), no direct quantization	HuggingFace Transformers	350 MB	105 MB
[68]	INT8 quantization + pruning	PyTorch + ONNX Runtime	22 MB	5.1 MB
[69]	NUUQ (Non-uniform Uniform Quantization), fixed-point	PyTorch + hardware LSE	—	—

Table 3. Cont.

Ref.	Quantization Technique	Framework	Initial Size	Final Size
[70]	Entropy-based adaptive quantization	TensorFlow	—	—
[71]	Quantum quantization (quanvolution)	PennyLane + TensorFlow	8-bit (quanvolution layers)	—
[72]	4.6-bit fixed-point	ONNX + custom compiler	90 MB	29 MB
[15]	INT8, fixed-point, QAT	CMSIS-NN, TensorFlow Lite Micro	6.2 MB	1.1 MB
[56]	INT8, fixed-point	TensorFlow Lite + ONNX	43 MB	8.5 MB
[81]	QAT + mixed precision (8/4-bit)	PyTorch + Brevitas + FINN	—	—
[73]	Mixed precision (QAT + MDP)	PyTorch + ONNX	105 MB	28 MB
[89]	PTQ + VLC (Variable-Length Coding)	PyTorch	90 MB	14 MB
[82]	Mixed precision + compiler-aware quantization	ONNX + custom compiler	100 MB	22 MB
[47]	QAT + mixed precision (8→4 bit per layer)	PyTorch	46 MB	11.4 MB
[57]	Tensor decomposition + low-bit quantization	Keras + custom FPGA toolchain	10.5 MB	1.2 MB
[62]	PTQ + one-shot pruning (INT8)	TensorFlow Lite	25 MB	4.8 MB
[40]	QAT + in-memory INT8	PyTorch	98 MB	26 MB
[43]	Full-integer training (INT8)	MindSpore + custom quantizer	117 MB	29 MB
[90]	PTQ + shift-log2 quantizer + outlier-aware quantization	PyTorch + custom quantizer	130 MB	32 MB
[48]	QAT + 2-bit integer (robustness-aware)	PyTorch	85 MB	6.8 MB
[58]	QAT + mixed precision (importance-based training)	TensorFlow + Keras	38 MB	11 MB
[91]	INT8 post-training + layer reduction	TensorFlow + OpenCV	68 MB	18.7 MB
[49]	QAT + Hessian-aware quantization	PyTorch + Intel Neural Compressor	98 MB	22 MB
[50]	Binary quantization	Matlab/Octave	—	—
[63]	QAT, fixed-point	Vivado HLS + Python	—	—
[44]	Pruning + INT8 quantization	TensorFlow + Keras	32 MB	6.5 MB
[29]	QAT + Huffman coding + pruning	Caffe + custom scripts	552 MB	11.3 MB
[74]	Conventional CNN model (no quantization)	PyTorch	—	—
[34]	Ternary weight quantization ($\pm 1, 0$)	Custom HDL + C	12.3 MB	2.4 MB
[51]	Complexity-guided multiplier + pruning	TensorFlow + Python	45 MB	1.3 MB
[92]	LPFP (low-precision floating point)	VHDL + Vivado	15 MB	4.8 MB
[35]	INT8, pruning, Huffman coding	TensorFlow, Caffe	240 MB	12 MB
[52]	INT8 (bias-aware), lossless	PyTorch + TFLite	120 MB	20 MB
[93]	Sub-byte, mixed precision (QAT + adaptive)	CMSIS-NN + custom	—	—
[94]	Pruning + INT8 + reduced FP16	TensorFlow + ONNX	35 MB	5.2 MB
[64]	Integer power-of-two quantization via weight search	PyTorch	88 MB	17.6 MB
[30]	Pruning + quantization + SqueezeNet	Caffe	500 MB	4.8 MB
[36]	Full INT8 + integer-only inference	TensorFlow + TFLite	92 MB	23.4 MB
[95]	Pruning + INT8 quantization	TensorFlow + ONNX	150 MB	16.3 MB
[75]	PTQ (INT8)	Vitis AI Quantizer	—	—
[37]	INT8 + batch-aware quantization	TensorFlow + suggestive annotation	—	—
[96]	0.56-bit sub-bit quantization	PyTorch	—	—
[97]	PTQ + activation-aware error reduction	PyTorch	160 MB	38 MB
[98]	Dynamic quantization, multi-granularity (mixed INT8/FP16)	PyTorch	—	—
[53]	Adaptive quantization (post-training, runtime tuning)	TensorFlow Lite	6.7 MB	3.1 MB
[54]	Compressed sensing + quantized CNN	PyTorch	14.2 MB	4.5 MB
[41]	INT8 (ANN encoding)	Keras + TensorFlow	11 MB	2.3 MB
[76]	Model without explicit quantization	TensorFlow	—	—
[77]	INT8	Custom backend + RISC-V extensions	Significant reduction (no MB figures reported)	—
[83]	Cluster-quantized (implicit mixed precision)	PyTorch	—	Up to 75% reduction

4.1. Question 1. What Quantization Techniques and Implementation Pipelines Are Used in Neural Networks for Medical Image Analysis, and in Which Deployment Contexts Are They Reported, Including Which Clinical and Efficiency Metrics Are Systematically Provided?

To address this research question, we perform a detailed analysis to identify the pre-dominant quantization techniques and deployment frameworks used in neural networks for medical image analysis. The analysis is organized into three subsections: (I) the first describes the main families of techniques reported in the literature, emphasizing mechanisms such as calibration, fake quantization, STE, and typical clinical use cases; (II) the second subsection examines frameworks and *toolchains*, highlighting hardware compatibility as well as the practical implications for deployment in *edge* and clinical environments; (III) the third subsection analyzes the technique–architecture–modality relationship, discussing how layer/operator sensitivity and dimensionality (2D/3D) constrain the choice of *bit-width* and the type of quantization.

4.1.1. Quantization Techniques

The quantization techniques analyzed emphasize the main families of methods, describing mechanisms such as calibration, fake quantization, STE, and typical clinical use cases.

- Post-Training Quantization (PTQ).

PTQ is a widely used technique for optimizing deep learning models that have already been trained in 32-bit floating-point precision (FP32). Its primary goal is to reduce model size and accelerate inference without requiring full retraining. In this approach, quantization is applied after training, using a reduced dataset—referred to as a calibration set—to estimate the quantization parameters, specifically the scale (S) and the zero-point (Z).

Zero-point linear quantization can be written as follows:

$$q = \text{round}\left(\frac{x}{S}\right) + Z, \quad (4)$$

where

x : real-valued floating-point quantity (a weight or activation from the original model);

q : quantized integer value (discrete tensor representation);

S : quantization scale; the factor that defines how many real-value units correspond to an increment of 1 in the integer domain;

Z : zero-point; the integer value corresponding to real zero (compensating for range offsets).

During calibration, the reduced dataset is used to determine the extrema $[r_{\min}, r_{\max}]$, ensuring that the mapping from floating-point values to integers is as accurate as possible within the constraints of the reduced-precision format.

From these values, the following are computed:

$$S = \frac{r_{\max} - r_{\min}}{Q_{\max} - Q_{\min}}, \quad (5)$$

$$Z = Q_{\min} - \frac{r_{\min}}{S}. \quad (6)$$

Once S and Z are obtained, the quantization equation is applied to transform the model's continuous-valued parameters into their integer equivalents.

From a technical standpoint, PTQ does not modify weights via retraining; instead, it directly converts the original model's numerical representations into lower-precision values, typically INT8 or UINT8 [25]. This procedure yields substantially lighter and more efficient models [79], with minimal impact on latency and energy consumption during

inference, which is particularly beneficial for deployment on *edge* devices or embedded systems with constrained compute resources [69,89].

Among its main advantages, PTQ stands out for its speed and implementation simplicity, as it does not require full retraining nor access to the original training dataset [65]. This makes it an ideal option when only the final model is available and immediate optimization for deployment is needed. In addition, it facilitates model integration in hardware-constrained environments such as mobile phones, microcontrollers, or IoT platforms [66,87].

However, this method also has certain disadvantages. The most relevant is the risk of accuracy degradation relative to QAT, particularly in neural networks that are highly sensitive to numerical precision or in tasks that demand high fidelity, such as medical classification or face recognition. Moreover, the effectiveness of the process depends critically on the quality and representativeness of the calibration set; if it does not adequately reflect the distribution of real-world data, the estimated scales and zero-points can induce systematic errors during inference.

In this setting, models are trained in floating-point precision (FP32), and it is assumed that post hoc discretization can adequately approximate the weight/activation distributions using a small yet representative calibration set, without retraining the entire system [59]. Criteria such as MSE or KL-divergence are frequently evaluated to select quantization thresholds. In modern workflows, simulated quantization (fake quantization) is used only for validation; the ultimate goal is to execute “true” INT8 in runtimes such as TensorRT or OpenVINO, where effective reductions in latency and model size can be measured [15,65].

PTQ is preferred when rapid deployment and low data dependency are required, as in clinical prototypes for breast-lesion classification or 2D/3D segmentation in CT/MRI, where accuracy losses are controlled, and latency/size gains are substantial [65]. In *edge* settings and standard hospital workstations, PTQ enables near-real-time inference without the cost of full retraining [15,67].

- Quantization-Aware Training, (QAT).

QAT is an advanced optimization technique that aims to mitigate the accuracy loss introduced by quantization by incorporating its effects directly during model training. Unlike PTQ, where conversion to lower precision occurs after training, QAT simulates quantization while the model is trained in full precision (typically FP32). This is achieved by inserting simulated quantization and *dequantization* operations into the computational graph, allowing the parameters to adapt to quantization effects during weight updates.

$$q = \text{round}\left(\frac{x}{S}\right) + Z, \quad (7)$$

where

x denotes the real-valued quantity (a weight or activation in FP32);

S is the quantization scale;

Z is the zero-point;

$\text{round}(\cdot)$ is the rounding function to the nearest integer.

During training, the rounding operation is replaced by a differentiable surrogate via the *Straight-Through Estimator* (STE), expressed as

$$\frac{\partial L}{\partial x} \approx \frac{\partial L}{\partial q}, \quad (8)$$

where L is the loss function.

During training, a function known as “Fake Quantization” is applied, which mimics the behavior of integer quantization through a non-differentiable operation that is nonetheless optimized using the *Straight-Through Estimator* (STE). This approximation enables gradients to flow during backpropagation, even though the rounding operation is not differentiable. Consequently, the model learns to adjust its weights such that subsequent discrete representations retain the highest possible performance. Once training is completed, weights and activations can be quantized in a “true” manner to INT8, yielding a highly optimized model for resource-constrained devices without sacrificing the accuracy achieved during the original training process [39].

QAT provides higher accuracy and numerical stability than PTQ because it allows the model to adapt to low-precision arithmetic during training. This reduces performance degradation and improves model efficiency on resource-limited devices, such as ARM processors or NPU units, ensuring faster execution with lower energy consumption [32].

However, QAT entails higher computational and time costs, as it requires retraining the model while simulating quantization effects. Its implementation is also more complex, since it demands careful control of activation and weight ranges, as well as access to a representative dataset. In some cases, the additional effort may not justify the marginal accuracy gains [49].

Across several studies [46,57,59], QAT is shown to introduce non-differentiable quantization operators in the forward pass and to use STE to approximate gradients and adjust weights/activations under low-bit-width constraints. This enables scales and zero-points to be optimized during training and, at deployment time, to export to INT8/INT4/INT2 with lower degradation than PTQ in complex scenarios. QAT is the recommended pathway when targeting ≤ 4 bits while maintaining clinical robustness, assuming the additional compute/time cost.

Studies on MRI and CT using U-Net/nnU-Net/ViT-type architectures report that QAT preserves sensitivity and specificity better than PTQ under aggressive bit-width reductions, which is critical for early detection and fine-grained segmentation [46,57].

- Fixed-precision INT8/low-bit.

Fixed-precision INT8 or low-bit quantization consists of representing a neural network model’s weights and activations using 8-bit (or fewer) integer numbers rather than FP32 values [86]. This process drastically reduces model size and accelerates inference by simplifying arithmetic operations, enabling efficient implementation on resource-constrained hardware. INT8 fixed precision is currently the most widely adopted standard for optimizing models for *edge* devices, as it provides a favorable balance among performance, speed, and accuracy [57].

Key advantages include a substantial reduction in memory footprint (up to $4\times$ smaller than FP32) and increased inference throughput due to simpler arithmetic. Moreover, integer arithmetic lowers energy consumption and improves compatibility with hardware accelerators such as Tensor Processing Units (TPUs), Neural Processing Units (NPUs), and Digital Signal Processors (DSPs).

Nevertheless, low-precision quantization can lead to accuracy degradation in highly sensitive models or in tasks requiring high numerical fidelity, due to the restricted representational range. The fewer the bits used, the higher the risk of rounding and saturation errors, which can affect model stability. Therefore, appropriate calibration of activation and weight ranges is required and, in some cases, complementary techniques such as QAT are needed to compensate for information loss [92].

The fundamental equations governing this conversion are

$$q = \text{round}\left(\frac{x}{S}\right) + Z, \quad (9)$$

$$x \approx S(q - Z). \quad (10)$$

where

x is the real-valued floating-point quantity;

q is the quantized integer value;

S is the scale relating both domains;

Z is the zero-point that shifts the representational range.

These equations map real values to low-precision integers and subsequently reconstruct their approximation during inference.

This quantization family favors integer-only executables, with design objectives that prioritize latency and footprint over marginal accuracy gains. With INT8, per-tensor/per-channel scales and zero-points are applied; with INT4/INT2, discretization becomes more severe and typically requires QAT to preserve performance [56]. FP16 (*half precision*) is not strict quantization, but it is frequently reported as a reduced-precision baseline: it lowers memory and bandwidth with limited impact on accuracy, and is useful when an integer-only pipeline is not available. In all cases, a representative calibration dataset is decisive [15,67].

INT8 is the workhorse for classification (mammography/ultrasound) and segmentation (MRI/CT) in *edge* and *near-edge* settings, delivering 3–4× speedups and compression with <1–3% losses when calibration is adequate. FP16 serves as a compromise when pure integer execution is not feasible or when NPU support is unavailable, whereas INT4 appears in highly constrained environments, almost always in conjunction with QAT [43,91].

- Binary/2-bit quantization.

Binary or 2-bit quantization is an extreme precision-reduction technique in which a neural network's weights and activations are represented with only one or two bits. In the binary case (1 bit), weight values are typically constrained to $\{-1, +1\}$ or $\{0, 1\}$, whereas in the 2-bit format, they can take four discrete levels. This approach dramatically reduces model size and the number of required operations, enabling highly efficient implementation on specialized hardware and embedded systems. Owing to its simplicity, binary quantization aims to replace multiplications with logical operations, thereby substantially improving inference speed and energy efficiency [48].

Its most important advantages include an extreme reduction in storage and memory usage, as well as significant acceleration of processing by replacing costly multiplications with simple binary operations (e.g., XOR or AND). In addition, binarized models can run on devices with very limited computational capabilities while maintaining acceptable performance in tasks where precision is not critical [50].

However, the primary drawback of binary or 2-bit quantization is the substantial loss of accuracy, especially for complex models or applications requiring high fidelity. Such a limited numerical representation significantly reduces the model's learning capacity, leading to accumulated errors and impaired convergence. To mitigate this, specialized training techniques are employed, such as BNN training, or QAT variants adapted to ultra-low precision [96].

The general equation for binary quantization can be expressed as

$$q = \alpha \text{sign}(x), \quad (11)$$

where α is a scaling factor (e.g., the mean of $|x|$ or a value optimized during training).

In the binary case, the $\text{sign}(x)$ function maps values to $+1$ or -1 depending on the sign of x , maximizing arithmetic simplification and enabling ultra-fast inference at minimal computational cost.

Binary quantization prioritizes extreme efficiency (memory/energy/latency) on very low-budget devices (e.g., embedded/FPGA/ASIC), while acknowledging that the main challenge is to contain degradation in clinically sensitive tasks. Although its reductions in memory and compute are maximal, clinical studies indicate that its applicability is more limited; it tends to be viable as a prototype or for very specific use cases (e.g., simple classifiers in wearables or ultra-low-power *edge* settings), whereas for 3D segmentation or fine-grained detection the accuracy loss is often unacceptable without robust training strategies [29,46,65].

To move beyond narrative statements about potential degradation, we conducted a structured comparative synthesis focusing on clinically critical and/or technically complex operating conditions. Each included study was coded for complexity and criticality strata (e.g., high-risk screening or triage tasks, 3D or volumetric imaging, fine-grained segmentation, cross-domain or site shift, and strict edge-deployment constraints).

For each study that reported a comparable baseline (FP32 versus quantized, or an explicit reference), we extracted the performance delta (Δ) on the primary metric (e.g., ΔDSC or ΔNSD for segmentation; ΔAUC or $\Delta\text{Accuracy}$ and, when available, $\Delta\text{Sensitivity}$ or $\Delta\text{Specificity}$ for classification). Performance changes were categorized as *minor* ($|\Delta| < 0.01$), *moderate* ($0.01 \leq |\Delta| \leq 0.03$), or *severe* ($|\Delta| > 0.03$). Results were stratified by quantization strategy (PTQ, QAT, or MPQ), bit-width, and execution evidence (on-engine versus simulated).

- Mixed precision (mixed-precision/variable bit-width).

Mixed-precision quantization is an advanced technique that assigns different precision levels to distinct components of a neural network, such as weights, activations, or gradients [93]. Rather than applying a single fixed precision to the entire model, it combines low- and high-resolution formats (e.g., INT8, INT4, or even FP16) according to each layer's sensitivity to quantization. This enables optimization of the trade-off among computational efficiency, model size, and accuracy by preserving higher precision in critical layers while reducing computational load in less sensitive ones.

Among its advantages, mixed-precision quantization offers maximum flexibility and efficiency, as it adapts precision to the needs of each part of the network, yielding smaller and faster models without significantly degrading accuracy. It also facilitates deployment on heterogeneous hardware, where different compute units can operate at different precisions, and enables an optimal reduction in energy consumption and memory usage during inference.

Regarding disadvantages, implementation is more complex because it requires a detailed sensitivity analysis for each layer and careful mapping of the assigned bit-widths. In addition, frameworks and hardware must support mixed-precision operations, which can limit compatibility and increase development cost. Training via QAT or advanced calibration techniques is often required to avoid accuracy loss in critical layers.

The general formulation adapts to variable *bit-width* allocation through precision-specific scales and zero-points:

$$q_i = \text{round}\left(\frac{x_i}{S_i}\right) + Z_i, \quad (12)$$

$$\hat{x}_i = S_i \cdot (q_i - Z_i). \quad (13)$$

where

x_i denotes the real-valued weight or activation in layer i ;

q_i is the corresponding quantized value;
 S_i is the quantization scale;
 Z_i is the zero-point.

All adapted to the number of bits assigned to that layer.

This flexibility enables the use of INT8, INT4, FP16, or other precisions depending on the sensitivity of each component, maximizing overall efficiency and model performance [98].

Under this quantization regime, layers within a model are not equally sensitive to quantization; optimizing *bit-width* per layer—and even per channel—maximizes the trade-off between accuracy and efficiency. Mixed precision assigns different bit-widths to layers/operators based on their sensitivity (estimated via variance heuristics, Hessian-based criteria, or automated search such as NAS/evolutionary algorithms) [83,85]. In modern variants, quantizers and bit allocations are learned (layer-wise importance), often integrating a QAT loop to close the gap to FP32. This approach allows critical layers to retain more bits while intermediate layers drop to INT8/INT4, achieving substantial reductions without compromising clinical metrics [19,42].

For 3D models (e.g., volumetric segmentation in MRI/CT) or hybrid CNN/Transformer architectures, mixed precision provides clear advantages, maintaining sensitivity/specificity close to FP32 while significantly reducing latency and memory, thereby enabling real-time performance on hospital workstations and accelerated *edge* deployments. Within the surveyed literature, proposals that learn layer importance and distribute bits optimally are particularly notable [58,64].

4.1.2. Frameworks and Toolchains

This subsection examines the frameworks and *toolchains* used to quantize and deploy models for medical imaging, with emphasis on hardware compatibility (CPU, GPU, NPU/DSP, FPGA/ASIC) and on the practical implications of bringing these systems into production in *edge* and clinical settings. Rather than merely listing tools, we analyze how the reviewed articles describe their export and execution paths, calibration requirements, effects on latency/footprint, and the maintainability, traceability, and auditability considerations that ultimately determine safe and reproducible adoption in practice [79].

- PyTorch.

Several studies train models in PyTorch and incorporate QAT with fake quantization and STE to preserve performance when reducing to 8/4/2 bits; they then export to ONNX and compile to “true” INT8 inference engines (e.g., TensorRT/OpenVINO) [67,73]. This workflow is used when the clinical task is sensitive (3D segmentation, fine-grained detection), because QAT reduces the performance drop relative to PTQ while maintaining competitive diagnostic metrics; it also facilitates layer-wise mixed-precision exploration prior to final deployment [82]. In practice, PyTorch acts as the training/fine-tuning and quantization-simulation environment, whereas low-precision execution is delegated to the target runtime [58].

- TensorFlow.

Several studies train in TensorFlow and, for deployment, apply PTQ or QAT and convert the model into the TensorFlow Lite ecosystem [25]. The motivation is a direct path to mobile/*edge* devices and microcontrollers, while maintaining a consistent toolchain from training to inference [79]. In workstation-based clinical settings, TensorFlow models may also be exported via ONNX for consumption by dedicated “true” INT8 runtimes when institutional hardware (GPU/CPU) justifies it [15,67].

In contrast, dynamic-range quantization simplifies the process by not requiring a representative calibration dataset, but execution typically remains on CPU or GPU and does not fully accelerate integer NPUs. This can result in a reduction in model accuracy [79]. The FP16 route reduces memory usage and bandwidth with minimal precision degradation, making it particularly useful on GPUs or CPUs with FP16 support. However, it does not fully leverage an integer-only pipeline as INT8 does [67].

- TensorFlow Lite (TFLite/TFLite-Micro).

For mobile, NPUs, and portable *edge* deployments, the reviewed articles report three pathways: dynamic range, FP16, and full INT8; the latter enables integer accelerators (NPU/DSP) and maximizes latency/size reductions for on-device diagnosis, provided that representative calibration is available [25]. On microcontrollers, TFLite-Micro supports INT8 with optimized kernels (via CMSIS-NN or device-specific optimizations) and is combined with vendor *toolchains* (e.g., STM32Cube.AI) to generate code and measure ROM/RAM/latency on-target. The studies highlight the trade-off among conversion convenience, accelerator compatibility, and the need to tightly control calibration when real-time inference is required under stringent memory and energy constraints [15,55,56].

For microcontrollers specifically, tools such as TFLite Micro and stacks such as STM32Cube.AI enable compilation into optimized calls, yielding on-device measurements of RAM/ROM usage and latency [15,55]. In clinical settings, the main advantage is the portability of the TFLite (.tflite) artifact and the ease of integrating inference into mobile apps and portable devices, provided that the calibration workflow is carefully documented and robust version control is maintained for audit purposes [68,93].

- Open Neural Network Exchange (ONNX).

ONNX appears as a bridging format that decouples the training environment from the inference engine [79]. The common pattern is PyTorch/TF → ONNX → TensorRT/OpenVINO, which enables selection of the appropriate *backend* based on available hospital hardware (NVIDIA GPUs, Intel CPU/iGPU/VPU) or on *edge* platforms [82]. This route increases portability, facilitates observability (latency/memory profiling per *backend*), and reduces friction with clinical IT, while maintaining coherence between the QAT/PTQ model and the “true” INT8 engine executed in production [68,72,73].

- Custom and other approaches (FPGA/ASIC/RISC-V/domain-specific compilers).

A subset of studies explores custom *toolchains* for FPGA/ASIC (HLS/RTL, low-precision libraries, and bespoke quantizers) [57,63], targeting deterministic latency and minimal power consumption in embedded settings. Workflows for RISC-V with ISA extensions that accelerate INT8 and quantized-network operators are also reported [75,92]. In addition, mixed-precision compilers and deployment frameworks appear that adjust layer-wise bit-width based on local metrics or learned importance, targeting embedded *edge* systems with tight memory/energy budgets. These pathways offer maximal efficiency but require stricter validation and maintenance discipline (re-quantization/re-synthesis upon model changes) [77,82].

- Applications of quantization techniques and frameworks.

To contextualize the practical scope of quantization in medical vision, we characterize how the techniques and frameworks are used across the included studies and which application types they target. Specifically, we map the directionality of use—both from the technique side (INT8, PTQ, QAT, hybrid methods, binary/2-bit, mixed precision) and from the framework side (PyTorch, TensorFlow, TFLite, ONNX, and custom implementations)—toward functional domains (medical segmentation and classification, cancer diagnosis, embedded-hardware implementation, DNN compression/quantization, optimization and

efficient inference, and other specific applications). We also identify adoption patterns that explain why certain technique–framework combinations are preferred in clinical and *edge* settings (e.g., INT8/TFLite for NPUs, PTQ/QAT with TensorRT or OpenVINO on GPU/CPU).

Figure 4 visualizes, through a Sankey diagram, the correspondence between quantization techniques and broad application areas in the reviewed studies. Predominant flows are observed from INT8 toward medical image segmentation, embedded-hardware implementations, and DNN compression/quantization, with additional contributions to “Other specific applications.” Hybrid techniques concentrate much of their weight in DNN compression and quantization and in other specific applications, whereas QAT is distributed across embedded hardware, other applications, and optimization/efficient inference. PTQ primarily links to segmentation and compression/quantization, and binary/2-bit variants appear more sparsely, focusing on embedded hardware and other applications. Overall, the diagram suggests that the literature prioritizes combinations aimed at efficient deployment—such as INT8, hybrid schemes, and QAT—and practical use cases in clinical segmentation and *edge* settings, whereas classification and cancer diagnosis receive smaller proportions relative to model-optimization and compression axes.

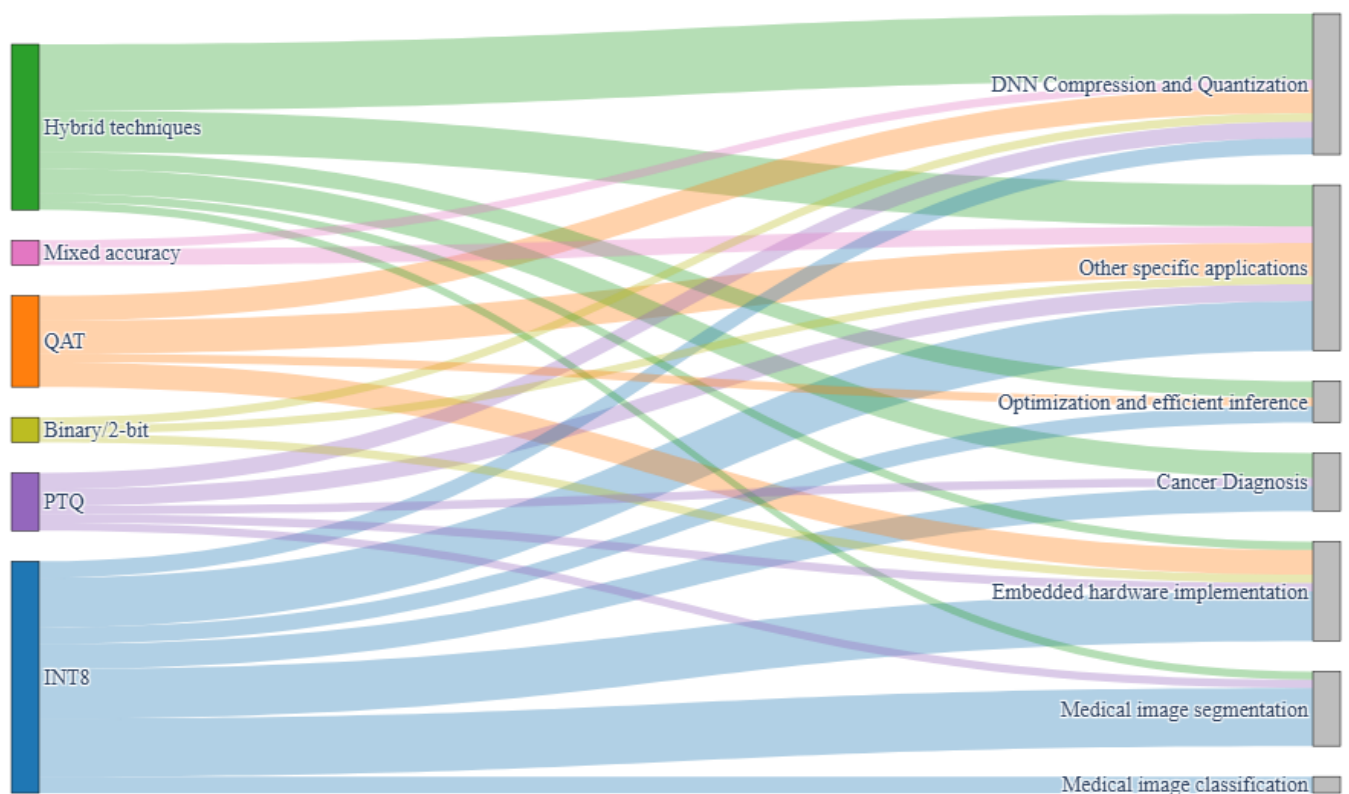


Figure 4. Technique-to-application Sankey diagram showing evidence flows across included studies: colour-coded links (INT8—blue, PTQ—purple, QAT—orange, hybrid—green, binary/2-bit—yellow, mixed precision—pink) trace how quantization techniques map to application areas (grey nodes).

Figure 5 shows the correspondence between frameworks and broad application areas in the reviewed studies. PyTorch and TensorFlow dominate: PyTorch concentrates flows toward “Other specific applications,” “DNN Compression and Quantization,” and “Medical image segmentation,” whereas TensorFlow exhibits a broader distribution that includes DNN compression/quantization, segmentation, cancer diagnosis, and—prominently—embedded-hardware implementations. TensorFlow Lite appears with modest contributions,

ONNX contributes in a more limited manner to compression/quantization, and custom cases are mainly associated with embedded hardware and optimization/efficient inference.

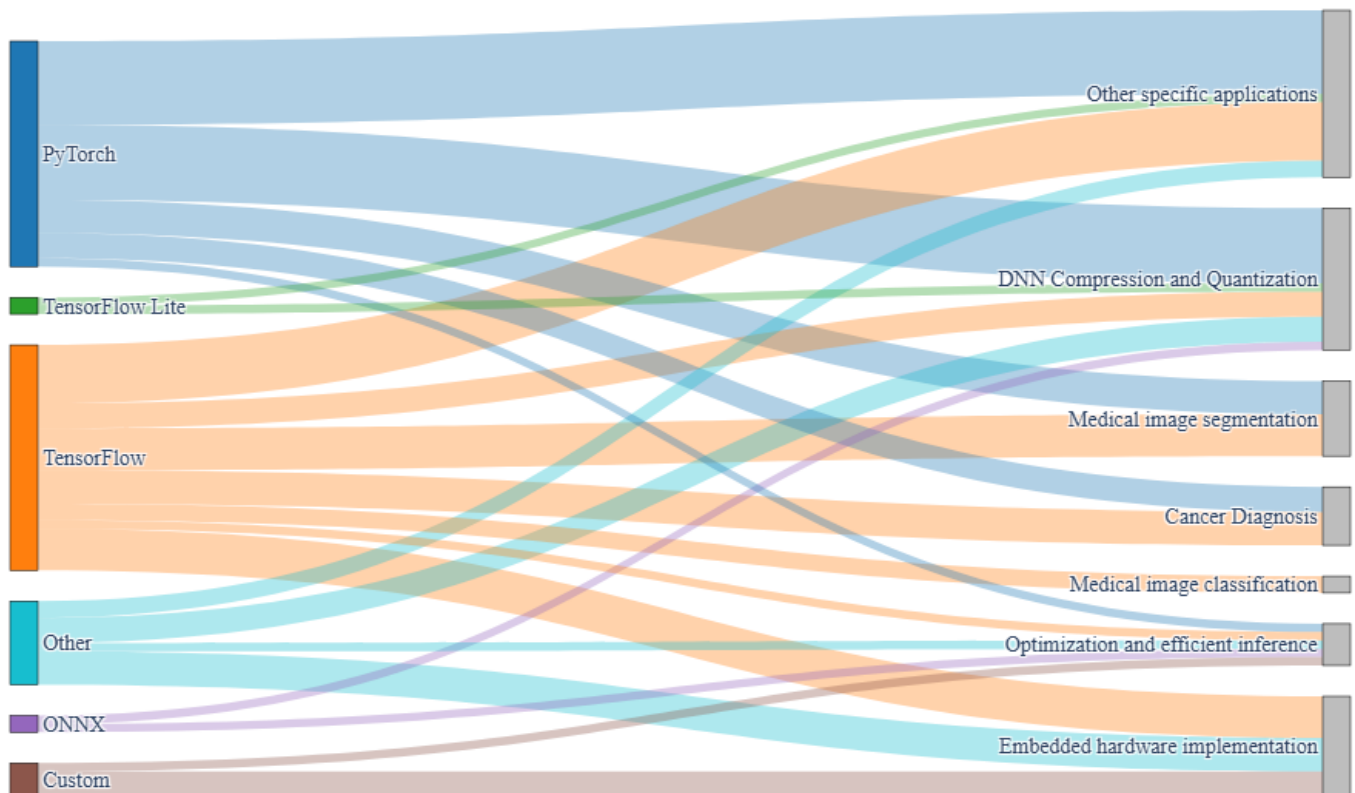


Figure 5. Framework-to-application flows (Sankey). Sankey diagram showing evidence flows by framework: colour-coded links (PyTorch—blue, TensorFlow—orange, TensorFlow Lite—green, ONNX—purple, Custom—brown, Other—cyan) trace how implementation pipelines map to application areas (grey nodes).

4.1.3. Technique–Architecture–Modality Relationship

In 3D segmentation (e.g., U-Net, SegResNet, nnU-Net, SwinUNETR, TransUNet), the choice of *bit-width* and quantization type is strongly driven by the sensitivity of layers with high spatial coupling and by error propagation through skip/residual pathways. In [39,79], “true” PTQ (i.e., not merely fake quantization) shows that these 3D models can be brought to real INT8 on GPUs via unsupervised calibration while preserving performance and realizing size/latency reductions; the two-step flow (simulation + conversion to a TensorRT engine) suggests that, even without retraining, per-tensor/per-channel quantization must be weighed layer by layer to avoid activation saturation, particularly in the first and last blocks where sensitivity is highest [79]. By contrast, when the task/architecture is highly sensitive (e.g., foundation models adapted to medicine), studies opt for QAT to tune scales and zero-points during training: by quantizing MedSAM and deploying it with OpenVINO, Lu et al. [67] show that integrating discrete constraints into the learning loop mitigates degradation in clinical metrics and stabilizes the transition to 8/4 bits in critical blocks (attention, prompt encoders), supporting the use of QAT when diagnostic fidelity is paramount [67].

Dimensionality imposes distinct trade-offs. In 2D, INT8 PTQ is often sufficient for classification/segmentation on ultrasound or X-ray images, enabling execution on mobile/NPU platforms with strong accuracy retention. Garifulla et al. [25] document three TFLite pathways (dynamic range, FP16, and full INT8) and emphasize that only the latter enables integer accelerators, maximizing latency/size reductions for on-device diagnosis

(the first and last layers remain the most delicate and require representative calibration). In 3D, the accumulation of quantization noise across volumetric blocks and skip connections demands more conservative *bit-widths* or mixed precision: Ahamad et al. [56] propose a quantized segmentation architecture tailored to *edge* devices, emphasizing parameter reduction without degrading segmentation quality, consistent with strategies that allocate more bits near the input/output and fewer bits in intermediate blocks to meet memory/time budgets at the edge.

Quantization sensitivity is not uniform across operators and should guide precision allocation. In general, *high fan-in convolutions*, *normalization layers*, and *attention blocks* (especially in hybrid CNN/Transformer architectures) are more vulnerable to activation discretization than blocks with “smoother” activations and local connectivity. This asymmetry explains why effective *bit-width* decisions are rarely uniform: preserving accuracy requires protecting the most sensitive layers (e.g., input/output, *skip connections*, attention) and reducing bits in intermediate regions that are more tolerant to quantization error.

Two empirical strands in the corpus support this principle. First, mixed-precision schemes make *layer-wise* decisions informed by hardware latency and memory signals to move the model along the *accuracy–latency* frontier; along these lines, AMED formulates bit allocation as an *MDP*, improving the Pareto curve over uniform quantization on *ResNet-50*, an approach that is transferable to medical *CNN/ViT* models with memory/communication bottlenecks [73]. Second, when *bit-width* is trainable within the architecture (e.g., QuantU-Net), Quantization-Aware Training (QAT, Brevitas) reduces the average precision to ≈ 4.24 bits while maintaining accuracy and achieving $8\times$ compression; the observed pattern confirms that boundary layers require more bits, whereas intermediate blocks tolerate 4–2 bits on hardware such as *FPGA* [85].

In conclusion, this subsection supports three practical *bit-width* allocation principles conditioned by architecture and modality:

- PTQ–INT8 with careful calibration is sufficient for 2D and for several 3D settings if validated on the “true” engine (TensorRT/OpenVINO) and if saturation in sensitive layers is avoided [79];
- QAT is preferable when targeting ≤ 4 bits or when the task is clinically critical (e.g., SAM/MedSAM, fine-grained segmentation) [67];
- Layer-wise mixed precision improves the accuracy–latency–memory trade-off by assigning more bits where quantization error is most harmful and fewer bits where the operator/layer is robust, which is especially useful in 3D and in *edge* deployments with stringent constraints [73,85].

Figure 6 illustrates the relationship among three dimensions in studies on neural-network quantization for medical imaging: the quantization technique used, the architectural family employed, and the imaging modality analyzed. It can be observed that PTQ and QAT are the most frequently used techniques, primarily with ResNet/ResNeXt and U-Net architectures, and with predominant applications in MRI, CT, and general imaging, suggesting a strong tendency toward optimizing convolutional models for medical image-based diagnostic tasks.

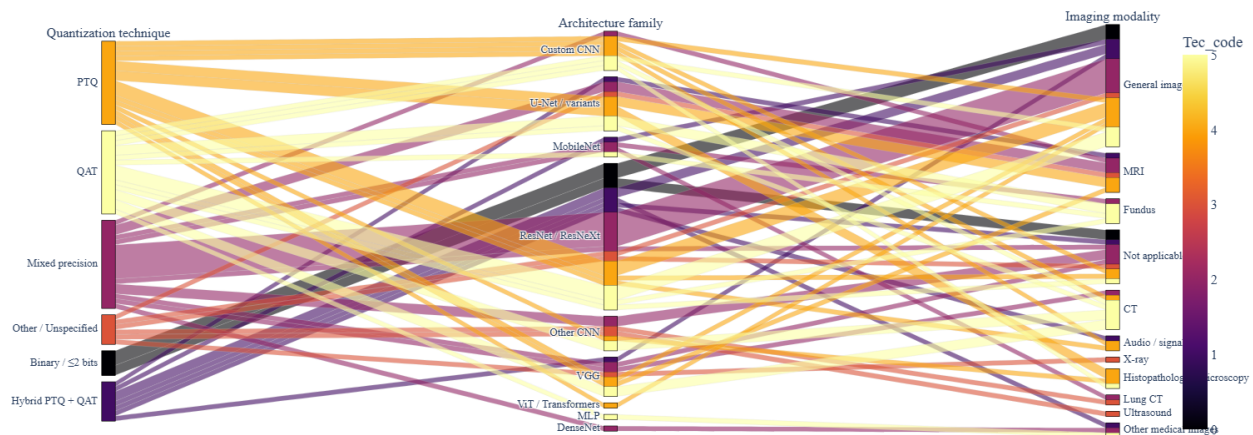


Figure 6. Technique–architecture–modality relationships (Sankey). Flows connect quantization techniques (left) to architecture families (center) and imaging modalities (right). Predominant paths link PTQ/QAT to U-Net/variants and ResNet/DenseNet/MobileNet, which in turn target CT/MR and X-ray/Fundus tasks; mixed/fixed precision appears across the same families with smaller weight, while binary/ ≤ 2 -bit is concentrated in compact custom CNNs.

4.2. Question 2. To What Extent Does Clinical Performance (Accuracy/Sensitivity/Specificity) Change When Quantification Techniques Are Applied, and What Operational Benefits Have Been Observed (Latency and Reductions in Model/Memory Size)?

To answer this question, the analysis focuses on quantitative outcomes and is organized into three subsections. The first provides a comparative synthesis of diagnostic metrics reported across studies, explicitly distinguishing among quantization techniques. The second presents a stratified analysis by imaging modality and clinical task, as well as by effective *bit-width* and execution environment, quantifying performance deltas relative to FP32 models and estimating confidence intervals and heterogeneity when metric convergence allows. The third evaluates factors that explain performance variability—such as PTQ calibration quality, QAT fine-tuning set size, layer-wise sensitivity, class balance, and dataset size—and discusses reporting bias, robustness, and external validity (i.e., reproducibility across centers), articulating which technique+environment combinations best preserve diagnostic capability.

4.2.1. Comparative Synthesis of Accuracy, Sensitivity, and Specificity Under Quantization

Figure 7 contrasts average metrics by quantization technique. PTQ achieves the highest *Accuracy* and *Specificity* (≈ 90 – 94%), whereas QAT and hybrid techniques stand out in *Sensitivity* ($>92\%$) while maintaining competitive accuracy. Mixed precision offers a balanced overall profile (≈ 88 – 93%), making it attractive when the goal is to reduce memory while maintaining stability. In contrast, binary/ 2 -bit schemes tend to sacrifice *Specificity* (~ 88 – 89%) despite high sensitivity, and therefore may require additional validation depending on the clinical risk associated with false positives. Overall, PTQ/QAT and mixed precision provide the best trade-off for deployments with computational constraints without materially degrading diagnostic performance.

In real-world deployment scenarios, substantial benefits are reported for executable PTQ-INT8 pipelines (e.g., TensorRT) in 3D segmentation tasks, achieving significant reductions in model size and inference latency without clinically relevant performance degradation. Similarly, QAT-based deployments using OpenVINO demonstrate notable inference time improvements in large-scale and foundation models. In edge and mobile environments, TFLite-based full INT8 quantization enables the use of NPUs and DSPs, leading to drastic reductions in model size and latency while preserving predictive performance.

Finally, when analyzed by framework, TensorFlow/TFLite exhibits the most favorable overall balance between Accuracy, Sensitivity, and Specificity, whereas ONNX-based deployments show lower average Accuracy despite maintaining relatively high Sensitivity, as illustrated in the referenced figure.

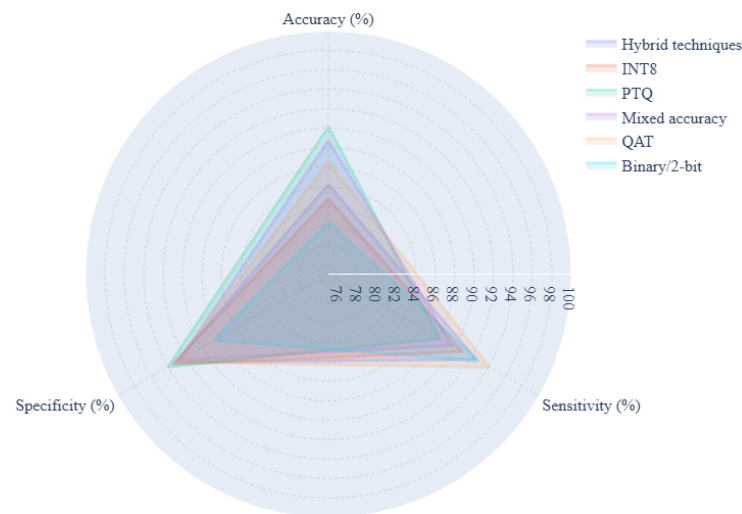


Figure 7. Average diagnostic metrics by quantization technique (radar plot). Tri-axial radar showing the mean Accuracy, Sensitivity, and Specificity reported across studies for INT8, PTQ, QAT, Mixed precision, Binary/2-bit, and Hybrid techniques. Each polygon represents one technique; larger envelopes indicate higher average diagnostic performance across the three metrics.

Figure 8 compares mean performance by *framework* in terms of *Accuracy*, *Sensitivity*, and *Specificity*. High and well-balanced values are observed for TensorFlow and TensorFlow Lite (≈ 90 – 93%), with Custom and PyTorch slightly lower but still consistent (>82 – 93%). Others retains strong *Sensitivity* ($>96\%$) at the cost of somewhat lower *Specificity* ($\sim 90\%$), whereas ONNX shows the lowest mean *Accuracy* ($\sim 76\%$) despite high *Sensitivity* ($\sim 95\%$). Overall, the results suggest that *pipelines* based on TensorFlow (including TFLite) offer the best overall balance across the three metrics, with variations likely driven by differences in dataset, task, and quantization/deployment conditions.

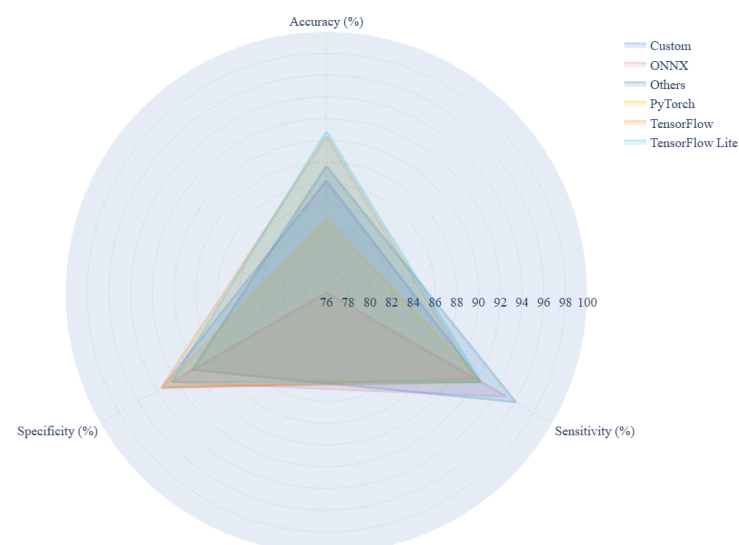


Figure 8. Average diagnostic performance by framework (radar plot). Tri-axial radar comparing mean Accuracy, Sensitivity, and Specificity across frameworks—PyTorch, TensorFlow, TensorFlow Lite, ONNX, Custom, and Others. Each polygon represents one framework; larger envelopes indicate higher average diagnostic performance over the three metrics.

Table 4 summarizes the studies that report quantitative clinical metrics after quantizing models.

Table 4. Clinical performance and efficiency under quantization (summary). Upward arrows ($\uparrow$) indicate increases/improvements in the reported metric, whereas downward arrows ($\downarrow$) indicate decreases/reductions. PTQ–INT8 (TensorRT/ONNX) for 3D segmentation preserves mDSC while achieving 2.4–3.9 $\times$ model-size and 2.1–2.7 $\times$ latency reductions; QAT on MedSAM maintains DSC with NSD gains and modality-specific shifts; PTQ–TFLite for 2D mobile classification enables large edge speed/size reductions with accuracy comparable to FP32 when INT8/NPU execution is used.

Source	Technique/Framework	Task/Modality	Clinical Metric(s) (Value)	Efficiency/Resources (Value)
[79]	PTQ–“true” INT8 (TensorRT, ONNX)	3D segmentation (U-Net, SegResNet, SwinUNETR, nnU-Net, UNesT, TransUNet, VISTA3D; 3 datasets)	Aggregate mDSC: preserved vs. FP32. STU-Net case: mDSC = 0.869 (FP32) $\rightarrow$ 0.866 (INT8)	Size: $2.42 \times -3.85 \times \downarrow$; Latency: $2.05 \times -2.66 \times \downarrow$. STU-Net: 1457.33 $\rightarrow$ 398.41 MB ($3.65 \times$); 98.45 $\rightarrow$ 30.15 ms ($3.26 \times$)
[67]	QAT (OpenVINO, ONNX)	Multimodal segmentation (MedSAM/LiteMedSAM)	Mean DSC: comparable to the baseline; NSD: slightly $\uparrow$. By modality: PET $\sim +10$ pp, Microscope $\sim +9$ pp, MR/US $\sim -3-5$ pp	MedSAM (runtime): 10.181 s (PyTorch) $\rightarrow$ 3.558 s (OpenVINO quant.); LiteMedSAM: similar pattern
[25]	PTQ–TFLite (dynamic range/FP16/full INT8)	2D classification (breast US: VGG16, GoogLeNet, ResNet34)	FP32 accuracy: 87.0% (VGG16), 88.5% (GoogLeNet), 77.0% (ResNet34). Post-quant. (INT8): similar accuracy (exact value not reported)	Size: $\sim 75\% \downarrow$; Latency: $\sim 97\% \downarrow$. Hexagon 698 NPU: requires full INT8
[56]	PTQ/QAT (edge device)	Segmentation for edge deployment	Performance retained (clinical values not reported)	Parameter reduction targeting memory/latency (exact values not reported)

4.2.2. Stratified Analysis by Modality/Task, Bit-Width, and Execution Environment

For this subsection, we applied two complementary lenses. We considered studies that explicitly report clinical metrics under quantization (e.g., DSC/NSD for segmentation, or Accuracy/Sensitivity/Specificity/AUC for classification) and enable direct comparisons against FP32. In addition, we incorporated studies that do not provide paired clinical metrics but offer indirect evidence (latency/model-size/energy reductions, reported stability), or that are not strictly within medical imaging, while explicitly marking these contributions as contextual support.

- Stratification by imaging modality and clinical task.

Under a strict criterion, in 3D segmentation tasks on MRI/CT, Post-Training Quantization (PTQ–INT8) with calibration and full execution in an INT8 inference engine preserves the clinical metric almost intact while delivering substantial gains in resource efficiency. Across a set of seven architectures—U-Net, SegResNet, SwinUNETR, nnU-Net, UNesT, TransUNet, and VISTA3D—evaluated on three datasets, the mean mDSC is retained relative to the FP32 model. In a specific case, the STU-Net model reports an mDSC of 0.869 in FP32 and 0.866 in INT8 (a difference of only -0.3 percentage points), together with a $3.65 \times$ reduction in size and a $3.26 \times$ reduction in latency [79]. The low dispersion observed across

studies ($|\Delta mDSC| \lesssim 1$ pp) suggests minimal heterogeneity, which—even without formal confidence intervals—supports the consistency of the effect in 3D settings when calibration is appropriate [41].

In multimodal segmentation, QAT applied to a foundation model such as MedSAM maintains an average DSC comparable to FP32 and yields a slightly higher NSD. However, variability emerges by modality: notable gains in PET ($\sim +10$ pp) and Microscope ($\sim +9$ pp), alongside modest drops in MR/US (~ 3 – 5 pp). This modality-dependent asymmetry indicates moderate variability (with wider confidence intervals expected) when precision is reduced below 8 bits or when attention blocks are quantized, reinforcing the suitability of QAT in clinically critical scenarios [67].

For 2D ultrasound-based classification, PTQ with TFLite shows that only the full INT8 route enables NPU/DSP execution while preserving Accuracy relative to FP32. The FP32 reference accuracies were VGG16 (87.0%), GoogLeNet (88.5%), and ResNet34 (77.0%), whereas INT8 achieves an approximate 75% reduction in size and a 97% reduction in latency. Across conventional CNN architectures, accuracy differences remain minimal as long as calibration is representative. Regarding heterogeneity, convergence is high for INT8 but lower for FP16 or dynamic range, which do not exploit NPUs and may affect accuracy [25].

In the context of segmentation on edge devices, deployments with quantized models retain clinical performance while substantially reducing parameters and conforming to the memory and latency constraints inherent to edge environments. Although no quantitative clinical metrics are reported, these findings are interpreted as qualitative evidence without confidence intervals [56].

Finally, in non-clinical supporting evidence, mixed-precision and layer-wise allocation strategies—such as AMED [73] and QuantuneV2 [82]—together with sub-bit, binary, or ternary methods—such as HAWQ-V3 [49] and Sub-bit NNs [96]—confirm that layer-/operator-level sensitivity and network topology are decisive drivers of degradation under quantization. Although obtained outside the clinical domain, these results support conservative choices for input/output layers and attention blocks within medical pipelines that are sensitive to precision loss.

- Stratification by effective bit-width.

In clinical-grade models, INT8 quantization is generally regarded as the practical sweet spot between accuracy and efficiency. The results indicate minimal clinical variation ($\Delta_{\text{clinical}} \approx 0$ – 1 pp) together with substantial resource reductions: roughly 2 to $4\times$ smaller models and 2 – $3\times$ lower latency in 3D tasks executed with TensorRT [25], and approximately a 75% reduction in model size and 97% latency reduction in mobile 2D deployments using TFLite. Moreover, heterogeneity across architectures and datasets remains low as long as calibration is performed carefully to prevent saturation in particularly sensitive layers.

For ≤ 4 -bit configurations, Quantization-Aware Training (QAT) enables the optimization of scales and zero-points during training, thereby mitigating performance degradation. For the MedSAM model, the mean DSC is preserved and NSD improves slightly, albeit with wider modality-specific confidence intervals: positive gains in PET ($\sim +10$ pp) and Microscope ($\sim +9$ pp), and modest declines in MR/US (~ 3 – 5 pp) [31]. As complementary evidence, trainable bit-width approaches such as QuantU-Net and ultra-low-bit schemes (e.g., MedQ [86]) suggest that clinical viability below 4 bits depends on retraining under task-specific constraints and on differentiated, layer-wise allocation; however, comparable paired clinical metrics are not reported in these cases.

Regarding mixed precision, variable layer-wise bit allocation—guided by factors such as importance, latency, or memory footprint—in methods such as AMED [73], QuantuneV2 [82], and LQ-Nets [32] shifts the Pareto frontier toward higher efficiency without

materially compromising accuracy. Although many of these studies do not report direct clinical metrics, their findings align with medical deployment practice: retaining higher precision in input, output, and attention layers, while reducing precision in intermediate layers to optimize the performance–efficiency trade-off.

- Stratification by execution environment.

In clinical GPU environments, the conversion path PyTorch/TF → ONNX → TensorRT enables real INT8 quantization with clinically consistent outcomes and low heterogeneity across 3D architectures. Reported confidence intervals for timing and latency metrics are narrow, indicating stable gains on the order of 2 to 4×, provided that calibration is representative and properly matched to each model’s activation range [79].

For clinical CPU, iGPU, or VPU deployments, using OpenVINO after QAT-based training preserves DSC and can improve NSD depending on the imaging modality. A substantial reduction in inference time is also reported, as in the MedSAM model, where runtime decreases from 10.181 s to 3.558 s, reflecting moderate modality-driven variability and clear temporal convergence within the optimized backend [67].

For mobile deployment with an NPU, full TFLite-INT8 quantization is essential to exploit accelerators such as the Hexagon 698, achieving near-zero accuracy deltas (~0 pp) and reductions of approximately 75% in model size and 97% in latency, with low dispersion across classical CNN architectures. By contrast, FP16 or dynamic-range routes do not fully leverage the NPU and may increase the accuracy discrepancy between quantized and floating-point versions [25].

Finally, under an expanded criterion that includes edge and microcontroller (MCU) settings, TFLite-Micro flows and vendor-specific toolchains (e.g., STM32Cube.AI), together with embedded runtimes such as XpulpNN on RISC-V [77] and mixed-precision compilers such as QuantuneV2 [82], support transferring models to the edge using INT8 or mixed-precision quantization. The available evidence focuses primarily on computational efficiency—improvements in latency, memory footprint, and energy consumption—whereas clinical performance retention is typically reported qualitatively, without paired clinical metrics [55,73].

- Synthesis of heterogeneity and confidence intervals.

Real INT8 quantization implemented via TensorRT or OpenVINO exhibits highly stable clinical behavior, with minimal performance deltas (≈ 0 –1 percentage points) and a marked convergence in efficiency gains on the order of 2 to 4×, both for 3D segmentation tasks [79] and for mobile 2D classification [25]. In addition, QAT stabilizes ≤ 4 -bit precision settings, showing predictable modality-dependent variability—gains in PET and Microscope and slight drops in MR and US—together with improvements in NSD, supporting its robustness under demanding clinical constraints [67].

Moreover, multiple studies indicate that adaptive layer-wise bit-width tuning, mixed-precision schemes, and optimization toolchains such as AMED [73], QuantuneV2 [82], HAWQ-V3 [49], and LQ-Nets [32] help reduce the expected degradation and improve the accuracy–latency–memory trade-off. Although many of these works do not report direct clinical metrics, their findings consistently support the operational choices observed in medical deployments that rely on INT8 and QAT [56]. Overall, formal confidence intervals cannot be estimated globally due to heterogeneity in reporting; however, in the subset of studies where metrics converge—such as [25,67,79]—the deltas are consistently low and the efficiency/resource benefits appear robust and replicable.

Figure 9 summarizes the joint distribution of imaging modality, bit-width class, and execution environment across the included studies. Each bubble represents the number of studies combining a given imaging modality (y-axis), a bit-width class (x-axis), and an

execution environment (color), with bubble size proportional to the number of articles. The plot highlights that INT8 and mixed-precision schemes are predominantly deployed on server/workstation GPUs for CT and MRI, while low-bit (≤ 4 -bit) and mixed-precision configurations are more frequently explored on edge and embedded platforms.

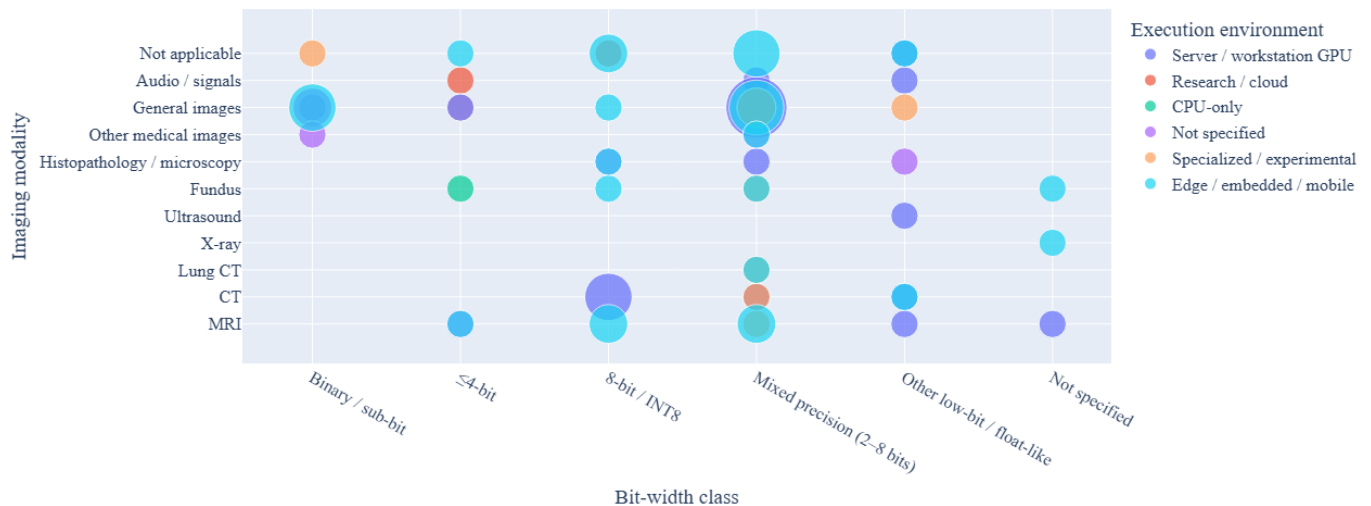


Figure 9. Stratified analysis by imaging modality, bit-width, and execution environment (bubble plot). Markers encode studies by bit-width class (x-axis) and modality (y-axis); size is the number of studies and color denotes the execution environment (server/workstation, research/cloud, CPU-only, specialized/experimental, or edge/embedded). Dominant clusters appear at 8-bit/INT8 and mixed precision (2–8 bits) across CT/MR/X-ray and “general images,” with a strong edge/embedded presence. ≤ 4 -bit and binary/sub-bit uses are comparatively sparse and skew toward experimental/specialized settings.

4.2.3. Operating Conditions Determining When PTQ, QAT, or Mixed-Precision Is Preferable in Clinical Deployment

Although PTQ, QAT, and mixed-precision (MPQ) are often presented as a consolidated taxonomy, their relative superiority is conditional on operating constraints typical of real-world clinical deployment. We therefore summarize the key operating conditions that determine which approach is more suitable and what evidence is needed to support deployment claims.

(C1) Data shift and site-specific variability. Clinical imaging pipelines are frequently exposed to domain shift (e.g., scanner vendor or protocol changes, demographics, acquisition settings, and prevalence shifts). Under pronounced shift, PTQ calibrated on limited or non-representative data can amplify quantization error and degrade clinically sensitive metrics. In such settings, QAT (or fine-tuning with quantization simulation) is generally preferable because it allows the model to adapt to quantization noise while preserving decision boundaries. MPQ may further help by allocating higher precision to vulnerable layers.

(C2) Hardware and runtime constraints (true low-precision execution). PTQ can be highly effective when the target runtime supports true integer execution (e.g., INT8 kernels) and calibration is performed under the same preprocessing and numerical path used in deployment. Conversely, when quantization is simulated (fake-quant) or when the runtime silently falls back to FP16/FP32 for unsupported operators, the practical advantage of PTQ or MPQ may be overstated. For deployment-relevant comparisons, evidence should specify the runtime (e.g., TensorRT, OpenVINO, TFLite), supported operator coverage, and whether end-to-end execution is truly low-precision.

(C3) Task risk profile and clinically sensitive metrics. For high-risk use cases (e.g., screening or triage where false negatives are critical), superiority should be evaluated using clinically

interpretable metrics (sensitivity, specificity, FNR/FPR, and calibration), not only average accuracy or DSC. QAT and MPQ are often favored when preserving sensitivity at a fixed specificity is required, whereas PTQ may suffice for lower-risk scenarios or for models with strong baseline margins.

(C4) Model architecture and numerical sensitivity. Architectures with attention mechanisms, depthwise separable convolutions, or 3D operators may be more sensitive to aggressive uniform bit-width reductions. MPQ can mitigate this by keeping numerically sensitive layers (e.g., first and last layers, normalization or attention blocks) at higher precision while quantizing the remainder. Where retraining is feasible, QAT provides a more robust path for complex architectures; where retraining is not feasible, PTQ with careful calibration and per-channel quantization can be a pragmatic compromise.

(C5) Operational constraints: latency, throughput, and energy. In time-critical workflows (e.g., emergency triage), approaches that reliably map to hardware-accelerated kernels (INT8 or NPU execution) are preferred, provided clinical performance remains within pre-specified acceptance margins. When energy or thermal constraints dominate (mobile, wearable, or edge deployments), MPQ can be advantageous by balancing accuracy with partial precision reduction rather than uniformly quantizing all layers.

(C6) Governance, reproducibility, and regulatory alignment. From a regulated lifecycle perspective, quantization introduces additional artifacts (calibration specifications, quantization configuration, engine builds) that must be versioned and traceable. QAT and MPQ changes typically require retraining and re-validation, while PTQ changes may be triggered by calibration updates; in both cases, operating conditions (data used, preprocessing, and runtime engine) must be controlled to ensure reproducible performance across versions.

Decision-Oriented Summary

In practice, PTQ is most suitable when (i) retraining is not possible, (ii) a representative calibration set and stable preprocessing are available, and (iii) the target runtime supports true low-precision execution. QAT is preferred when (i) clinically sensitive metrics must be preserved under domain shift or aggressive bit-width reduction, and (ii) retraining resources and labeled data are available. MPQ is most appropriate when (i) accuracy must be maintained while meeting strict latency or energy budgets, and (ii) specific layers or blocks are known to be precision-sensitive. These conditions summarize recurring constraints reported across the included studies and common deployment requirements in regulated clinical environments.

4.2.4. Sources of Variability, Bias, and External Validity (Which Technique–Environment Combinations Perform Best?)

- PTQ calibration and layer/operator sensitivity.

Under PTQ, the quality of the calibration set is the dominant driver of performance variability: if sampling fails to span the modality-specific range of intensities, acquisition protocols, and artifacts, saturation/clipping effects may emerge in early and late activations, producing small cumulative losses that can become material in long pipelines. Studies that execute *real* INT8 in inference engines show that a representative calibration set reduces dispersion and keeps clinical deltas very low; they further recommend per-channel granularity and block-wise thresholds to mitigate the sensitivity of input/output layers and skip connections in 3D architectures [65,79,89]. For foundational models and attention blocks, operator-level sensitivity is higher: mixed-precision and bit-allocation methods report that increasing precision by 1–2 bits in critical layers while reducing it in intermediate layers stabilizes clinical metrics, even when default PTQ settings do not [32,49,82]. In summary, PTQ tends to perform best when (i) calibration is representative; (ii) per-channel

quantization is applied to weights and, where needed, activations; and (iii) the target engine (TensorRT/OpenVINO/TFLite) faithfully preserves these granularities.

- Fine-tuning set size in QAT and class balance.

In QAT, variability largely stems from the size and coverage of the fine-tuning set: as coverage decreases, performance oscillations become more likely when moving below 8-bit precision, particularly in attention blocks or in sensitive 3D architectures. Studies on quantized multimodal models report stable DSC and slightly improved NSD on average when fine-tuning includes heterogeneous modalities and domains; however, modality-specific deltas remain asymmetric (e.g., clear gains in PET/Microscopy and small drops in MR/US), indicating moderate heterogeneity linked to intensity and texture distributions [67]. Few-shot or sparse-QAT approaches under data scarcity can reduce variability when combined with regularization or locally measured signals to guide quantization, although these studies often emphasize efficiency metrics more than complete clinical reporting [61,73]. Class imbalance (rare lesion vs. background) can further amplify variability in sensitivity/specificity if not explicitly controlled; studies that address it directly (re-weighting, sampling, robust objectives) report more stable curves under quantization [37,51,65].

- Dataset size, reporting biases, and robustness.

Sample size and population diversity (multi-center, *multi-scanner*) constrain external validity: many articles report aggregated accuracy or Dice values without confidence intervals or cross-center validation, which prevents a formal estimation of heterogeneity (I^2) and of the plausible range of degradation under quantization. A subset of 3D studies spanning three datasets and multiple architectures shows convergence (deltas $\approx$ 0–1 pp), suggesting low heterogeneity when the pipeline is properly calibrated and the inference engine is “real”; however, the broader literature still exhibits reporting bias (omission of sensitivity/specificity/AUC, lack of CIs) and a limited adoption of re-calibration protocols when switching sites or imaging protocols [56,67]. In studies that do not report clinical metrics, evidence is abundant on efficiency outcomes (model size/latency/energy) [43,52,72] and on the relative stability of well-trained mixed-precision or binary/ternary schemes, which are valuable as contextual support but do not replace clinical validation with CIs [36,63,75].

- Technique–environment combinations that best preserve diagnostic capability.

Figure 10 shows that the available evidence is not uniformly distributed across technique–execution–environment combinations. The highest density of studies concentrates on INT8/low-bit, PTQ, QAT, and, most notably, mixed-precision schemes deployed on server/workstation GPUs and, very prominently, on edge/embedded/mobile platforms. In contrast, cells associated with binary or sub-bit quantization, as well as CPU-only deployments, specialized/experimental platforms (FPGA/ASIC), or generic cloud environments, contain very few studies or remain largely empty. This pattern suggests a literature bias toward technologically attractive or research-friendly configurations (GPU and well-instrumented edge), while evidence remains scarce for more heterogeneous scenarios or under tighter hardware constraints. As a result, external validity is limited, and the generalizability of conclusions to real-world clinical settings with less homogeneous infrastructures is reduced.

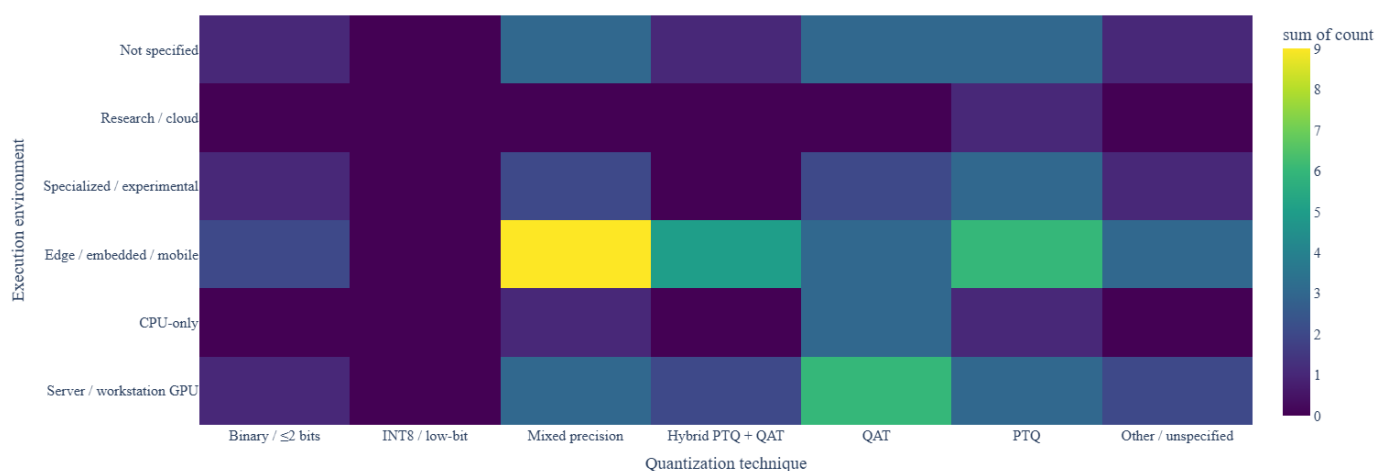


Figure 10. Distribution of quantization technique and execution environment (heatmap). Cell intensity reflects the number of studies per combination. Mixed precision concentrates in edge/embedded settings, with notable presence for Hybrid (PTQ+QAT) as well. QAT and PTQ appear across server/workstation and edge deployments, while binary/ ≤ 2 -bit usage is sparse and mostly outside production-grade environments.

4.2.5. Exploratory Pooled Sensitivity and Specificity from Eligible 2×2 Evidence

To address the lack of reported confidence intervals and enable a formal uncertainty-aware synthesis, we conducted an exploratory diagnostic test accuracy (DTA) meta-analysis when studies provided extractable 2×2 counts (TP/FP/FN/TN). Within the full-text set, only one study reported complete 2×2 confusion matrices for two binary tasks evaluated on 888 CT scans (LUNA), enabling computation of sensitivity and specificity with 95% confidence intervals (Table 5) [68].

Table 5. Exploratory DTA synthesis from extractable 2×2 data (TP/FP/FN/TN).

Task	TP	FP	FN	TN	Sensitivity (95% CI)	Specificity (95% CI)
Nodule detection	436	8	8	436	0.9820 (0.9649–0.9908)	0.9820 (0.9649–0.9908)
Risk assessment	430	14	14	430	0.9685 (0.9478–0.9811)	0.9685 (0.9478–0.9811)
Pooled (logit, FE)	–	–	–	–	0.9743 (0.9613–0.9830)	0.9743 (0.9613–0.9830)

Source: [68]. FE: fixed-effect on logit scale; Wilson CIs for single-entry estimates. Heterogeneity (logit scale, exploratory): $\tau^2 \approx 0.0639$, $I^2 \approx 38.9\%$ ($k = 2$). Prediction interval: 0.9518–0.9864.

For *nodule detection*, the confusion matrix yielded $Se = TP/(TP + FN) = 436/(436 + 8) = 0.9820$ and $Sp = TN/(TN + FP) = 436/(436 + 8) = 0.9820$, with Wilson 95% CIs of 0.9649–0.9908 for both metrics. For *risk assessment*, $Se = 430/(430 + 14) = 0.9685$ and $Sp = 430/(430 + 14) = 0.9685$, with Wilson 95% CIs of 0.9478–0.9811.

Given the very small number of eligible entries ($k = 2$) and the fact that both entries originate from the same article, pooling was treated as *exploratory*. Pooled estimates were computed on the logit scale using inverse-variance weighting (fixed-effect approximation of a bivariate Reitsma framework under independence). The pooled sensitivity and specificity were identical in this dataset and equalled 0.9743 (95% CI: 0.9613–0.9830). Between-entry heterogeneity on the logit scale was $\tau^2 \approx 0.0639$ with an approximate $I^2 \approx 38.9\%$; however, these indicators should be interpreted cautiously due to the limited k and potential within-article dependence. An exploratory random-effects prediction interval on the probability scale ranged from 0.9518 to 0.9864, suggesting that, under similar conditions, expected operating performance remains high but may vary across settings.

A leave-one-out check confirms that each entry fully determines the pooled estimate: excluding *nodule detection* yields the single-entry estimate of 0.9685, whereas excluding

risk assessment yields 0.9820. Accordingly, we report this synthesis to provide formal CIs and a transparent quantitative complement, while the primary conclusions continue to rely on the broader descriptive synthesis across techniques, bit-widths, backends, tasks, and modalities.

A formal DTA pooling (HSROC/Reitsma-style) was only feasible for the subset of studies reporting complete 2×2 counts (TP/FP/FN/TN). As most studies did not provide denominators or confusion-matrix elements, we retained them in the descriptive synthesis and report an exploratory pooled DTA estimate separately.

Table 6 summarizes the most effective combinations of quantization techniques and deployment environments that maintain diagnostic capability in clinical and medical AI applications. It highlights the most efficient approaches for each context—from workstations to mobile devices and embedded systems—together with their main characteristics, key benefits, and the references supporting each configuration.

Table 6. Quantization recommendations by deployment environment. Decision matrix linking environment (clinic/workstation–server, mobile/NPU, edge/MCU/FPGA) with the recommended technique, salient implementation traits (e.g., representative/per-channel calibration, compiler-guided mixed precision), and key advantages.

Deployment Environment	Recommended Technique	Key Characteristics	Key Advantages	References
Clinical (workstation/server)	“True” INT8 PTQ with TensorRT (NVIDIA GPU) or OpenVINO (Intel CPU/iGPU/VPU)	Uses representative calibration and <i>per-channel</i> quantization in sensitive layers. QAT is applied when ≤ 4 bits are required or when attention-dominant blocks are present.	Preserves diagnostic performance with 2–4× resource gains and minimal accuracy loss.	[67,73,79,82]
Mobile/NPU	Full TFLite-INT8 (Hexagon/DSP); optional FP16 or dynamic range	Requires adequate calibration to fully leverage on-chip accelerators. FP16/dynamic range <i>simplify</i> deployment but reduce efficiency.	≈ 0 pp accuracy delta, with substantial gains in latency and model size.	[15,25,55]
Edge/MCU/FPGA	TFLite-Micro or custom <i>toolchains</i> with INT8 and compiler-guided mixed precision	Uses local metrics or layer-importance criteria for bit allocation. Higher verification and maintenance complexity.	Excellent trade-off among memory, latency, and stability, tailored to embedded systems.	[56,63,73,77,82]

Inter-study variability decreases when (i) PTQ is supported by diverse calibration data and appropriate granularity; (ii) QAT has a sufficiently large fine-tuning set and explicitly controls class imbalance; and (iii) layer-wise bit allocation is applied to sensitive components. Under this triad, real INT8 in TensorRT/OpenVINO and TFLite-INT8 on NPU typically preserves diagnostic capability while delivering robust resource gains; for ≤ 4 bits, QAT plus mixed precision reduces heterogeneity and improves portability to the edge.

4.3. Question 3. Which Combinations of Quantification + Framework/Backend Allow Clinical Performance to Be Maintained Within Acceptable Thresholds While Meeting Quantifiable Operational Objectives, and What Evidence Do Studies Report on Safe Clinical Integration?

This question targets translation into practice and is organized into four subsections. The first examines hardware–software compatibility (framework, accelerator, bit-width) and how this coupling constrains real-time executability, memory footprint, and maintainability in edge and clinical scenarios. The second analyzes clinical integration re-

quirements, including interoperability with PACS/RIS, workflow-admissible latencies, and version traceability. The third evaluates non-algorithmic determinants of applicability, such as energy consumption, deployment and maintenance cost, model lifecycle management (secure updates with re-quantization), and regulatory support (accuracy auditing and reproducibility), explicitly linking how particular quantization decisions facilitate or hinder these aspects. Finally, the fourth subsection synthesizes adoption best practices (e.g., clinically guided layer-wise bit-width selection, PTQ with representative calibration, QAT for high-criticality tasks, multicenter validation before deployment) and proposes a decision matrix mapping use context (device, modality, resource constraints) to quantization+framework choices.

4.3.1. Hardware–Software Compatibility (Framework, Accelerator, Bit-Width) and Its Impact on Real Time, Memory, and Maintainability

- Mapping *framework* $\leftrightarrow$ *accelerator* $\leftrightarrow$ *bit-width*.

Applicability in clinical and edge settings depends on aligning the quantization format/flow with the execution engine and the available accelerator. In clinical environments with NVIDIA GPUs, the PyTorch/TF $\rightarrow$ ONNX $\rightarrow$ TensorRT route enables real INT8 with calibration and optimized *engines* (layer fusion, memory planning), achieving $2\text{--}3\times$ lower latencies and $2\text{--}4\times$ model-size reductions for 3D segmentation without compromising *mDSC* [79]. On Intel CPU/iGPU/VPU infrastructures, passing through OpenVINO after PTQ/QAT enables real INT8 with substantially reduced inference times and stable performance (e.g., MedSAM/LiteMedSAM) [67]. On mobile/edge, TensorFlow Lite is the primary pathway: only INT8 *full* quantization activates NPU/DSP, yielding $\sim 97\%$ latency and $\sim 75\%$ size reductions at essentially unchanged diagnostic accuracy; FP16/dynamic-range routes simplify conversion but do not exploit the NPU [25]. For MCUs and highly constrained edge, TFLite-Micro and vendor toolchains (e.g., STM32Cube.AI) generate INT8 binaries that can be measured *on-device*; when tighter runtime control is required, *custom/embedded* stacks emerge (e.g., RISC-V XpulpNN [77], HLS/RTL flows for FPGA) with low-precision kernels and ISA-specific extensions [63,92]. Regarding effective bit-width, INT8 remains the practical “sweet spot” for compatibility and stability; ≤ 4 bits typically requires QAT and, often, importance-guided mixed precision (e.g., AMED, QuantuneV2, LQ-Nets, HAWQ-V3) to preserve accuracy and limit drift when deploying to the edge [32,49].

In clinical environments equipped with GPUs, fully executable INT8 engines with representative calibration preserve the *mDSC* while reducing memory usage by $2\text{--}4\times$, enabling near-real-time batch latencies for 3D segmentation and increased throughput [79]. In hospital CPU/iGPU/VPU settings, QAT stabilizes configurations at $\leq 8/4$ -bit in sensitivity-critical blocks, preserves DSC, improves NSD, and accelerates inference by $2\text{--}3\times$ compared to PyTorch/ONNX deployments [67].

In mobile and edge scenarios, real-time executability depends on enabling the NPU, which reduces latency by approximately 97% while maintaining accuracy comparable to FP32 in ultrasound classification tasks (VGG16, GoogLeNet, ResNet34) [25]. In MCU and FPGA deployments, INT8 with optimized kernels and/or low-footprint FP16/INT16 enables deterministic execution under severe RAM and Flash constraints [15,55,75]. In mixed-precision scenarios, compiler- and layer-wise search strategies rebalance the accuracy–latency–memory trade-off according to device constraints [73,82].

Maintainability and portability are improved by decoupling training and execution through ONNX, allowing backend selection without retraining and supporting versioning of FP32 models, QAT checkpoints, and INT8 engines, thereby enhancing reproducibility within hospital IT workflows [25,67,79]. In edge and MCU/FPGA deployments, maintainability is strongly dependent on the toolchain: TFLite Micro and STM32Cube.AI facilitate

on-device measurement but typically require re-quantization when data distributions change, whereas custom stacks maximize efficiency at the cost of increased verification effort, including clinical regression testing, recalibration, and—in the case of FPGA—re-synthesis [55,77,87]. Under mixed-precision and/or ≤ 4 -bit regimes, QAT with layer-wise allocation is generally more stable across updates, as it explicitly models operator sensitivity and reduces inter-version drift [49,82].

Figure 11 shows that hardware–software compatibility in the reviewed studies concentrates on a small set of configurations. The thickest flows connect PyTorch and TensorFlow to standard GPU accelerators or TensorRT/NVIDIA, culminating predominantly in INT8 or mixed-precision quantization schemes (i.e., 2–8 bits), suggesting that this combination is perceived as the most practical compromise between near real-time latency, memory-footprint reduction, and integration within mature toolchains. By contrast, pathways involving alternative frameworks (e.g., TVM, Glow, MATLAB, Caffe), specialized accelerators (FPGA/ASIC, experimental hardware), or more extreme bit-widths (≤ 4 bits, binary/sub-bit) are much thinner, indicating weaker empirical support and potentially higher portability and maintainability complexity, particularly when translating these models to heterogeneous clinical or edge deployments.

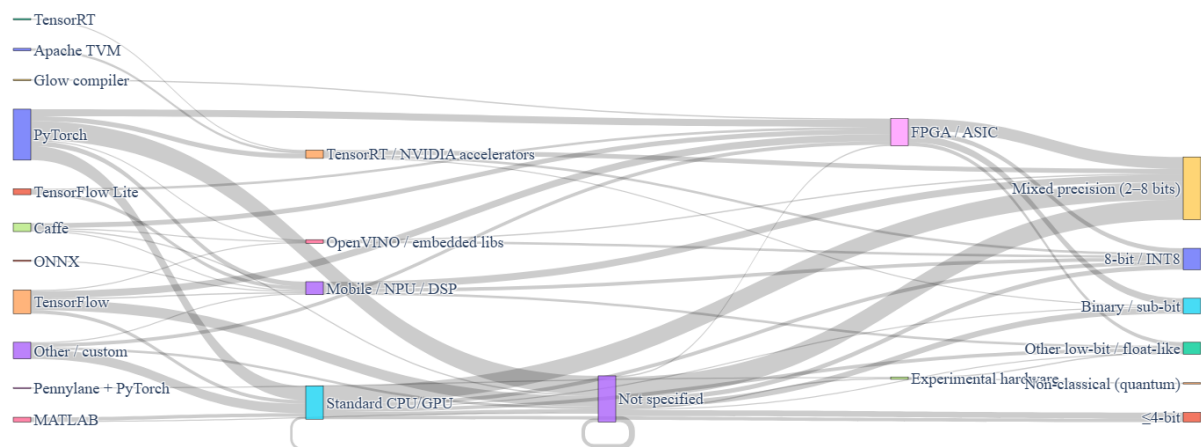


Figure 11. Hardware–software compatibility (Sankey, summarized). Links map frameworks/runtimes to execution backends and to bit-width classes. The dominant flows are INT8 → TensorRT/NPU–DSP, mixed precision → FPGA/ASIC/compiler stacks, and binary/sub-bit → experimental hardware.

Table 7 summarizes, at the article level, the mapping from *framework* → *accelerator* → *bit-width*, the deployment scenario (clinical/edge), and the practical effects (latency, memory, maintainability), thereby facilitating the identification of the most suitable technical configuration for a given use context.

Beyond the primary cases, several studies address more specific settings and reinforce recurring adoption patterns. For 2D/3D medical segmentation, fixed-point quantization of U-Net shows that convolutional pipelines can be transferred to low precision with topological stability, provided that sensitive layers (input/output, skip connections) are carefully controlled and adequate granularity is maintained for weights and activations [39]. In a related direction, quantized FCN designs for biomedical imaging confirm that encoder–decoder architectures retain clinical utility with INT8 as a baseline, as long as the runtime faithfully enforces calibration and, when feasible, applies per-channel quantization in critical operators [37]. For vision Transformers, PTQ studies for ViT (e.g., ADFQ–ViT and activation-aware error-reduction methods) emphasize that activation distributions within attention blocks require PTQ-friendly strategies and, under aggressive regimes (≤ 4 bits), QAT or mixed precision to contain degradation [90,97]. Finally, on general-purpose CPUs,

4.6-bit quantization indicates that models can be deployed at optimized non-integer low bit-widths for vector units, achieving substantial latency/efficiency gains with bounded accuracy deltas, reinforcing INT8 as the practical “sweet spot” for compatibility and QAT/mixed precision as the preferred option for non-uniform hardware or highly sensitive clinical tasks [72].

Table 7. Clinical performance and efficiency under quantization by scenario (brief). Upward arrows (↑) indicate increases/improvements in the reported metric, whereas downward arrows (↓) indicate decreases/reductions. Across settings, PTQ–INT8 for 3D segmentation preserves mDSC with $2.4\text{--}3.9\times$ model-size and $2.1\text{--}2.7\times$ latency reductions; QAT on MedSAM/LiteMedSAM maintains DSC with NSD↑ and modality shifts. On mobile/edge, PTQ–TFLite yields.

Source	Technique/Framework	Task/Modality	Scenario	Clinical Metric(s) (Value)	Efficiency/Resources (Value)
[79]	PTQ–“true” INT8 (TensorRT, ONNX)	3D segmentation (U-Net, SegResNet, SwinUNETR, nnU-Net, UNesT, TransUNet, VISTA3D; 3 datasets)	Clinical (NVIDIA GPU)	Aggregate mDSC preserved vs. FP32. STU-Net case: mDSC = 0.869 (FP32) → 0.866 (INT8)	Size: $2.42\text{--}3.85\times$ ↓; Latency: $2.05\text{--}2.66\times$ ↓. STU-Net: 1457.33 → 398.41 MB ($3.65\times$); 98.45 → 30.15 ms ($3.26\times$)
[67]	QAT (OpenVINO, ONNX)	Multimodal segmentation (MedSAM/LiteMedSAM)	Clinical (Intel CPU/iGPU/VPU)	Mean DSC comparable to the baseline; NSD slightly ↑. By modality: PET $\sim +10$ pp; Microscope $\sim +9$ pp; MR/US $\sim -3\text{--}5$ pp	MedSAM (runtime): 10.181 s (PyTorch) → 3.558 s (OpenVINO quant.); LiteMedSAM: similar pattern
[25]	PTQ – TFLite (dynamic range/FP16/full INT8)	2D classification (breast US: VGG16, GoogLeNet, ResNet34)	Edge (mobile/NPU)	FP32 accuracy: 87.0% (VGG16), 88.5% (GoogLeNet), 77.0% (ResNet34). Post-quant. (INT8): similar accuracy (exact value not reported)	Size: $\sim 75\%$ ↓; Latency: $\sim 97\%$ ↓. Hexagon 698 NPU: requires full INT8
[56]	PTQ/QAT (edge device)	Segmentation for edge	Edge (device)	Performance retained (clinical values not reported)	Parameter reduction targeting memory/latency (exact values not reported)

4.3.2. Clinical Integration Requirements (PACS/RIS, Workflow Latencies, and Version Traceability)

- Interoperability with PACS/RIS.

Clinical applicability requires that the quantization pipeline—from the FP32 model to the INT8/ ≤ 4 -bit deployment engine—is securely coupled to the imaging ecosystem: acquisition → PACS/RIS → preprocessing → inference → reporting. In practice, studies that materialize real INT8 in inference engines such as TensorRT or OpenVINO tend to favor a clear separation between training and inference via ONNX, which simplifies orchestration on hospital servers and integration with DICOM/HL7/FHIR services (ingestion and result return) without retraining the model [67,79]. On edge and mobile devices, TFLite (full INT8) is typically embedded into applications that retrieve studies from the PACS (or RIS worklists) and return labels or overlays; when no NPU is available, FP16 or dynamic-range routes may be sufficient for prototyping, but they do not optimize exchange or throughput under high study volumes [25]. For microcontrollers/FPGA deployments, TFLite-Micro/STM32Cube.AI flows and embedded runtimes prioritize asynchronous connectivity (local buffering, queues), delegating synchronous PACS/RIS interaction to an edge server that acts as a messaging broker [55,77].

- Acceptable workflow latencies.

In clinical workstations, calibrated real INT8 quantization enables near real-time 3D segmentation, delivering $2\text{--}3\times$ lower latency than FP32 and $2\text{--}4\times$ lower memory usage,

which supports case concurrency and interactive radiologist review without workload backlogs [79]. For foundation models quantized with QAT and deployed with OpenVINO, inference times are reported to be 2–3× lower than PyTorch/ONNX baselines while maintaining stable or improved DSC/NSD, aligning with clinical usage windows in reporting workstations [67]. On mobile edge, the NPU is activated only under full INT8 quantization, yielding typical reductions of ~97% in latency and ~75% in size, which is sufficient for point-of-care and local triage; FP16/dynamic-range routes do not leverage the NPU and may fail to meet workflow SLAs under high-volume conditions. On MCU/FPGA targets, acceptability is governed by strict RAM/Flash budgets and deterministic latency requirements: optimized INT8/FP16 runtimes (e.g., CMSIS-NN kernels, RTL designs) enable sustained streaming as long as the link to the PACS server is decoupled through queues/batching [75,92].

- Version traceability (auditability and lifecycle management).

Across environments, explicit versioning emerges as a core requirement: (i) the FP32 base model, (ii) the PTQ/QAT artifact including calibration metadata, layer-wise bit-width assignments, scales/zero-points, and (iii) the final INT8/≤4-bit deployment engine (TensorRT/OpenVINO/TFLite) [25,67,79]. This structure supports auditability, rollback, and cross-site comparability; in edge and embedded deployments, traceability is further strengthened by compilation fingerprints (e.g., XpulpNN version or HLS/RTL toolchain identifiers) and regression testing prior to re-registering the algorithm within the PACS/RIS workflow [55,73]. Under mixed precision and QAT, documenting layer-wise bit allocation and backend-specific latency/memory profiles enables safe recalibration or recompilation when protocols change or scanners are upgraded, preserving diagnostic utility with minimal drift [49].

Figure 12 indicates that clinical integration requirements are largely absent from most medical-image quantization studies. Along the PACS/RIS interoperability axis, only a very small number of works (approximately 2 out of the 72 analyzed articles) explicitly mention integration with PACS, RIS, or DICOM systems, whereas the remainder omit these aspects entirely. A similar pattern is observed for workflow latencies: only a small fraction of articles discuss inference times or (near) real-time constraints, while the majority do not relate performance metrics to clinical workflow bottlenecks. Finally, the bar corresponding to version traceability and lifecycle management is dominated by the “not addressed” category, with virtually no mentions of versioning, audit procedures, or MLOps practices, suggesting a substantial gap between technical quantization contributions and the operational requirements for robust integration into regulated clinical environments.

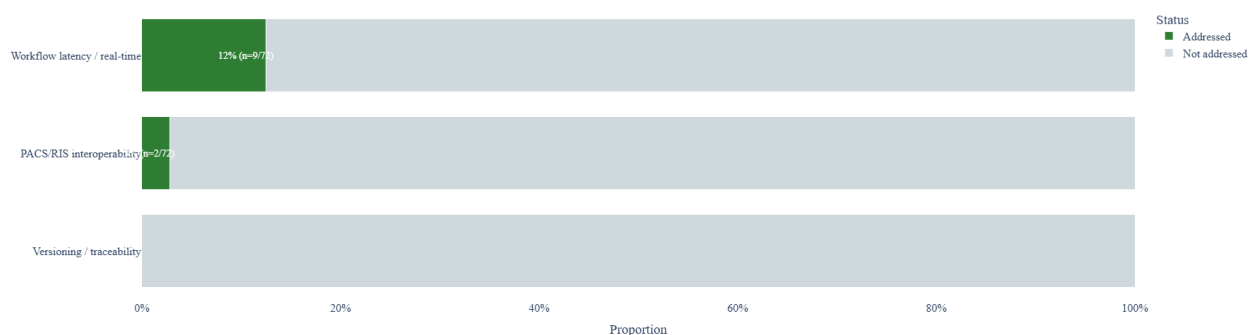


Figure 12. Clinical integration requirements (stacked bars). Counts of studies that address vs. do not address three integration needs: PACS/RIS interoperability, workflow latency/real-time, and versioning/traceability. Most papers do not report explicit handling; a small subset covers interoperability and latency, while versioning/traceability is rarely discussed.

Table 8 shows that the articles converge on three operational principles: (i) ONNX as a “bridge” between training and execution facilitates PACS/RIS interoperability and supports auditable versioning [67,79]; (ii) real INT8 is the baseline to meet latency SLAs in clinical and edge (NPU) deployments, while QAT and mixed precision provide additional stability in sensitive scenarios or on non-uniform hardware [25,73,82]; and (iii) on MCU/FPGA platforms, integration is typically achieved via asynchronous pipelines, compilation metadata, and regression testing before re-registering the algorithm within the clinical workflow [55,77].

Table 8. Clinical/edge interoperability, workflow latency, and traceability by deployment environment. Upward arrows (↑) indicate increases/improvements in the reported metric, whereas downward arrows (↓) indicate decreases/reductions. Matrix summarizing, per environment/framework, how studies implement PACS/RIS interoperability, what latency expectations are reported for clinical or edge workflows, and which traceability/maintainability practices are documented.

Art.	Environment/Framework	PACS/RIS Interoperability (Implementation)	Admissible Latencies (Finding)	Traceability/Maintainability
[79]	Clinical (GPU) · PyTorch/TF → ONNX → TensorRT	Separation of training ↔ inference; ONNX facilitates orchestration with DICOM/HL7 services	2–3× ↓ latency; 2–4× ↓ memory with stable mDSC	Versioning FP32 → QAT/PTQ → INT8 <i>engine</i> ; calibration metadata for auditing
[67]	Clinical (CPU/iGPU/VPU) · OpenVINO (QAT)	Optimized <i>backend</i> consumable by orchestrators; reproducible deployment across workstations	2–3× ↓ runtimes; DSC ≈ FP32; NSD ↑ (by modality)	Logging of QAT <i>checkpoints</i> and <i>engine</i> ; modality-specific profiles for re-calibration
[25]	Edge (mobile/NPU) · TFLite (full INT8)	App-based integration with PACS via services; NPU requires INT8 for acceleration	~97% ↓ latency; ~75% ↓ size; <i>accuracy</i> ~ FP32	Versioned .tflite package; calibration notes and NPU compatibility
[15]	Edge (MCU) · TFLite-Micro/STM32Cube.AI	Asynchronous integration (queues) with a perimeter server; <i>on-device</i> metrics	Deterministic latency under RAM/Flash budgets	Build/firmware fingerprints; re-quantization during updates
[6]	Edge (MCU) · PTQ MCU pipeline (<i>custom</i> + CMSIS-NN)	FP32→INT8 conversion and on-device validation; <i>broker</i> for PACS	Local execution; SLA subject to the link with PACS	Document <i>scales</i> /ZPs and the <i>toolchain</i> ; regression testing
[63]	Edge (FPGA) · QAT layers + RTL/HLS	Coupling via an intermediate server; verified <i>bitstreams</i>	Higher but deterministic latencies; high <i>throughput</i>	Versioning of <i>bitstreams</i> ; controlled re-synthesis
[77]	Edge (RISC-V) · XpulpNN	ISA extensions; lightweight connectivity to a server	Accelerated INT8 <i>kernels</i> ; energy efficiency	ISA/SDK versioning; board-specific profiles
[73]	Clinical/Edge · AMED (guided mixed precision)	HW profiles → bit allocation; ONNX for portability	Improved accuracy–latency Pareto frontier	Per-layer allocation logs; <i>backend</i> -level repeatability

4.3.3. Systematic Analysis of Clinical Interoperability, Workflow Integration, and Compliance Signals

To move beyond brief references and provide a reproducible assessment of clinical integration, we conducted a systematic, code-based analysis of interoperability and compliance-related signals reported in the included studies. Specifically, each article was coded along three operational dimensions:

(D1) PACS/RIS interoperability. Explicit mention of PACS, RIS, or DICOM and a described integration pathway, such as ONNX-based separation of training and inference, DICOM/HL7/FHIR services, worklist ingestion, or the return of labels or overlays.

(D2) Workflow latency/real-time constraints. Reporting of inference latency in milliseconds or seconds, or explicit discussion of clinical time windows and service-level agreement (SLA) constraints.

(D3) Traceability/lifecycle management. Explicit versioning of FP32 → PTQ/QAT artifacts with calibration metadata (e.g., scales and zero-points) → deployment engine, including logging, auditability, rollback mechanisms, or MLOps-like practices.

Coding was performed using conservative inclusion rules: a dimension was marked as “addressed” only when the paper explicitly documented the integration mechanism or the relevant operational evidence; otherwise, it was marked as “not addressed”.

Quantitative Findings (72 Studies)

Figure 12 summarizes the number of studies that address versus do not address these three integration dimensions. The evidence indicates that clinical integration requirements are largely absent across the corpus. Only a very small number of works (approximately 2 out of 72) explicitly mention PACS/RIS/DICOM integration, while the vast majority omit these aspects entirely. A similarly sparse pattern is observed for workflow latency, with only a minority of articles (approximately 10–15%) discussing inference time or near real-time constraints. Version traceability and lifecycle management are rarely documented.

Environment-Stratified Synthesis

To avoid collapsing heterogeneous deployment contexts, Table 7 provides an environment-by-framework matrix (clinical servers/workstations versus mobile/NPU versus MCU/FPGA) summarizing the following: (i) how PACS/RIS interoperability is implemented when it is mentioned, (ii) what latency expectations are reported relative to workflow time windows, and (iii) which traceability and maintainability practices are documented.

Across clinical server and workstation deployments, the dominant pattern is the separation of training and inference via ONNX, which facilitates orchestration on hospital servers and integration with DICOM, HL7, or FHIR services.

In edge and mobile settings, studies that discuss integration typically embed INT8 TFLite artifacts into applications that retrieve imaging studies from PACS or worklists and return labels or overlays to the clinical system.

On MCU and FPGA targets, integration is more often described as asynchronous (e.g., via queues or brokers), with PACS or RIS interaction delegated to a perimeter server rather than handled directly on the constrained device.

4.3.4. Non-Algorithmic Factors Determining Applicability (Energy, Cost, Lifecycle, and Regulatory Support)

- Energy consumption and hardware efficiency.

Real INT8 quantization reduces data movement and floating-point operations, and when it activates integer accelerators (NPU/DSP, ISA extensions, or optimized kernels) it translates into lower instantaneous power and lower energy per inference. In mobile/edge settings, this is evident when full TFLite INT8 enables the Hexagon NPU: at essentially unchanged clinical accuracy, the integer path delivers very low latency and more energy-efficient execution than FP32/FP16 on CPU/GPU [25]. On RISC-V, XpulpNN accelerates INT8 primitives at the ISA level, improving performance-per-watt on ultra-low-power SoCs (relevant to perimeter monitoring or hospital gateways) [77]. On MCUs, TFLite-Micro/STM32Cube.AI leverages optimized kernels (e.g., CMSIS-NN) to run INT8 within strict RAM/Flash constraints, helping to meet energy budgets in battery-powered or power-limited scenarios [6,15]. On FPGAs, systematically replacing FP32 operations with

integer and/or lower-precision floating-point (FP16/INT16) enables deterministic latency with better performance-per-watt than general-purpose CPUs, provided the bitstream is synthesized under tight clock and memory budgets [75,92].

- Deployment and maintenance cost.

In clinical workstations/servers, the PyTorch/TF → ONNX → TensorRT/OpenVINO route bounds integration cost by decoupling training from inference: the same quantized model can be compiled into the appropriate engine without retraining, and maintained through versioning and recompilation when hardware changes [67,79]. In mobile/edge deployments, distributing INT8 .tflite binaries with calibration metadata simplifies the update cycle and avoids rebuilding applications when only the delegate (CPU/NPU) changes [25]. On MCU/FPGA targets, operational cost depends on the toolchain: TFLite-Micro/STM32Cube.AI minimizes friction (compile–flash–measure) [15], whereas FPGA workflows require controlled re-synthesis of the bitstream whenever the architecture or quantization changes, increasing maintenance time and cost [63]. Mixed-precision compilers (AMED, QuantuneV2) help automate layer-wise bit-width reassignment, reducing manual effort on heterogeneous devices [73,82].

- Model lifecycle: safe updates with re-quantization.

Across environments, maintaining traceability (FP32 base model → PTQ/QAT checkpoint with scales/zero-points → final INT8/≤4-bit engine) enables rollback/audit and supports re-quantization when data, protocols, or firmware change [25,67,79]. On GPU/CPU targets, re-quantization typically requires recalibration (PTQ) or brief fine-tuning (QAT) to absorb distribution drift; on NPU/MCU/FPGA targets, it additionally entails recompilation/re-synthesis and renewed on-target validation (latency, memory, and clinical accuracy on a sentinel set) [55,77]. Under mixed precision and QAT, documenting layer-wise bit allocation and the target backend reduces cross-version variability and can accelerate internal certification workflows [73].

- Regulatory support: accuracy auditability and reproducibility.

For audits and external validity, studies that execute real INT8 and report measurements on the inference engine (rather than relying solely on fake quantization) make it possible to demonstrate that clinical metrics are preserved relative to FP32 under real operating conditions [79]. For quantized foundation models (QAT + OpenVINO), DSC/NSD are reported across multiple modalities alongside clearly reduced inference times, which is valuable for reproducibility dossiers across workstations and sites [67]. In edge/mobile deployments, packaging INT8 .tflite models with calibration metadata, hashes, and latency profiles supports traceability in production [25]. In embedded settings, preserving compilation fingerprints (library/ISA versions, bitstreams) and conducting clinical regression tests before re-registering the algorithm in the PACS/RIS workflow is critical for compliance and controlled change management [63,77].

To conclude applicability from an operational perspective, this subsection consolidates the non-algorithmic factors that determine whether a quantized model can be sustained in production: energy, deployment and maintenance cost, lifecycle management with safe re-quantization, and regulatory support (auditability and reproducibility). Table 9 summarizes, per article, the quantization/framework decision (e.g., real INT8, QAT, mixed precision), the key operational driver (energy, latency, ROM/RAM, versioning, re-synthesis), the evidence reported, and the practical implication (what facilitates or hinders clinical/edge adoption). This view enables rapid identification of which technique–environment combinations offer the best performance-per-watt, the lowest maintenance friction, and the traceability required for cross-site audits.

Table 9. Non-algorithmic factors (stacked bars). Upward arrows (↑) indicate increases/improvements in the reported metric, whereas downward arrows (↓) indicate decreases/reductions. Counts of studies that address vs. do not address four deployment concerns: energy consumption/HW efficiency, deployment and maintenance cost, model lifecycle/safe re-quantization, and regulatory support/auditability.

Art.	Quantization Decision/Framework	Highlighted Non-Algorithmic Factor	Reported Evidence/Impact	Practical Implication
[25]	Full INT8 PTQ (TFLite)	Energy and latency on the NPU	NPU supports only INT8; ~97% ↓ latency and ~75% ↓ size at comparable accuracy	Better performance per watt at the <i>point of care</i> ; straightforward distribution of <code>.tflite</code> artifacts
[79]	“True” INT8 PTQ (TensorRT)	Reproducible clinical deployment	2–4× ↓ memory and 2–3× ↓ latency with stable <i>mDSC</i>	Easy to audit (<i>engine-level</i> measurements); re-quantization/re-compilation without retraining
[67]	QAT + OpenVINO	2–3× ↓ runtimes and modality validity	DSC ≈ FP32; NSD ↑ by modality; 2–3× faster inference	Multimodal reproducibility dossier; updates enabled by brief QAT cycles
[55]	PTQ on MCU + CMSIS-NN	Power/ROM/RAM on MCUs	INT8 execution under tight RAM/Flash budgets; <i>on-device</i> measurements	Compile–flash–validate loop; asynchronous queues toward PACS
[15]	TFLite-Micro/STM32Cube.AI	Cost and maintenance on microcontrollers	Vendor <i>toolchain</i> ; <i>on-device</i> metrics	Low operational cost; re-quantization upon <i>firmware</i> changes
[77]	XpulpNN (RISC-V, INT8)	SoC energy efficiency	INT8 <i>kernels</i> accelerated via ISA extensions	Sustained low power; traceability via ISA/SDK versioning
[63]	QAT + FPGA (RTL/HLS)	Deterministic latency/maintenance	High <i>throughput</i> at low precision; requires re-synthesis	Excellent performance per watt; higher maintenance cost
[92]	<i>Low-precision</i> FP (FPGA)	Performance per watt and memory	FP16/INT16 reduce memory with stable latency	Useful when INT8 does not fit the numerical <i>pipeline</i>
[75]	<i>Quantization effects</i> (FPGA)	Guidance on bit limits	Operator-level <i>trade-off</i> analysis	Establishes minimum <i>bit-width</i> to preserve stability
[73]	AMED (mixed precision)	Lifecycle and portability	Guided per-layer allocation; improved Pareto frontier	Guided re-compilation across <i>sites</i> ; reduced drift across versions
[82]	QuantuneV2 (guided mixed precision)	Maintenance cost	Compilation based on local metrics	Safer updates with device-specific profiles
[32]	LQ-Nets (≤4 bits, QAT)	Stability at low <i>bit</i> widths	Learned quantizers stabilize deltas	Useful on non-uniform hardware; requires tuning data
[49]	HAWQ-V3 (dyadic)	Sensitivity-based allocation	Per-layer bit selection using a Hessian criterion	Less ad hoc testing; clear per-layer documentation

Figure 13 highlights that non-algorithmic factors have been scarcely considered in the medical-image quantization studies included in this review. Only a very small number of works explicitly address energy consumption and hardware efficiency, or the cost of deployment and maintenance, while mentions of model lifecycle management are even rarer. More concerning, none of the articles explicitly discusses regulatory support and auditability (e.g., compliance, reproducibility, or traceability for inspections), suggesting that although quantization proposals advance in terms of accuracy and computational

savings, their readiness for real-world healthcare settings—where energy, cost, maintenance, and regulation are decisive—remains limited.

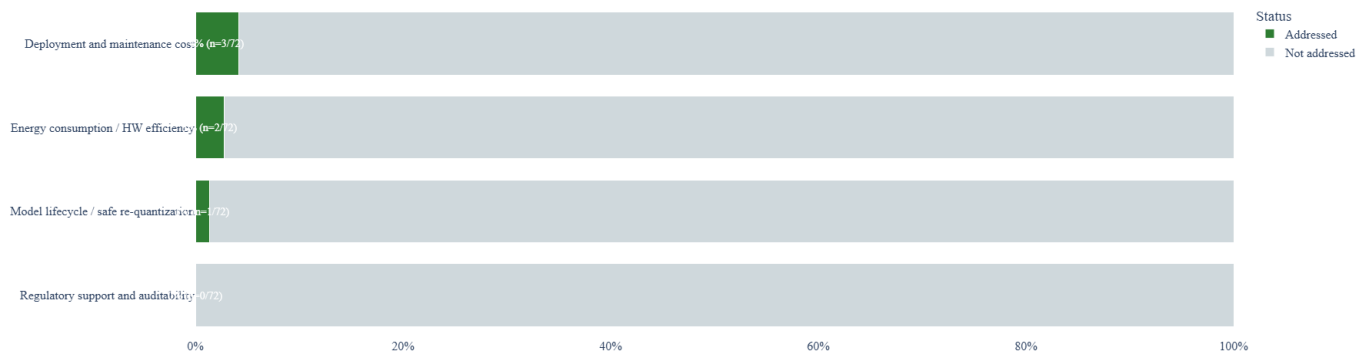


Figure 13. Non-algorithmic factors (stacked bars). Counts of studies that address vs. do not address four deployment concerns: energy consumption/HW efficiency, deployment and maintenance cost, model lifecycle/safe re-quantization, and regulatory support/auditability.

Applicability does not depend solely on the clinical metric: decisions such as using *real* INT8 (rather than fake quantization only), enabling NPU/ISA acceleration, and documenting layer-wise scales and bit-widths reduce energy, operating cost, and update risk, while improving regulatory traceability. In clinical settings, TensorRT/OpenVINO + INT8 provides a robust balance between accuracy and operational performance; in mobile/edge deployments, TFLite-INT8 is a prerequisite for competitive performance-per-watt; and in MCU/FPGA contexts, embedded toolchains deliver determinism and efficiency at the cost of stricter maintenance discipline. With QAT/mixed precision, lifecycle stability improves under data/hardware shifts, enabling safer re-quantization and cross-site auditability.

4.3.5. Adoption Best Practices and Decision Matrix (Context ↔ Quantization + Framework)

Layer-wise bit-width selection should be guided by clinical sensitivity and operator-level sensitivity [73]. In practice, it is advisable to retain higher precision in input/output layers, skip connections, and attention blocks, while reducing precision in intermediate layers [82]. This pattern stabilizes $\leq 8/4/2$ -bit configurations and shifts the Pareto frontier toward better accuracy–latency–memory trade-offs when mixed precision is used with importance-based assignment, MDP-style formulations, or Hessian-based criteria (e.g., AMED, QuantuneV2, LQ-Nets, HAWQ-V3) [49].

For PTQ, representative calibration is essential (ideally per-channel for weights/activations where appropriate) and must be verified on a *real* engine—not via fake quantization only—so that executable INT8 is realized and latency/memory are measured under the effective runtime conditions (TensorRT/OpenVINO/TFLite) [65,89].

When the task is high-criticality or the target is ≤ 4 bits, QAT is preferable, incorporating quantization nonlinearity during training (via STE) to reduce modality-dependent variability; for attention-heavy or sensitive 3D settings, QAT is often combined with mixed precision to improve stability (e.g., in multimodal MedSAM) [85,86].

Before deployment, multicenter *on-engine* validation is recommended, using sentinel sets, confidence intervals when feasible, and transparent end-to-end versioning (FP32 → PTQ/QAT checkpoint with scales/ZPs/bit-widths → INT8/ ≤ 4 -bit engine). This scheme supports auditability and rollback when protocols or equipment change.

The framework should be aligned with the target hardware: full INT8 is required to activate NPU/DSP with TFLite [25]; real INT8 on NVIDIA GPUs with TensorRT [79] or on Intel CPU/iGPU/VPU with OpenVINO [67]; and, on MCU/FPGA, the use of TFLite-Micro/STM32Cube.AI or RTL/HLS toolchains, always with on-device testing and formal

re-quantization/re-synthesis cycles [75]. When the hardware does not support integer arithmetic well, FP16/INT16 is a stable compromise on FPGA/MCU; if the target is a general-purpose CPU, 4.6-bit quantization optimized for vector units provides intermediate pathways with good efficiency while keeping accuracy within acceptable margins [62,72].

Figure 14 summarizes framework–quantization adoption patterns as a function of the deployment context. In the Edge/embedded panel, the highest concentration of studies is observed, dominated by PyTorch and TensorFlow, with recurrent use of PTQ, QAT, and mixed-precision or binary schemes, indicating that quantization is primarily explored with resource-constrained devices in mind. The Offline/unspecified panel also shows a diverse combination of frameworks, but without a clearly defined operational context, which limits extrapolation to real clinical workflows. By contrast, the Clinical/hospital and Cloud/research lab panels contain very few studies: only a small subset of works report explicit deployments in clinical environments or cloud infrastructures, and these almost always rely on PyTorch/TensorFlow with PTQ or QAT. Therefore, the figure suggests that the “best practices” implicit in the literature—and thus the decision matrix—are supported mainly by evidence from edge/embedded and offline scenarios, whereas clinical and cloud-based use cases remain comparatively rare and less well characterized.

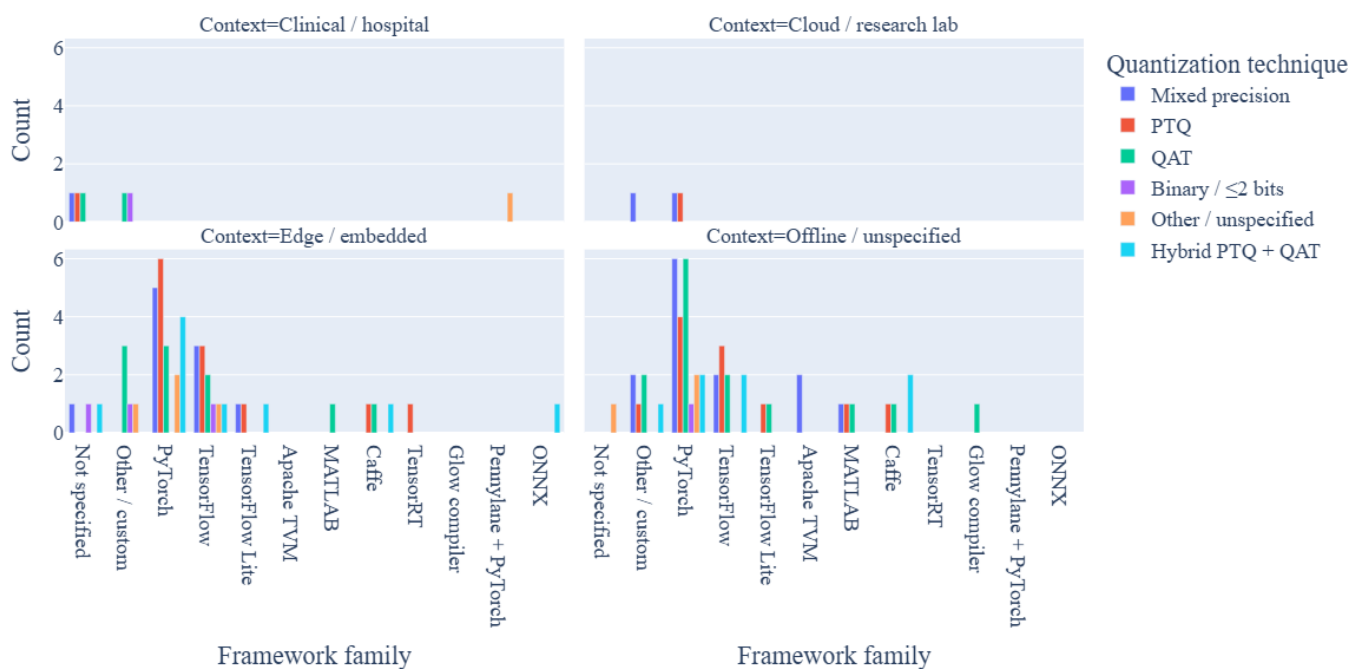


Figure 14. Framework–quantization adoption patterns by operational context (faceted bars). Counts of studies per framework family (x-axis) and quantization technique (color), stratified by context: clinical/hospital, cloud/research, edge/embedded, and offline/unspecified. Edge panels highlight TFLite/TensorFlow/PyTorch with INT8, mixed precision, and hybrid PTQ+QAT; clinical/cloud show sparser but more diversified use, including TensorRT, OpenVINO, and ONNX pipelines.

Table 10 presents a decision matrix that links the usage context (GPU/CPU clinical settings, mobile–edge with NPU, SBC/edge without NPU, MCU/FPGA, general-purpose CPU, and model families such as U-Net or ViT) to the recommended quantization technique (real PTQ INT8, QAT ≤ 4 -bit, and layer-wise mixed precision), the most suitable framework/runtime (e.g., PyTorch/TF $\rightarrow$ ONNX $\rightarrow$ TensorRT or OpenVINO, TFLite/TFLite-Micro, FPGA runtimes), and validation/operation guidelines (on-engine measurement of mDSC/NSD/latency, representative calibration, FP32 $\rightarrow$ PTQ/QAT $\rightarrow$ engine versioning, multicenter sentinel sets, and backend-specific profiling). From a practical standpoint, the matrix prioritizes real PTQ INT8 with TensorRT for clinical environments

equipped with NVIDIA GPUs, QAT (≤ 4 bits) plus mixed precision with OpenVINO on Intel CPU/iGPU/VPU platforms, full TFLite INT8 for mobile NPUs, and embedded toolchains (TFLite-Micro/STM32, RTL/HLS) on MCU/FPGA; all of which target the best trade-off among clinical accuracy, latency, memory footprint, and lifecycle maintainability.

Table 10. Adoption best practices and decision matrix (context $\leftrightarrow$ quantization + framework). Guides the choice of technique/bit-width, framework/runtime, and validation/operations per use context.

Articles	Use Context	Typical Task/Modality	Recommended Technique and Bit-Width	Framework/Runtime	Validation and Operations
[49,65,73,79]	Clinical (WS/server) with NVIDIA GPU	3D seg. (MRI/CT), 2D det.	“True” INT8 PTQ (<i>per-channel</i> where sensitive). Mixed precision if attention/fragile 3D blocks	PyTorch/TF $\rightarrow$ ONNX $\rightarrow$ TensorRT	Measure <i>mDSC</i> /latency <i>on-engine</i> ; version FP32 $\rightarrow$ PTQ $\rightarrow$ <i>engine</i> ; multicenter sentinels
[32,67,73,82]	Clinical (Intel CPU/iGPU/VPU)	Multimodal seg. (foundation models), 2D/3D	INT8 QAT/ ≤ 4 bits + <i>per-layer</i> mixed precision	PyTorch $\rightarrow$ ONNX/OpenVINO	Compare DSC/NSD by modality; 2–3 $\times$ runtimes; document the <i>bit-width map</i>
[15,25]	Mobile/ <i>edge</i> with NPU/DSP	2D classif. (US/X-ray), <i>triage</i>	Full INT8 PTQ (NPU requirement)	TensorFlow Lite (INT8)	$\sim 97\%$ $\downarrow$ latency, $\sim 75\%$ $\downarrow$ size; <i>accuracy</i> $\sim$ FP32; representative calibration
[56,73,82]	SBC/ <i>edge</i> without a dedicated NPU	Lightweight 2D/3D seg./det.	INT8 PTQ or FP16; mixed precision if feasible	ONNX Runtime/OpenVINO/TFLite	Backend profiling; select EP by HW; local sentinels
[15,55]	MCU (ARM Cortex-M)	Simple classif., lightweight det.	INT8 with optimized <i>kernels</i>	TFLite-Micro/STM32Cube.AI	Measure ROM/RAM/latency <i>on-device</i> ; PACS queues; re-quantize upon changes
[63,75,92]	FPGA	Deterministic real-time seg./det.	INT8/ ≤ 4 bits (QAT + mixed) or FP16/INT16	QAT $\rightarrow$ RTL/HLS, FPGA <i>runtimes</i>	Versioned <i>bitstreams</i> ; controlled re-synthesis; regression tests
[36,52,72]	General-purpose CPU (no GPU)	Optimized local inference	INT8 or optimized 4.6-bit vector quantization	ONNX Runtime/CPU compiler	Measure <i>speed-up</i> and accuracy Δ ; compilation <i>logs</i>
[67,90,97]	Transformers (ViT)	Multimodal seg./det.	PTQ “friendly” + QAT/mixed precision in attention blocks	OpenVINO/TensorRT (by HW)	Monitor modality sensitivity; per-block bit maps
[37,39,79]	2D/3D U-Net/FCN	Oncologic segmentation	INT8 baseline; mixed precision in <i>skip</i> /I/O paths	TensorRT/OpenVINO/TFLite	Dice/NSD <i>on-engine</i> ; sensitive-layer stability
[32,49,73,82]	Learned mixed precision	Various (edge/clinic)	Per-layer allocation (MDP/Hessian-guided)	AMED/QuantuneV2 (guided compilation)	Accuracy–latency Pareto frontier; <i>backend-level</i> reproducibility

4.3.6. Reporting Prevalence of Clinically and Operationally Critical Metrics

Across the 72 included studies, only 15/72 reported both sensitivity and specificity (20.8%, 95% CI 13.1–31.6%). Inference latency was available in 21/72 studies (29.2%, 95% CI 19.9–40.5%), while numeric power reporting was present in 14/72 (19.4%, 95% CI 12.0–30.0%). Energy per inference (reported or derivable from power and latency) was

available in 15/72 (20.8%, 95% CI 13.1–31.6%). Model size was reported more frequently (38/72, 52.8%, 95% CI 41.4–63.9%).

These results demonstrate that many clinically meaningful diagnostic accuracy metrics and deployment-critical efficiency measures are inconsistently reported, which directly constrains the feasibility of a statistically valid meta-analysis of clinical performance. Full extraction and unit standardization for latency, power, energy, and model size are provided in Table A1.

In studies that explicitly reported sensitivity (Se) and specificity (Sp) ($n = 15$), diagnostic error rates were estimated as $FPR = 1 - Sp$ and $FNR = 1 - Se$. Figure 15 shows the ROC space (Se vs. FPR) with points per study at the reported operating point. In order to provide an aggregate visualization of the Se–FPR trade-off, a descriptive SROC/HSROC-like summary curve was added, obtained by logit scale regression ($\text{logit}(Se)$ as a function of $\text{logit}(FPR)$), accompanied by a bootstrap-generated uncertainty band at the study level. This approach allows for a practical reading of the balance between detection (Se) and false positives (FPR), as well as the potential impact on the misdiagnosis rate.

However, it should be emphasized that this curve is descriptive and does not replace a formal HSROC/bivariate model, since most studies do not consistently report the information necessary for inferring diagnostic accuracy (e.g., TP/FP/FN/TN or, at least, positive/negative denominators that allow for estimating variances). Consequently, formal HSROC analysis is considered methodologically limited by the lack of standardized reporting; this limitation is interpreted as an evidence gap and is proposed as a recommendation for future publications.

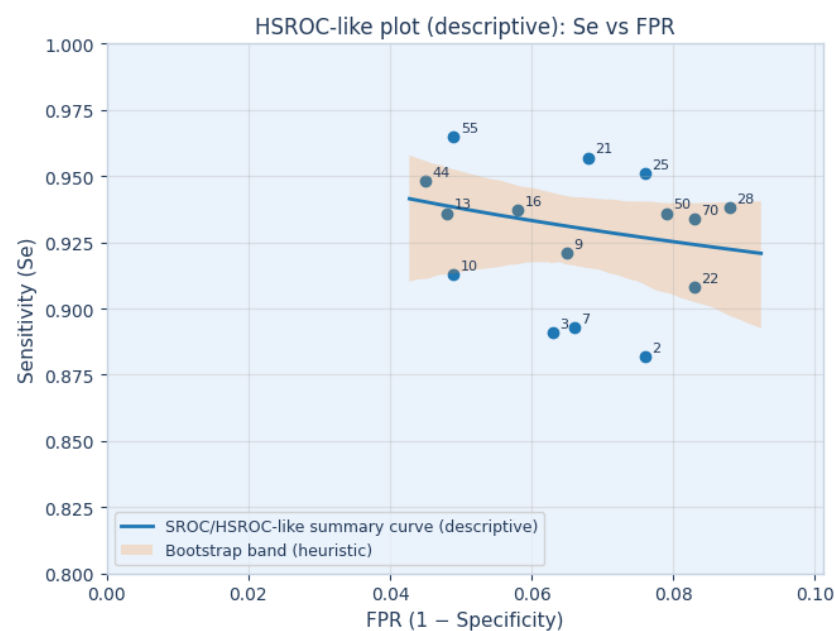


Figure 15. ROC-space plot of study-level sensitivity (Se) versus false positive rate ($FPR = 1 - \text{specificity}$) for studies reporting both metrics ($n = 15$), with a descriptive SROC/“HSROC-like” summary curve and bootstrap band; not a formal HSROC due to missing per-study 2×2 counts/denominators.

4.3.7. Operational Framework for the Secure Deployment of Quantized Models in Clinical Settings

In view of the fact that the reviewed evidence reports limited coverage of interoperability, regulatory compliance, and operational traceability aspects, we propose an operational framework to support the safe deployment of quantized models for medical imaging within real clinical workflows. This framework is structured around five components: (i) governance and change management, (ii) interoperability and integration with clinical systems,

(iii) MLOps and technical auditing, (iv) post-deployment validation and surveillance, and (v) security and risk management.

(i) Governance and change management

It is recommended to establish a governance model with clearly defined roles and responsibilities: a clinical owner (responsible for usage and acceptance criteria), an AI/ML team (training, quantization, and verification), IT/infrastructure staff (integration and availability), and quality/regulatory personnel (compliance and documentation). Deployment should operate under a formal change management policy that includes pre-deployment acceptance criteria, version control, approval by a technical–clinical committee, and a rollback plan in case of performance degradation or incidents.

(ii) Interoperability and integration with clinical systems

To ensure adoption and safety, integration should align with clinical data exchange and storage standards (e.g., DICOM for imaging and HL7/FHIR for clinical messaging, where applicable), such that the acquisition → preprocessing → inference → visualization/reporting pipeline is reproducible and interoperable with PACS/RIS/HIS systems. It is recommended to explicitly document integration points, input/output formats, target latencies, and exception-handling mechanisms (e.g., corrupted images, incomplete metadata, or non-standard DICOM series).

(iii) MLOps and technical auditing (end-to-end traceability)

Safe deployment requires that each prediction be traceable to the artifacts that generated it. Therefore, it is recommended to version and audit at least the following elements: (a) the FP32 baseline model, (b) the quantized model (PTQ/QAT/MPQ), (c) calibration datasets and parameters (including scales and zero-points, when applicable), (d) the inference engine and toolchain (e.g., TensorRT, OpenVINO, TFLite), (e) hardware configuration and software dependencies, and (f) preprocessing and postprocessing scripts. Structured logging (without exposing sensitive data), artifact integrity hashes, access control, and change logs should be enabled. In addition, automated testing (unit and integration tests) and a reproducible pipeline are recommended to allow controlled re-quantization when hardware, libraries, or data distributions change.

(iv) Validation, monitoring, and post-deployment surveillance

Beyond offline validation, a site-specific validation prior to clinical use is recommended, using risk-aligned metrics (e.g., sensitivity/specificity depending on the clinical context, or segmentation metrics such as DSC/NSD). After deployment, continuous monitoring should be implemented for performance (when delayed ground truth is available), distributional stability (drift), pipeline failure rates, and inference latencies. Safety thresholds and retraction policies should be defined: if significant degradation is detected, a rollback to a stable version should be triggered, or automated inference should be suspended.

(v) Security, privacy, and risk management

The framework should incorporate security controls (authentication, authorization, encryption in transit and at rest, and environment segregation) and align with institutional privacy policies. A risk management approach is recommended, including hazard identification (e.g., systematic errors due to out-of-distribution data), mitigation strategies (e.g., OOD detection and confidence-based rules), and documentation of intended use and limitations to reduce the risk of misuse.

4.3.8. Regulatory Pathways

Quantized medical imaging models are typically deployed as Software as a Medical Device (SaMD) or as software driving/influencing a medical device. Regulatory expectations

are not specific to “quantization” per se, but quantization changes the implemented model (numerical precision, calibration parameters, runtime engine) and therefore must be managed under change control, validation, and traceability requirements across jurisdictions.

United States (FDA): 510(k)/De Novo/PMA and Change Control for AI-enabled Devices

In the U.S., SaMD may be reviewed through 510(k) clearance, De Novo classification, or PMA, depending on intended use and risk. FDA guidance emphasizes that modifications to software (including AI-enabled device software functions) may require a new submission depending on the significance of the change and its risk impact.

To support iterative improvement while maintaining safety and effectiveness, the FDA has issued guidance on Predetermined Change Control Plans (PCCPs) for medical devices and AI-enabled device software functions. A PCCP describes what modifications are planned and how they will be assessed/validated, enabling pre-authorized updates within an agreed control plan.

Quantization-specific implication in PTQ introduces a calibration procedure and associated artifacts (e.g., calibration dataset specification, scaling/zero-point parameters, quantization configuration). From a regulatory quality perspective, these artifacts should be treated as version-controlled configuration items and linked to the released device version. If quantization settings or calibration data are altered, this constitutes a software modification that should be addressed through formal change control (and, where applicable, a PCCP with predefined validation evidence).

European Union (EU MDR): Conformity Assessment, Software Classification, and “Significant Change”

Under the EU MDR (Regulation (EU) 2017/745), software that provides information used for diagnosis, monitoring, prediction/prognosis, or therapeutic decisions generally falls within the MDR scope and must undergo conformity assessment leading to CE marking. MDCG guidance (including the updated MDCG 2019-11 on qualification and classification of software) provides practical criteria for determining whether software is medical device software and how it should be classified. In addition, the MDCG 2021-24 guidance supports consistent device classification.

Quantization-specific implications in quantization may change the executable model and its performance profile. Within MDR quality management and technical documentation, quantization should be documented as part of the software lifecycle and verification/validation evidence. If quantization changes are introduced post-market, manufacturers should evaluate whether the change is “significant” and therefore requires notified body involvement per the applicable conformity route, while maintaining traceability of model and calibration artifacts across versions.

China (NMPA/CMDE): Registration Review and AI-specific Evaluation Expectations

China’s regulatory ecosystem for AI-based medical device software has evolved rapidly, with guidance emphasizing algorithm performance evaluation and lifecycle documentation. Reviews of the Chinese framework report that NMPA guidance exists for the evaluation of AI medical devices and that approvals for AI medical device products have increased across domains (e.g., imaging decision support). Public-facing planning/updates by NMPA also indicate ongoing work on technical guidelines and change management for AI-powered medical devices.

Quantization-specific implication for quantized deployments, documentation should clearly identify the released algorithm version, runtime environment, and evaluation evidence under the claimed operating conditions. As with other jurisdictions, calibration and quantization configurations should be treated as controlled artifacts tied to a specific version; if they change, re-verification and re-validation evidence may be required.

Known Certified/Cleared Cases (Illustrative Examples)

- FDA (U.S.): the autonomous diabetic retinopathy screening system IDx-DR received FDA marketing authorization via the De Novo pathway (DEN180001).
- FDA (U.S.): multiple imaging AI tools have been cleared through 510(k); for example, Viz.ai-related clearances are documented in FDA summaries and company announcements.

5. Conclusions

This PRISMA 2020-compliant systematic review synthesizes evidence from 72 studies on neural network quantization for medical image analysis, covering modalities such as CT, MRI, X-ray, ultrasound, and histopathology. The literature shows a clear predominance of INT8 and PTQ/QAT as reference strategies, with small accuracy losses relative to FP32 and substantial gains in latency and memory footprint that enable deployment on edge/embedded platforms and standard clinical workstations. At the architectural level, U-Net/FCN models for segmentation and ResNet-like/CNN models for classification account for most implementations; mixed precision emerges as a practical solution to protect sensitive layers, whereas BNN/sub-8-bit regimes still exhibit notable accuracy variability.

Regarding clinical applicability, the evidence remains uneven: very few studies report PACS/RIS integration, version traceability, or workflow-stage latency budgets, and almost none quantify energy consumption, operational cost, or procedures for safe re-quantization after updates. Based on the framework–accelerator–bit-width mapping, we propose context-specific best practices (clinical, edge, research), highlighting TensorRT/OpenVINO/TFLite as mature backends and ONNX as a key enabler of portability.

PTQ is generally preferable when retraining is impractical (due to governance, time, cost, or limited labeled data), a representative calibration set and stable preprocessing are available, and the deployment stack supports true low-precision execution (e.g., verified INT8 kernels in TensorRT/OpenVINO/TFLite with known operator coverage), enabling rapid efficiency gains, provided site-specific validation confirms clinically sensitive metrics remain within acceptable margins. QAT is preferred for clinically critical use cases and challenging conditions—such as expected domain shift, aggressive bit-width targets (≤ 4 -bit), or numerically sensitive architectures (e.g., 3D and attention-heavy models)—because it better preserves decision boundaries under quantization noise, particularly when maintaining sensitivity at a fixed specificity is required. Mixed-precision is most suitable when strict latency/energy constraints must be met without uniformly reducing precision, or when runtime limitations necessitate hybrid execution, allowing sensitive layers to retain higher precision while still delivering meaningful efficiency improvements.

Quantization is currently a pragmatic pathway for translating deep learning models into clinical practice with improved efficiency and maintainability; however, external validity depends on closing three gaps: (i) standardized reporting of metrics and real-world latency measurements, (ii) interoperability and version governance supported by reproducible auditing, and (iii) systematic evaluation of energy and cost. Future work should adopt comparable protocols, release executable artifacts, and further explore sensitivity-guided mixed/layer-wise precision, alongside multicenter evaluations that strengthen clinical adoption in embedded medical systems.

Supplementary Materials: The following supporting information can be downloaded at: <https://www.mdpi.com/article/10.3390/technologies14010076/s1>. Reference [99] is cited in the supplementary materials.

Author Contributions: Conceptualization, E.F.R.-G. and V.E.R.-B.; methodology, L.F.G.-V.; software, E.F.R.-G.; validation, E.F.R.-G., L.F.G.-V., and V.E.R.-B.; formal analysis, L.F.G.-V.; investigation,

E.F.R.-G.; resources, L.F.G.-V.; data curation, E.F.R.-G.; writing—original draft preparation, E.F.R.-G.; writing—review and editing, E.F.R.-G. and L.F.G.-V.; visualization, E.F.R.-G. and L.F.G.-V.; supervision, V.E.R.-B.; project administration, V.E.R.-B.; funding acquisition, E.F.R.-G. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The study did not generate new primary clinical data. The data supporting the findings of this systematic review are derived from the included publications. The extraction matrix and supporting analysis files are available from the corresponding author upon reasonable request.

Acknowledgments: A la Universidad Politécnica Salesiana en conjunto con el programa de doctorado en Ciencias computacionales.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
DNN	Deep Neural Network
CNN	Convolutional Neural Network
FCN	Fully Convolutional Network
ViT	Vision Transformer
PTQ	Post-Training Quantization
QAT	Quantization-Aware Training
BNN	Binary Neural Network
MPQ	Mixed Precision Quantization
INT8/INT4/INT2	8/4/2-bit integer representation for weights and activations
FP32/FP16	32/16-bit floating point
STE	Straight-Through Estimator
Q/DQ	Quantize/dequantize nodes used for “fake quantization”
Scale/ZP	Per-tensor/per-channel scale and zero-point
AUC/AUROC	Area under the ROC curve
ROC	Receiver Operating Characteristic
DSC (Dice)	Dice Similarity Coefficient
IoU	Intersection over Union
NSD	Normalized Surface Dice
ONNX	Open Neural Network Exchange
TensorRT	NVIDIA inference engine for optimized deployment
OpenVINO	Intel toolkit for optimized inference (CPU/iGPU/VPU)
TFLite/TFLite-Micro	TensorFlow Lite for <i>edge</i> /MCU
GPU/CPU/MCU/FPGA	Graphics processing unit/central processing unit/microcontroller unit/field-programmable gate array
RTL/HLS	Register-Transfer Level/High-Level Synthesis
NPU/DSP	Neural Processing Unit/Digital Signal Processor
PACS/RIS	Picture Archiving and Communication System/Radiology Information System
DICOM/HL7	Standards for medical imaging and clinical data exchange
KD	Knowledge Distillation
MDP	Markov Decision Process

Appendix A

Table A1. Standardized deployment-efficiency metrics extracted from the included studies ($n = 72$).

Study (Citation)	Task	Hardware/ Accelerator	Framework/ Runtime	Latency (ms/Inference)	Model Size (MB)	Power (mW) + Source	Energy (mJ/Inference)	Data Quality	Notes
[84]	Segmentation	NVIDIA GeForce RTX; GeForce RTX 3090; RTX 3090	PyTorch	Missing data	65,536	Missing data	Missing data	Missing	size:reported_GB
[25]	Segmentation	ARM cores as CPUs; CPU	TFLite	Missing data	74.1	Missing data	Missing data	Missing	size:reported_MB
[45]	Detection	Not reported	TFLite	Missing data	41.6	Missing data	Missing data	Missing	size:reported_MB
[38]	Detection	CPU	Not reported	0.7	Missing data	Missing data	34,000	Derived	input = 40×40 ; latency:reported_ms; energy:reported_J
[65]	Segmentation	RTX 2080; GPU	PyTorch	Missing data	65,536	Missing data	Missing data	Missing	input = 256×256 ; size:reported_GB
[79]	Segmentation	NVIDIA Abstract; RTX 4090; GPU	TensorRT	2.62	23.11	1000.000 mW (unspecified)	2.62	Derived	input = 96×96 ; latency:reported_ms; power:reported_W; size:reported_MB
[85]	Segmentation	Tesla T4; GPU	PyTorch	Missing data	16,384	Missing data	Missing data	Missing	input = 128×128 ; size:reported_GB
[31]	Detection	Not reported	Not reported	Missing data	Missing data	Missing data	Missing data	Missing	nan
[66]	Detection	Not reported	Not reported	Missing data	Missing data	Missing data	Missing data	Missing	input = 224×224
[39]	Segmentation	CPU	OpenVINO	Missing data	Missing data	Missing data	Missing data	Missing	input = 512×512
[78]	Segmentation	GPU	TensorRT	Missing data	Missing data	Missing data	Missing data	Missing	input = 4000×4000
[46]	Classification	Not reported	Not reported	200	Missing data	Missing data	Missing data	Missing	input = 3218×10 ; latency:reported_ms
[86]	Segmentation	NVIDIA driver version 440; RTX 2080; arm model diversity and its en-	PyTorch	Missing data	5	Missing data	Missing data	Missing	input = 256×256 ; size:reported_MB
[55]	Detection	Nvidia TensorRT; ARM Cortex M-based microcon; CPU	TensorRT	30	0.25	Missing data	Missing data	Missing	input = 32×39 ; latency:reported_ms; size:reported_KB
[42]	Classification	FPGA	PyTorch	Missing data	500	Missing data	Missing data	Missing	size:reported_MB
[59]	Detection	arm Optimization for	Not reported	Missing data	Missing data	Missing data	126,000	Derived	energy:reported_J

Table A1. Cont.

Study (Citation)	Task	Hardware/ Accelerator	Framework/ Runtime	Latency (ms/Inference)	Model Size (MB)	Power (mW) + Source	Energy (mJ/Inference)	Data Quality	Notes
[32]	Detection	GPU	TensorFlow	Missing data	0.5	Missing data	Missing data	Missing	input = 107×106 ; size:reported_MB
[80]	Segmentation	Intel Core i7 processor with 16 GB of RAM and a 1 TB SSD; arm Sinica B; FPGA	Not reported	540	16,384	45,000.000 mW (unspecified)	24,300	Derived	input = 512×512 ; latency:reported_s; power:reported_W; size:reported_GB
[60]	Segmentation	NVIDIA DGX v100 machine was utilized to perform; jetson nano; arm Optimization	Not reported	Missing data	0.377	Missing data	Missing data	Missing	size:reported_KB
[67]	Segmentation	NVIDIA GeForce RTX 3090; GeForce RTX 3090; RTX 3090	OpenVINO	1180	163,840	Missing data	2.025×10^6	Derived	batch = 2; input = 6256×256 ; latency:reported_s; energy:reported_J; size:reported_GB
[61]	Segmentation	Not reported	TFLite	Missing data	19.6	Missing data	Missing data	Missing	input = 40×100 ; size:reported_MB
[33]	Segmentation	Nvidia Tesla M40; Tesla M40; GPU	Not reported	Missing data	2048	Missing data	Missing data	Missing	input = 000×100 ; size:reported_GB
[87]	Segmentation	Nvidia V100 Tensor Core GPUs and Intel i5 8th g; CPU	PyTorch	22	240	Missing data	Missing data	Missing	batch = 64; latency:reported_ms; size:reported_MB
[88]	Segmentation	RTX 3090	PyTorch	Missing data	358,400	Missing data	Missing data	Missing	size:reported_GB
[68]	Segmentation	CPU	Not reported	Missing data	Missing data	Missing data	Missing data	Missing	nan
[69]	Segmentation	FPGA	Not reported	Missing data	Missing data	Missing data	Missing data	Missing	nan
[70]	Segmentation	Nvidia RTX 2080Ti; RTX 2080; Intel Core i5-8300H CPU	PyTorch	Missing data	11,264	2000.000 mW (measured)	2000	Measured	input = 256×256 ; power:reported_W; energy:reported_J; size:reported_GB; power_source:measured
[71]	Segmentation	Not reported	PyTorch	Missing data	Missing data	Missing data	Missing data	Missing	input = 28×28
[72]	Segmentation	Nvidia GeForce TITAN X GPU; ARM and x86 CPUs; CPU	PyTorch	Missing data	Missing data	Missing data	Missing data	Missing	input = 32×32

Table A1. Cont.

Study (Citation)	Task	Hardware/ Accelerator	Framework/ Runtime	Latency (ms/Inference)	Model Size (MB)	Power (mW) + Source	Energy (mJ/Inference)	Data Quality	Notes
[15]	Segmentation	NVidia Quadro; Intel Core i7-8850H; ARM Cortex-M4F-based microcontrollers	TFLite	50	0.5	16.000 mW (TDP)	0.8	Estimated	input = 5259×10 ; latency:reported_ms; power:reported_W; size:reported_MB; power_source:TDP
[56]	Segmentation	GPU	Not reported	Missing data	Missing data	Missing data	Missing data	Missing	input = 96×96
[81]	Segmentation	FPGA	PyTorch	15.4	Missing data	Missing data	Missing data	Missing	input = 64×64 ; latency:reported_s
[73]	Segmentation	Nvidia Docs Hub; arm to the performance; FPGA	PyTorch	1	Missing data	Missing data	Missing data	Missing	latency:reported_ms
[89]	Segmentation	Tesla P100; GPU	PyTorch	Missing data	16,384	Missing data	Missing data	Missing	size:reported_GB
[82]	Classification	NVIDIA TensorRT; ARM NN; CPU	TensorRT	Missing data	13,1072	Missing data	2000	Derived	input = 27×86 ; energy:reported_J; size:reported_GB
[47]	Segmentation	Nvidia Turing GPU; GPU	Not reported	Missing data	170	Missing data	Missing data	Missing	input = 32×32 ; size:reported_MB
[57]	Segmentation	arm ptimization for global optimization; CPU	Not reported	Missing data	MissOng data	Missing data	1000	Derived	energy:reported_J
[62]	Segmentation	NVIDIA Xavier NX hardware; Jetson Nano and Raspberry Pi 4	TensorRT	75.56	Missing data	Missing data	Missing data	Missing	input = 32×32 ; latency:reported_ms
[40]	Detection	CPU	Not reported	Missing data	65,536	130,000.000 mW (avg)	Missing data	Missing	input = 112×112 ; power:reported_W; size:reported_GB; power_source:avg
[43]	Segmentation	FPGA	TensorFlow	Missing data	Missing data	Missing data	Missing data	Missing	input = 95×10
[90]	Segmentation	NVIDIA 3090 GPU; GPU	PyTorch	23.69	344	76,750.000 mW (unspecified)	1818.21	Derived	input = 16×16 ; latency:reported_ms; power:reported_W; size:reported_MB
[48]	Segmentation	NVIDIA TITAN Xp as the experimental platform	PyTorch	Missing data	12,288	Missing data	Missing data	Missing	input = 224×224 ; size:reported_GB
[58]	Classification	NVIDIA A100 GPUs; Apple M1 CPU; GPU	Not reported	Missing data	9	Missing data	Missing data	Missing	size:reported_MB

Table A1. Cont.

Study (Citation)	Task	Hardware/ Accelerator	Framework/ Runtime	Latency (ms/Inference)	Model Size (MB)	Power (mW) + Source	Energy (mJ/Inference)	Data Quality	Notes
[91]	Segmentation	arm Algorithm	Not reported	Missing data	Missing data	Missing data	Missing data	Missing	nan
[49]	Detection	Nvidia Turing Tensor Cores of T4 GPU for de-; RTX 6000; Tesla T4	PyTorch	1.06	1.9	Missing data	Missing data	Missing	batch = 128; input = 03×71 ; latency:reported_ms; size:reported_MB
[50]	Segmentation	Not reported	Not reported	Missing data	Missing data	1000.000 mW (unspecified)	Missing data	Missing	input = 28×28 ; power:reported_W
[63]	Segmentation	Arm Cortex-A53 processor and a Dual-core Arm Cortex-R5F; FPGA	TFLite	Missing data	Missing data	2050.000 mW (measured)	Missing data	Missing	input = 2048×1024 ; power:reported_W; power_source:measured
[44]	Segmentation	Not reported	TensorFlow	Missing data	Missing data	Missing data	2000	Derived	input = 217×512 ; energy:reported_J
[29]	Segmentation	NVIDIA GeForce GTX Titan X and the Intel; Jetson TK1 development board and; Intel Core i7 5930K as desktop processors	Not reported	50	240	12,800.000 mW (measured)	640	Derived	batch = 1; input = 5088×4096 ; latency:derived_from_fps; power:reported_W; size:reported_MB; power_source:measured
[74]	Segmentation	Not reported	Keras	Missing data	Missing data	Missing data	Missing data	Missing	input = 224×224
[34]	Classification	FPGA	Not reported	191.3509	0.0176	8200.000 mW (measured)	1569.08	Derived	input = 32×32 ; latency:derived_from_fps; power:reported_W; size:reported_KB; power_source:measured
[51]	Segmentation	Tesla P100; GPU	PyTorch	20,000	1	Missing data	Missing data	Missing	input = 976×976 ; latency:reported_s; size:reported_MB
[92]	Detection	Nvidia TITAN Xp GPU; Jetson Xavier NX; Intel Core I7-8700T CPU working under 2	TensorRT	Missing data	264	10,800.000 mW (unspecified)	Missing data	Missing	input = 224×224 ; power:reported_W; size:reported_MB
[35]	Segmentation	NVIDIA K40 machine; FPGA	Not reported	Missing data	Missing data	Missing data	Missing data	Missing	nan
[52]	Segmentation	CPU	Not reported	Missing data	Missing data	2000.000 mW (unspecified)	Missing data	Missing	input = 112×112 ; power:reported_W

Table A1. Cont.

Study (Citation)	Task	Hardware/ Accelerator	Framework/ Runtime	Latency (ms/Inference)	Model Size (MB)	Power (mW) + Source	Energy (mJ/Inference)	Data Quality	Notes
[93]	Segmentation	FPGA	PyTorch	16	0.0226	Missing data	Missing data	Missing	latency:reported_ms; size:reported_KB
[94]	Segmentation	NVIDIA GPU; Jetson Nano GPU; ARM CPU	TensorRT	Missing data	1	Missing data	Missing data	Missing	input = 256×256 ; size:reported_MB
[64]	Segmentation	GPU	Not reported	Missing data	Missing data	Missing data	Missing data	Missing	nan
[30]	Detection	Not reported	Not reported	Missing data	500	Missing data	Missing data	Missing	input = 35×49 ; size:reported_MB
[36]	Detection	ARM NEON; GPU	TFLite	33	1	Missing data	Missing data	Missing	latency:reported_ms; size:reported_MB
[95]	Segmentation	Nvidia cuDNN and Intel oneDNN; ARM Cortex-A72 core running at 1; GPU	PyTorch	700	500	12,800.000 mW (measured)	8960	Derived	input = 160×120 ; latency:reported_ms; power:reported_W; size:reported_MB; power_source:measured
[75]	Segmentation	Nvidia Jetson Nano; Jetson Nano; Arm Cortex-A53 MPCore up to	TensorFlow	14.298	Missing data	Missing data	Missing data	Missing	batch = 32; input = 1920×1080 ; latency:derived_from_fps
[37]	Segmentation	NVIDIA Tesla P100 GPUs; Tesla P100	Not reported	Missing data	Missing data	Missing data	Missing data	Missing	input = 64×80
[96]	Segmentation	FPGA	PyTorch	Missing data	Missing data	Missing data	Missing data	Missing	input = 32×32
[97]	Segmentation	NVIDIA 3090 GPU and an Intel Xeon 4214R CPU; GPU	PyTorch	517.9	Missing data	Missing data	Missing data	Missing	input = 16×16 ; latency:reported_ms
[98]	Segmentation	NVIDIA RTX 4090 GPUs with Pytorch; RTX 4090	PyTorch	Missing data	Missing data	Missing data	Missing data	Missing	input = 48×48
[53]	Segmentation	Not reported	Keras	Missing data	100	Missing data	Missing data	Missing	input = 416×416 ; size:reported_MB
[54]	Segmentation	NVIDIA Geforce 1070 Ti about; Intel Core i7-6700K CPU with an NVIDIA Geforce 1070 Ti about 4 h for each training; CPU	Not reported	Missing data	Missing data	Missing data	2000	Derived	energy:reported_J
[41]	Detection	arm Optimization	Not reported	Missing data	Missing data	Missing data	Missing data	Missing	nan
[76]	Segmentation	CPU	TensorFlow	Missing data	Missing data	Missing data	Missing data	Missing	input = 460×460

Table A1. *Cont.*

Study (Citation)	Task	Hardware/ Accelerator	Framework/ Runtime	Latency (ms/Inference)	Model Size (MB)	Power (mW) + Source	Energy (mJ/Inference)	Data Quality	Notes
[77]	Classification	ARM for 16-bit and 8-bit QNNs on; FPGA	Not reported	Missing data	0.5	1000.000 mW (avg)	Missing data	Missing	input = 16×16 ; power:reported_W; size:reported_KB; power_source:avg
[83]	Segmentation	Not reported	Not reported	Missing data	Missing data	Missing data	Missing data	Missing	input = 32×32

References

- Puttagunta, M.; Ravi, S. Medical image analysis based on deep learning approach. *Multimed. Tools Appl.* **2021**, *80*, 24365–24398. [[CrossRef](#)] [[PubMed](#)]
- Avanzo, M.; Stancanello, J.; Pirrone, G.; Drigo, A.; Retico, A. The evolution of artificial intelligence in medical imaging: From computer science to machine and deep learning. *Cancers* **2024**, *16*, 3702. [[CrossRef](#)]
- Mañay, E.D.; Martínez, D.; Chilibingua, M.D.; Criollo, V.D.; Rivera, E.F.; Toasa, R.M. Low-cost deep learning-based architecture for detecting cardiac arrhythmias in ECG signals. In Proceedings of the 2022 17th Iberian Conference on Information Systems and Technologies (CISTI), Madrid, Spain, 22–25 June 2022; pp. 1–6. [[CrossRef](#)]
- Loja-Morocho, A.; Rocano-Portoviejo, J.; Vega-Crespo, B.; Robles-Bykbaev, V.; Verhoeven, V. Intelligent system to provide support in the analysis of colposcopy images based on artificial vision and deep learning: A first approach for rural environments in Ecuador. In Proceedings of the International Conference on Information Technology & Systems, Cusco, Peru, 8–10 February 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 253–261. [[CrossRef](#)]
- Laçi, H.; Sevrani, K.; Iqbal, S. Deep learning approaches for classification tasks in medical X-ray, MRI, and ultrasound images: A scoping review. *BMC Med. Imaging* **2025**, *25*, 156. [[CrossRef](#)]
- Salehi, A.W.; Khan, S.; Gupta, G.; Alabdullah, B.I.; Almjally, A.; Alsolai, H.; Siddiqui, T.; Mellit, A. A study of CNN and transfer learning in medical imaging: Advantages, challenges, future scope. *Sustainability* **2023**, *15*, 5930. [[CrossRef](#)]
- Wahid, A.; Breslin, J.G.; Intizar, M.A. Prediction of machine failure in industry 4.0: A hybrid CNN-LSTM framework. *Appl. Sci.* **2022**, *12*, 4221. [[CrossRef](#)]
- Rivera, E.F.; Andaluz, V.H. Autonomous control of an electric vehicle by computer vision applied to the teaching–learning process. In *Perspectives and Trends in Education and Technology: Selected Papers from ICITED 2021*; Springer: Berlin/Heidelberg, Germany, 2021; pp. 419–432. [[CrossRef](#)]
- Fierro Saltos, W.R.; Fierro Saltos, F.E.; Elizabeth Alexandra, V.S.; Rivera Guzmán, E.F. Leveraging Artificial Intelligence for Sustainable Tutoring and Dropout Prevention in Higher Education: A Scoping Review on Digital Transformation. *Information* **2025**, *16*, 819. [[CrossRef](#)]
- Peña-Farfán, C.; Peralta-Bautista, F.; Panamá-Mazhenda, K.; Bravo-Buri, S.; Robles-Bykbaev, Y.; Robles-Bykbaev, V.; Lema-Condo, E. A Smart Mirror Based on Computer Vision and Deep Learning to Support the Learning of Sexual Violence Prevention and Self-care Health in Youth with Intellectual Disabilities. In Proceedings of the International Conference on Information Technology & Systems, Cusco, Peru, 8–10 February 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 273–282. [[CrossRef](#)]
- Lee, J.; Park, J.; Lee, Y. Towards Efficient Cancer Detection on Mobile Devices. *IEEE Access* **2025**, *13*, 34613–34626. [[CrossRef](#)]
- Rakka, M.; Fouda, M.E.; Khargonekar, P.; Kurdahi, F. A review of state-of-the-art mixed-precision neural network frameworks. *IEEE Trans. Pattern Anal. Mach. Intell.* **2024**, *46*, 7793–7812. [[CrossRef](#)]
- Rokh, B.; Azarpeyvand, A.; Khanteymoori, A. A comprehensive survey on model quantization for deep neural networks in image classification. *ACM Trans. Intell. Syst. Technol.* **2023**, *14*, 1–50. [[CrossRef](#)]
- Li, Z.; Li, H.; Meng, L. Model compression for deep neural networks: A survey. *Computers* **2023**, *12*, 60. [[CrossRef](#)]
- Novac, P.E.; Boukli Hacene, G.; Pegatoquet, A.; Miramond, B.; Gripon, V. Quantization and deployment of deep neural networks on microcontrollers. *Sensors* **2021**, *21*, 2984. [[CrossRef](#)]
- Tugwell, P.; Tovey, D. PRISMA 2020. *J. Clin. Epidemiol.* **2021**, *134*, A5–A6. [[CrossRef](#)]
- Wei, L.; Ma, Z.; Yang, C.; Yao, Q. Advances in the neural network quantization: A comprehensive review. *Appl. Sci.* **2024**, *14*, 7445. [[CrossRef](#)]
- Paddo, A.R.; Raju, R.T.; Gichoya, J.W.; Purkayastha, S. A Scoping Review on Quantization Methods for Medical Imaging AI. In Proceedings of the 2024 IEEE International Symposium on Biomedical Imaging (ISBI), Athens, Greece, 27–30 May 2024; pp. 1–5. [[CrossRef](#)]
- Kim, G.I.; Hwang, S.; Jang, B. Efficient compressing and tuning methods for large language models: A systematic literature review. *ACM Comput. Surv.* **2025**, *57*, 1–39. [[CrossRef](#)]
- Lin, Y.; Lei, M.; Niu, L. Optimization strategies in quantized neural networks: A review. In Proceedings of the 2019 International Conference on Data Mining Workshops (ICDMW), Beijing, China, 8–11 November 2019; pp. 385–390. [[CrossRef](#)]
- Sayed, R.; Azmi, H.; Shawkey, H.; Khalil, A.H.; Refky, M. A systematic literature review on binary neural networks. *IEEE Access* **2023**, *11*, 27546–27578. [[CrossRef](#)]
- Baur, D.; Kroboth, K.; Heyde, C.E.; Voelker, A. Convolutional neural networks in spinal magnetic resonance imaging: A systematic review. *World Neurosurg.* **2022**, *166*, 60–70. [[CrossRef](#)] [[PubMed](#)]
- Zhang, L.; Krestinskaya, O.; Fouda, M.E.; Eltawil, A.M.; Salama, K.N. Quantized convolutional neural networks: A hardware perspective. *Front. Electron.* **2025**, *6*, 1469802. [[CrossRef](#)]
- Xie, Y.; Oniga, S. A Comprehensive Review of Hardware Acceleration Techniques and Convolutional Neural Networks for EEG Signals. *Sensors* **2024**, *24*, 5813. [[CrossRef](#)]

25. Garifulla, M.; Shin, J.; Kim, C.; Kim, W.H.; Kim, H.J.; Kim, J.; Hong, S. A case study of quantizing convolutional neural networks for fast disease diagnosis on portable medical devices. *Sensors* **2021**, *22*, 219. [[CrossRef](#)] [[PubMed](#)]
26. Li, M.; Huang, Z.; Chen, L.; Ren, J.; Jiang, M.; Li, F.; Fu, J.; Gao, C. Contemporary advances in neural network quantization: A survey. In Proceedings of the 2024 International Joint Conference on Neural Networks (IJCNN), Yokohama, Japan, 30 June–5 July 2024; pp. 1–10. [[CrossRef](#)]
27. Dantas, P.V.; Sabino da Silva Jr, W.; Cordeiro, L.C.; Carvalho, C.B. A Comprehensive Review of Model Compression Techniques in Machine Learning. *Appl. Intell.* **2024**, *54*, 11804–11844. [[CrossRef](#)]
28. Prasad, N.S.; Sundar, S. Comprehensive Review on the Exploitation of Advanced Memory Optimization Strategies to Improve Performance for Convolutional and Spiking Neural Networks in Medical Imaging Using Hardware Accelerators. *IEEE Access* **2025**, *13*, 62449–62461. [[CrossRef](#)]
29. Han, S.; Mao, H.; Dally, W.J. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. *arXiv* **2015**, arXiv:1510.00149.
30. Ge, S.; Luo, Z.; Zhao, S.; Jin, X.; Zhang, X.Y. Compressing deep neural networks for efficient visual inference. In Proceedings of the 2017 IEEE International Conference on Multimedia and Expo (ICME), Hong Kong, China, 10–14 July 2017; pp. 667–672. [[CrossRef](#)]
31. Dietrich, O.; Raya, J.G.; Reiser, M.F. Magnetic resonance noise measurements and signal-quantization effects at very low noise levels. *Magn. Reson. Med.* **2008**, *60*, 1477–1487. [[CrossRef](#)]
32. Zhang, D.; Yang, J.; Ye, D.; Hua, G. LQ-Nets: Learned Quantization for Highly Accurate and Compact Deep Neural Networks. In Proceedings of the European Conference on Computer Vision (ECCV), Munich, Germany, 8–14 September 2018; pp. 373–390. [[CrossRef](#)]
33. Kong, B.; Sun, S.; Wang, X.; Song, Q.; Zhang, S. Invasive Cancer Detection Utilizing Compressed Convolutional Neural Network and Transfer Learning. In Proceedings of the Medical Image Computing and Computer Assisted Intervention–MICCAI 2018, Granada, Spain, 16–20 September 2018; Springer: Berlin/Heidelberg, Germany, 2018; pp. 156–164. [[CrossRef](#)]
34. Prost-Boucle, A.; Bourge, A.; Pétrot, F. High-Efficiency Convolutional Ternary Neural Networks with Custom Adder Trees and Weight Compression. *ACM Trans. Reconfig. Technol. Syst.* **2018**, *11*, 1–24. [[CrossRef](#)]
35. Cheng, Y.; Wang, D.; Zhou, P.; Zhang, T. Model compression and acceleration for deep neural networks: The principles, progress, and challenges. *IEEE Signal Process. Mag.* **2018**, *35*, 126–136. [[CrossRef](#)]
36. Jacob, B.; Kligys, S.; Chen, B.; Zhu, M.; Tang, M.; Howard, A.; Adam, H.; Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Salt Lake City, UT, USA, 18–23 June 2018; pp. 2704–2713. [[CrossRef](#)]
37. Xu, X.; Lu, Q.; Yang, L.; Hu, S.; Chen, D.; Hu, Y.; Shi, Y. Quantization of fully convolutional networks for accurate biomedical image segmentation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Salt Lake City, UT, USA, 18–23 June 2018; pp. 8300–8308. [[CrossRef](#)]
38. Huang, J.; Zhao, J.; Cai, W. Compressing Convolutional Neural Networks Using POD for the Reconstruction of Nonlinear Tomographic Absorption Spectroscopy. *Comput. Phys. Commun.* **2019**, *241*, 33–39. [[CrossRef](#)]
39. AskariHemmat, M.; Honari, S.; Rouhier, L.; Perone, C.S.; Cohen-Adad, J.; Savaria, Y.; David, J.P. U-net Fixed-Point Quantization for Medical Image Segmentation. In *Large-Scale Annotation of Biomedical Data and Expert Label Synthesis/Hardware Aware Learning for Medical Imaging and Computer Assisted Intervention*; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2019; pp. 115–124. [[CrossRef](#)]
40. Cheng, Y.; Wang, C.; Chen, H.B.; Yu, H. A Large-Scale In-Memory Computing for Deep Neural Network with Trained Quantization. *Integration* **2019**, *69*, 345–355. [[CrossRef](#)]
41. Kumari, G.V.; Rao, G.S.; Rao, B.P. New Artificial Neural Network Models for Bio Medical Image Compression: Bio Medical Image Compression. *Int. J. Appl. Metaheuristic Comput.* **2019**, *10*, 91–111. [[CrossRef](#)]
42. Long, X.; Zeng, X.; Ben, Z.; Zhou, D.; Zhang, M. A Novel Low-Bit Quantization Strategy for Compressing Deep Neural Networks. *Comput. Intell. Neurosci.* **2020**, *2020*, 7839064. [[CrossRef](#)] [[PubMed](#)]
43. Yang, Y.; Deng, L.; Wu, S.; Yan, T.; Xie, Y.; Li, G. Training High-Performance and Large-Scale Deep Neural Networks with Full 8-bit Integers. *Neural Netw.* **2020**, *125*, 70–82. [[CrossRef](#)] [[PubMed](#)]
44. Nalepa, J.; Antoniuk, M.; Myller, M.; Ribalta Lorenzo, P.; Marcinkiewicz, M. Towards resource-frugal deep convolutional neural networks for hyperspectral image segmentation. *Microprocess. Microsyst.* **2020**, *73*, 102994. [[CrossRef](#)]
45. Maiti, R.; Agarwal, P.; Kumar, R.R.; Bhat, A. Detection of Skin Cancer Using Neural Architecture Search with Model Quantization. In Proceedings of the 2021 5th International Conference on Intelligent Computing and Control Systems (ICICCS), Madurai, India, 6–8 May 2021; pp. 1807–1814. [[CrossRef](#)]
46. Zhang, Y.; Wu, Z.; Liu, S.; Guo, Z.; Chen, Q.; Gao, P.; Wang, P. A Quantized Convolutional Neural Network Implemented With Memristor for Image Denoising and Recognition. *Front. Neurosci.* **2021**, *15*, 717222. [[CrossRef](#)]

47. Chu, T.; Luo, Q.; Yang, J.; Huang, X. Mixed-Precision Quantized Neural Networks with Progressively Decreasing Bitwidth. *Pattern Recognit.* **2021**, *111*, 107647. [\[CrossRef\]](#)
48. Li, X.; Jiang, H.; Zhang, R.; Tian, F.; Huang, S.; Xu, D. Robustness-Aware 2-bit Quantization with Real-Time Performance for Neural Network. *Neurocomputing* **2021**, *455*, 12–22. [\[CrossRef\]](#)
49. Yao, Z.; Dong, Z.; Zheng, Z.; Gholami, A.; Yu, J.; Tan, E.; Wang, L.; Huang, Q.; Wang, Y.; Mahoney, M.; et al. Hawq-v3: Dyadic neural network quantization. In Proceedings of the International Conference on Machine Learning, PMLR, Virtual, 18–24 July 2021; pp. 11875–11886. [\[CrossRef\]](#)
50. Peric, Z.H.; Denic, B.D.; Savic, M.S.; Vucic, N.J.; Simic, N.B. Binary quantization analysis of neural networks weights on MNIST dataset. *Elektron. Elektrotehnika* **2021**, *27*, 55–61. [\[CrossRef\]](#)
51. Mishra, S.; Chen, D.Z.; Hu, X.S. Image Complexity Guided Network Compression for Biomedical Image Segmentation. *ACM J. Emerg. Technol. Comput. Syst.* **2021**, *17*, 43. [\[CrossRef\]](#)
52. Chen, W.; Qiu, H.; Zhuang, J.; Zhang, C.; Hu, Y.; Lu, Q.; Wang, T.; Shi, Y.; Huang, M.; Xu, X. Quantization of deep neural networks for accurate edge computing. *ACM J. Emerg. Technol. Comput. Syst.* **2021**, *17*, 1–11. [\[CrossRef\]](#)
53. Chin, H.H.; Tsay, R.S.; Wu, H.I. A high-performance adaptive quantization approach for edge CNN applications. *arXiv* **2021**, arXiv:2107.08382. [\[CrossRef\]](#)
54. Mirrashid, A.; Shirazi, A.A.B. RQCSNet: A deep learning approach to quantized compressed sensing of remote sensing images. *Expert Syst.* **2021**, *38*, e12755. [\[CrossRef\]](#)
55. Sailesh, M.; Selvakumar, K. A novel framework for deployment of CNN models using post-training quantization on microcontroller. *Microprocess. Microsyst.* **2022**, *94*, 104634. [\[CrossRef\]](#)
56. Ahamad, A.; Sun, C.C.; Kuo, W.K. Quantized Semantic Segmentation Deep Architecture for Deployment on an Edge Computing Device for Image Segmentation. *Electronics* **2022**, *11*, 3561. [\[CrossRef\]](#)
57. Nekooei, A.; Safari, S. Compression of Deep Neural Networks Based on Quantized Tensor Decomposition to Implement on Reconfigurable Hardware Platforms. *Neural Netw.* **2022**, *150*, 350–363. [\[CrossRef\]](#)
58. Tang, C.; Ouyang, K.; Wang, Z.; Zhu, Y.; Ji, W.; Wang, Y.; Zhu, W. Mixed-precision neural network quantization via learned layer-wise importance. In Proceedings of the European Conference on Computer Vision (ECCV), Tel Aviv, Israel, 23–27 October 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 259–275. [\[CrossRef\]](#)
59. Tmamna, J.; Ayed, E.B.; Ayed, M.B. Automatic Quantization of Convolutional Neural Networks Based on Enhanced Bare-Bones Particle Swarm Optimization for Chest X-Ray Image Classification. In *Computational Collective Intelligence; Lecture Notes in Computer Science*; Springer: Berlin/Heidelberg, Germany, 2023; pp. 125–137. [\[CrossRef\]](#)
60. Agarwal, M.; Gupta, S.K.; Biswas, K. Development of a compressed FCN architecture for semantic segmentation using Particle Swarm Optimization. *Neural Comput. Appl.* **2023**, *35*, 11833–11846. [\[CrossRef\]](#)
61. Aboutahoun, D.; Zewail, R.; Kimura, K.; Soliman, M.I. Cross-Domain Few-Shot Sparse-Quantization Aware Learning for Lymphoblast Detection in Blood Smear Images. In Proceedings of the Asian Conference on Pattern Recognition, Kitakyushu, Japan, 5–8 November 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 213–226. [\[CrossRef\]](#)
62. Jiang, B.; Chen, J.; Liu, Y. Single-shot Pruning and Quantization for Hardware-friendly Neural Network Acceleration. *Eng. Appl. Artif. Intell.* **2023**, *126*, 106816. [\[CrossRef\]](#)
63. Pistellato, M.; Bergamasco, F.; Bigaglia, G.; Gasparetto, A.; Albarelli, A.; Boschetti, M.; Passerone, R. Quantization-aware NN layers with high-throughput FPGA implementation for edge AI. *Sensors* **2023**, *23*, 4667. [\[CrossRef\]](#) [\[PubMed\]](#)
64. Lu, Q.; Jiang, W.; Xu, X.; Hu, J.; Shi, Y. Quantization through search: A novel scheme to quantize convolutional neural networks in finite weight space. In Proceedings of the 28th Asia and South Pacific Design Automation Conference (ASP-DAC), Tokyo, Japan, 16–19 January 2023; pp. 378–383. [\[CrossRef\]](#)
65. Zhang, R.; Chung, A.C.S. EfficientQ: An efficient and accurate post-training neural network quantization method for medical image segmentation. *Med. Image Anal.* **2024**, *97*, 103277. [\[CrossRef\]](#)
66. Patel, R.; Chaware, A. A Novel Hyperparameter Optimized Quantized MobileNet Architecture for Improving Diagnostic Accuracy in the Medical Image Classification. In Proceedings of the Role of AI in Bio-Medical Translations' Research for the Health Care Industry (AIBTR 2023), Communications in Computer and Information Science, Nagpur, India, 22–23 September 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 1–13. [\[CrossRef\]](#)
67. Lu, H.; Fu, Y.; Zhang, F.; Zhang, L. Efficient Quantization-Aware Training on Segment Anything Model in Medical Images and Its Deployment. In *Medical Image Segmentation Foundation Models. CVPR 2024 Challenge: Segment Anything in Medical Images on Laptop*; Springer: Berlin/Heidelberg, Germany, 2024; pp. 137–150. [\[CrossRef\]](#)
68. Bhatia, I.; Aarti; Ansarullah, S.I.; Amin, F.; Alabrah, A. Lightweight Advanced Deep Neural Network (DNN) Model for Early-Stage Lung Cancer Detection. *Diagnostics* **2024**, *14*, 2356. [\[CrossRef\]](#)
69. Zaccogna, F.G.; Lew, S.; Lutenberg, A. Flexible quantization for efficient convolutional neural networks. *Electronics* **2024**, *13*, 1923. [\[CrossRef\]](#)

70. Gong, J.; Chen, Q.; Zhu, W.; Wang, Z. A Convolutional Neural Network-Based Quantization Method for Block Compressed Sensing of Images. *Entropy* **2024**, *26*, 468. [\[CrossRef\]](#)
71. Matondo-Mvula, N.; Elleithy, K. Breast Cancer Detection with Quantvolutional Neural Networks. *Entropy* **2024**, *26*, 630. [\[CrossRef\]](#) [\[PubMed\]](#)
72. Trusov, A.; Limonova, E.; Nikolaev, D.; Arlazarov, V.V. 4.6-Bit Quantization for Fast and Accurate Neural Network Inference on CPUs. *Mathematics* **2024**, *12*, 651. [\[CrossRef\]](#)
73. Kimhi, M.; Rozen, T.; Mendelson, A.; Baskin, C. AMED: Automatic Mixed-Precision Quantization for Edge Devices. *Mathematics* **2024**, *12*, 1810. [\[CrossRef\]](#)
74. Singh, J.; Patel, P.; Ingole, B.S.; Inaganti, R.; Ramineni, V.; Krishnappa, M.S.; Patel, B.J. Advanced Computational Methods for Pelvic Bone Cancer Detection: Efficacy Comparison of Convolutional Neural Networks. In Proceedings of the 2024 IEEE 17th International Symposium on Embedded Multicore/Many-Core Systems-on-Chip (MCSoc), Kuala Lumpur, Malaysia, 16–19 December 2024; pp. 287–293. [\[CrossRef\]](#)
75. Pohl, D.; Vogel-Heuser, B.; Krüger, M.; Echtler, M. Quantization effects of deep neural networks on a FPGA platform. In Proceedings of the 2024 IEEE 7th International Conference on Industrial Cyber-Physical Systems (ICPS), St. Louis, MO, USA, 12–15 May 2024; pp. 1–8. [\[CrossRef\]](#)
76. Ansari, M.M.; Kumar, S.; Tariq, U.; Bin Heyat, M.B.; Akhtar, F.; Bin Hayat, M.A.; Sayeed, E.; Parveen, S.; Pomary, D. Evaluating CNN architectures and hyperparameter tuning for enhanced lung cancer detection using transfer learning. *J. Electr. Comput. Eng.* **2024**, *2024*, 3790617. [\[CrossRef\]](#)
77. Garofalo, A.; Tagliavini, G.; Conti, F.; Rossi, D.; Benini, L. XpulpNN: Accelerating Quantized Neural Networks on RISC-V Processors Through ISA Extensions. In Proceedings of the 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE), Grenoble, France, 9–13 March 2020. [\[CrossRef\]](#)
78. Jiang, Z.; Li, C.; Qu, T.; He, C.; Wang, D. MSQuant: Efficient Post-Training Quantization for Object Detection via Migration Scale Search. *Electronics* **2025**, *14*, 504. [\[CrossRef\]](#)
79. Qu, C.; Zhao, R.; Yu, Y.; Liu, B.; Yao, T.; Zhu, J.; Landman, B.A.; Tang, Y.; Huo, Y. Post-Training Quantization for 3D Medical Image Segmentation: A Practical Study on Real Inference Engines. *arXiv* **2025**, arXiv:2501.17343. [\[CrossRef\]](#)
80. Li, S.; Lu, J.; Hu, Y.; Mattos, L.S.; Li, Z. Towards scalable medical image compression using hybrid model analysis. *J. Big Data* **2025**, *12*, 45. [\[CrossRef\]](#)
81. Tian, L.; Wang, P. An Effective Mixed-Precision Quantization Method for Joint Image Deblurring and Edge Detection. *Electronics* **2025**, *14*, 1767. [\[CrossRef\]](#)
82. Kim, J.; Lee, J.; Kwon, Y.; Kim, D. QuantuneV2: Compiler-Based Local Metric-Driven Mixed Precision Quantization for Practical Embedded AI Applications. *Future Gener. Comput. Syst.* **2025**, *166*, 107718. [\[CrossRef\]](#)
83. Khan, N.A.; Rafat, A.M.S. Optimizing Deep Learning Models for Resource-Constrained Environments With Cluster-Quantized Knowledge Distillation. *Eng. Rep.* **2025**, *7*, e70187. [\[CrossRef\]](#)
84. Chen, D.; Lin, K.; Deng, Q. GAP: A Group-Based Automatic Pruning Algorithm via Convolution Kernel Fusion. *Neurocomputing* **2024**, *609*, 128488. [\[CrossRef\]](#)
85. Boerkamp, C.; Thomas, A.J. QuantU-Net: Efficient Wearable Medical Imaging Using Bitwidth as a Trainable Parameter. *arXiv* **2025**, arXiv:2503.08719.
86. Zhang, R.; Chung, A.C.S. MedQ: Lossless ultra-low-bit neural network quantization for medical image segmentation. *Med. Image Anal.* **2021**, *73*, 102200. [\[CrossRef\]](#)
87. Naveen, S.; Kounte, M.R. Optimized Convolutional Neural Network at the IoT Edge for Image Detection Using Pruning and Quantization. *Multimed. Tools Appl.* **2025**, *84*, 5435–5455. [\[CrossRef\]](#)
88. Zhang, L.; Lou, Z.; Ying, Y.; Yang, C.; Zhou, H. Efficient Fine-Tuning of Large Language Models via a Low-Rank Gradient Estimator. *Appl. Sci.* **2024**, *15*, 82. [\[CrossRef\]](#)
89. Abdel-Salam, R.; Abdel-Gawad, A.H.; Wassal, A.G. VLCQ: Post-training Quantization for Deep Neural Networks Using Variable Length Coding. *Future Gener. Comput. Syst.* **2025**, *166*, 107654. [\[CrossRef\]](#)
90. Jiang, Y.; Sun, N.; Xie, X.; Yang, F.; Li, T. ADFQ-ViT: Activation-Distribution-Friendly Post-Training Quantization for Vision Transformers. *Neural Netw.* **2025**, *186*, 107289. [\[CrossRef\]](#)
91. Thanikachalam, V.; Kabilan, K.; Erramchetty, S.K. Optimized deep CNN for detection and classification of diabetic retinopathy and diabetic macular edema. *BMC Med. Imaging* **2024**, *24*, 227. [\[CrossRef\]](#)
92. Wu, C.; Wang, M.; Chu, X.; Wang, K.; He, L. Low-precision floating-point arithmetic for high-performance FPGA-based CNN acceleration. *ACM Trans. Reconfig. Technol. Syst.* **2021**, *15*, 1–21. [\[CrossRef\]](#)
93. Daghero, F.; Burrello, A.; Xie, C.; Castellano, M.; Gandolfi, L.; Calimera, A.; Macii, E.; Poncino, M.; Pagliari, D.J. Human activity recognition on microcontrollers with quantized and adaptive deep neural networks. *ACM Trans. Embed. Comput. Syst.* **2022**, *21*, 1–28. [\[CrossRef\]](#)

94. Roth, W.; Schindler, G.; Klein, B.; Peharz, R.; Tschitschek, S.; Fröning, H.; Pernkopf, F.; Ghahramani, Z. Resource-efficient neural networks for embedded systems. *J. Mach. Learn. Res.* **2024**, *25*, 1–51. .. [[CrossRef](#)]
95. Bhalgaonkar, S.; Munot, M.; Anuse, A. Model compression of deep neural network architectures for visual pattern recognition: Current status and future directions. *Comput. Electr. Eng.* **2024**, *116*, 109180. [[CrossRef](#)]
96. Wang, Y.; Yang, Y.; Sun, F.; Yao, A. Sub-bit neural networks: Learning to compress and accelerate binary neural networks. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), Montreal, QC, Canada, 11–17 October 2021; pp. 5360–5369. [[CrossRef](#)]
97. Zhong, Y.; Huang, Y.; Hu, J.; Zhang, Y.; Ji, R. Towards accurate post-training quantization of vision transformers via error reduction. *IEEE Trans. Pattern Anal. Mach. Intell.* **2025**, *47*, 2676–2692. [[CrossRef](#)]
98. Wang, M.; Zhang, Z.; Li, F.; Xu, K.; Miao, K.; Wang, M. Thinking in Granularity: Dynamic Quantization for Image Super-Resolution by Intriguing Multi-Granularity Clues. In Proceedings of the AAAI Conference on Artificial Intelligence, Philadelphia, PA, USA, 25 February–4 March 2025; Volume 39, pp. 7826–7834. [[CrossRef](#)]
99. Page, M.J.; McKenzie, J.E.; Bossuyt, P.M.; Boutron, I.; Hoffmann, T.C.; Mulrow, C.D.; Shamseer, L.; Tetzlaff, J.M.; Akl, E.A.; Brennan, S.E.; et al. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ* **2021**, *372*, n71. [[CrossRef](#)] [[PubMed](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.